Einheitliche Shared Libraries (`.sl`) Wiederverwendung des Ladeverfahrens

Von den Ladeverfahren in Gen-DB und `subgraph-sat-solver-econ` über das Proposal zur Referenzimplementierung `sharedlib`: formales Modell, Beweise, Prüfung und Ausblick

Stephan Epp

M.Sc.

21. Oktober 2026

Zusammenfassung

Die Arbeit [1] katalogisiert achtzehn Wiederverwendungen des Subgraph Algorithmus und enthält ein Kapitel über das Laden der C/C++-Bibliothek in Gen-DB und `subgraph-sat-solver-econ`. Dieses Kapitel mündet in die Idee einer einheitlichen, betriebssystemunabhängigen Shared Library (`.sl`) mit standardisiertem Ladeverfahren. Die vorliegende Arbeit nimmt diese Idee als Ausgangspunkt, führt sie anhand des Proposals und der Referenzimplementierung `sharedlib` (Python-Paket `sl`) aus und belegt alle Aussagen formal. Sie entwickelt ein mathematisches Modell aus Dateisystem, Umgebung, Suchordnung, Container, Versionsbereichen und einer Zustandsmaschine des Loaders und beweist darin über fünfzig Sätze, Lemmata und Korollare. Dazu gehören: der Satz vom ersten Treffer und die Unabhängigkeit vom Arbeitsverzeichnis, eine exakte Charakterisierung der Rundreise der `SL_PATH`-Kodierung, die Konvexität von Versionsbereichen, die Rundreise und Injektivität des Containers, die Terminierung des Ladens mit Schranke $|E_r| + 1$ Aufrufen, die Korrektheit und Vollständigkeit der Zykluserkennung, die Invariante der Referenzzählung, die Atomarität (Rollback) fehlgeschlagener Öffnungen, die Ordnung von Initialisierung und Finalisierung, die Fehlerkausalität, die Parametrizität des Kerns bezüglich des Plattformadapters sowie die Nicht-Vereinheitlichbarkeit von Maschinencode zwischen Aufrufkonventionen. Die Aussagen werden durch eigene Läufe gestützt: $300\,000$ Zufallsfälle für die Pfadkodierung, $100\,000$ Signaturen, $2\,000$ Container-Rundreisen, Bitkipp-Tests und $3\,000$ modellbasierte Loader-Läufe mit $22\,559$ Invariantenprüfungen ohne Verletzung. Eine Messung zeigt, dass das Öffnen einer `.sl`-Datei in drei Läufen $0{,}16$ bis $0{,}39\,\mathrm{ms}$ plus $1{,}0$ bis $3{,}4\,\mathrm{ms/MiB}$ kostet, der einzelne Aufruf einer gebundenen Funktion aber keinen messbaren Mehraufwand hat. Fünfzehn Befunde, darunter ein Fehler der Pfadkodierung, ein Widerspruch zum Proposal bei der Sperre während der Initialisierung, die unaufgelöste native Symbolabhängigkeit zwischen Bibliotheken und ein nicht authentifizierter Metadatenkopf, werden mit Beweis oder Reproduktion und Verbesserung belegt. Die Arbeit schließt mit einer Zusammenfassung und einem sehr ausführlichen Ausblick bis zu mechanisierten Beweisen, einem Loader-Kern in C oder Rust, einem eigenen Abbilder für ELF, PE und Mach-O, SAT-basierter Abhängigkeitsauflösung und der Standardisierung.

Inhaltsverzeichnis

I Ausgangslage **1**

1 Einführung **2**
1.1 Fragestellung . . . 3
1.2 Arten von Aussagen in dieser Arbeit . . . 3
1.3 Beiträge . . . 4
1.4 Aufbau der Arbeit . . . 4

2 Formale Grundlagen **6**
2.1 Zeichenketten, Listen und endliche Abbildungen . . . 6
2.2 Dateisystem und Umgebung . . . 7
2.3 Gerichtete Graphen und Tiefensuche . . . 7
2.4 Versionen . . . 8
2.5 Ergebnisse, Fehler und Ereignisspuren . . . 8
2.6 Annahme zur Hashfunktion . . . 9
2.7 Symbolübersicht . . . 9

3 Projektlandschaft und Wiederverwendungsformen **10**
3.1 Die Repositories . . . 10
3.2 Was in `sharedlib` steckt . . . 11
3.3 Abhängigkeiten . . . 12
3.4 Befund aus den zwei Projekten . . . 12
3.5 Eine Systematik der Wiederverwendung . . . 12

II Das Problem **15**

4 Der Ist-Zustand des Ladens **16**
4.1 Unterschiede der drei Systeme . . . 16
4.2 Fünf Fehlerklassen . . . 16
4.3 Ein Kostenmodell . . . 16
4.4 Was Plattformcode ist und was nicht . . . 18

5 Grenzen der Vereinheitlichung **20**
5.1 Ziele und Nichtziele . . . 20
5.2 Aufrufkonventionen . . . 21
5.3 Datenmodelle . . . 22
5.4 Was bleibt . . . 22
5.5 Die Einwände des Proposals . . . 23

III Das Konzept: Modell und Beweise **25**

6 Das formale Modell der Ladeprozedur **26**
6.1 Konfiguration und Adapter . . . 26
6.2 Zustand und Operationen . . . 27
6.3 Annahmen . . . 27
6.4 Der Ablauf von `sl_open` . . . 29
6.5 Zuordnung von Modell und Quelltext . . . 29

7 Namensauflösung und Suchordnung **31**
7.1 Namensauflösung . . . 31
7.2 Die Kodierung von `SL_PATH` . . . 32
7.3 Die Kandidatenliste . . . 34

8 Container, Metadaten und Versionsbereiche **36**
8.1 Metadaten . . . 36
8.2 Versionsbereiche . . . 37
8.3 Das Containerformat . . . 38
8.4 Der Satz zu Kanten und Versionen . . . 41

9 Der Loader-Kern **42**
9.1 Der aufgelöste Graph und die Terminierung . . . 42
9.2 Zykluserkennung . . . 43
9.3 Die Invariante der Referenzzählung . . . 43
9.4 Atomarität: gescheitertes Öffnen hinterlässt nichts . . . 46
9.5 Lebenszyklus: Initialisierung und Finalisierung . . . 47
9.6 Fehler: Priorität und Ursache . . . 48
9.7 Export . . . 49
9.8 Identität: wann ist dieselbe Bibliothek dieselbe Instanz? . . . 50
9.9 Nebenläufigkeit . . . 51

10 Plattformadapter, Binding und Schnittstelle **53**
10.1 Die drei Adapter . . . 53
10.2 Der Kern ist adapterparametrisch . . . 54
10.3 Wie weit der Kern plattformfrei ist . . . 54
10.4 Das Abbilden: Entpacken und Laden . . . 55
10.5 Die C-artige Schnittstelle und das Python-Binding . . . 56

11 Codegenerierung und Build **58**
11.1 Was erzeugt wird . . . 58
11.2 Der Signaturparser . . . 59
11.3 Typbreiten im Binding . . . 61
11.4 Build-Matrix, Fat-Container und CMake . . . 61

IV Prüfung und Anwendung **63**

12 Anwendung auf Gen-DB und Subgraph-SAT-Solver **64**
12.1 Vorher und nachher . . . 64

12.2 Wann bleibt das Verhalten gleich? . . . 64
12.3 Prozess-Pool und Speicher . . . 66
12.4 Vergleich der zwei Ladeverfahren . . . 66
12.5 Ein Migrationsplan mit Abnahmekriterien . . . 66

13 Prüfung und Messung **68**
13.1 Umgebung und Methode . . . 68
13.2 Die Testsuite des Repositories . . . 68
13.3 Eigenschaftstests . . . 69
13.4 Reproduktionen der Befunde . . . 69
13.5 Kosten des Öffnens . . . 69
13.6 Grenzen der Prüfung . . . 72

14 Befunde **73**
14.1 Pfadkodierung und Suche (**B1**, **B2**, **B4**) . . . 73
14.2 Versionen (**B3**) . . . 75
14.3 Nebenläufigkeit (**B5**, **B6**) . . . 75
14.4 Native Abhängigkeiten (**B7**) . . . 75
14.5 Instanzen, Authentizität, Aufräumen (**B8** bis **B12**) . . . 75
14.6 Codegenerator und Binding (**B13**, **B14**) . . . 76
14.7 Legacy-Modus (**B15**) . . . 76
14.8 Gewichtung . . . 77

15 Konformität von Proposal und Implementierung **78**
15.1 Der Legacy-Modus . . . 78
15.2 Erzeugen und Aufrufen . . . 78

16 Zusammenfassung **80**

17 Ausblick **82**
17.1 Überblick und Reihenfolge . . . 82
17.2 Kurzfristig: Befunde beheben und Prüfung vervollständigen . . . 82
17.2.1 Eine Änderung je Befund, ein Test je Änderung . . . 82
17.2.2 Die ausstehende Prüfung . . . 82
17.3 Mechanisierte Beweise . . . 84
17.3.1 Zustandsmaschine in TLA$^+$ oder Alloy . . . 84
17.3.2 Interaktiver Beweisassistent . . . 84
17.3.3 Verfeinerung zum Quelltext . . . 84
17.3.4 Zufalls- und Fuzztests des Containers . . . 85
17.4 Spezifikation und Konformitätssuite . . . 85
17.4.1 Spezifikationstext . . . 85
17.4.2 Konformitätssuite aus dem Modell . . . 85
17.4.3 Zu klärende Entwurfsfragen . . . 86
17.5 Laden: Abbilder, native Abhängigkeiten und Entladen . . . 86
17.5.1 Native Abhängigkeiten lösen (**B7**) . . . 86
17.5.2 Inhaltsadressierte gemeinsame Ablage . . . 87
17.5.3 Ohne Extraktion: Speicherdateien und Streaming . . . 87
17.5.4 Eigener Abbilder für ELF, PE und Mach-O . . . 87
17.5.5 Threads, Entladen und Ausnahmen . . . 88

17.6 Nebenläufigkeit . . . 88
17.6.1 Das Protokoll für **B5** . . . 88
17.6.2 Feinere Sperren . . . 89
17.7 Sicherheit . . . 89
17.7.1 Der Authentifizierte Container (**B10**) . . . 89
17.7.2 Der gehärtete Modus . . . 90
17.7.3 Isolation . . . 90
17.7.4 Lieferkette und Reproduzierbarkeit . . . 90
17.8 Abhängigkeitsauflösung: Resolver und SAT . . . 90
17.8.1 Von der Prüfung zur Wahl . . . 90
17.8.2 Kodierung nach SAT . . . 91
17.8.3 Anschluss an die Subgraph-SAT-Kette . . . 92
17.8.4 Weitere Fragen des Resolvers . . . 92
17.9 Kern und Bindungen . . . 92
17.9.1 Loader-Kern in C oder Rust . . . 92
17.9.2 Weitere Sprachen . . . 93
17.9.3 C++-ABI . . . 93
17.9.4 Einbettung in Laufzeiten . . . 93
17.10 Build, Paketverwaltung und Betrieb . . . 94
17.10.1 Build-Matrix . . . 94
17.10.2 Anschluss an Paketverwaltungen . . . 94
17.10.3 Beobachtbarkeit . . . 94
17.11 Anwendungen und Systematik . . . 94
17.11.1 Gen-DB und der Solver . . . 94
17.11.2 Neue Wiederverwendungen . . . 95
17.11.3 Die Systematik der Vorarbeit . . . 95
17.12 Standardisierung und Organisation . . . 95
17.13 Bedrohungen der Gültigkeit . . . 96
17.14 Konkreter nächster Schritt . . . 96

A Beweisindex **97**

B Prüfskripte **99**

Teil I

Ausgangslage

Kapitel 1

Einführung

Die Vorarbeit [1] zeigt, dass ein Algorithmus, der an mehreren Stellen gebraucht wird, auf zwei Arten verankert werden kann: Jedes Projekt bringt seine eigene Implementierung mit, oder eine einzige Implementierung wird über eine schmale Schnittstelle von allen benutzt. Sie wendet das auf den Subgraph Algorithmus [2] an und führt achtzehn Wiederverwendungen **W1** bis **W18** auf. Ein Kapitel dieser Arbeit behandelt einen Sonderfall, der den Algorithmus selbst nicht berührt, aber jede der Wiederverwendungen betrifft, die eine C/C++-Bibliothek aus Python benutzt: das *Laden* dieser Bibliothek. Gen-DB und `subgraph-sat-solver-econ` laden ihre Bibliothek über `ctypes`, und beide enthalten dafür eigenen Plattformcode: Dateinamen je Betriebssystem, Suchpfade für abhängige Bibliotheken, Exportmakros und eine Sonderbehandlung der C++-Laufzeit. Keine dieser Stellen enthält Fachlogik.

Aus diesem Befund entstand das Proposal [3]: eine einheitliche Shared Library mit der Endung `.sl`, einer standardisierten Ladeprozedur und einem festgelegten Fehlermodell. Das Repository `github.com/naphets86/sharedlib` enthält inzwischen eine Referenzimplementierung dieser Schichten in Python (Paket `sl`, rund 1 170 Zeilen Quelltext, keine Laufzeitabhängigkeiten) mit Container, Loader-Kern, Plattformadaptern, Codegenerierung, Build-Orchestrierung und Kommandozeile. Das Ladeverfahren ist damit selbst zu einem Projekt geworden, das wiederverwendet wird, und die Frage der Vorarbeit kehrt sich um: Nicht mehr *welcher Algorithmus* wird wiederverwendet, sondern *welche Eigenschaften des Ladens* sich so festlegen lassen, dass alle Projekte sich darauf verlassen können.

Diese Arbeit beantwortet die Frage formal. Sie baut ein mathematisches Modell des Proposals, beweist darin die Eigenschaften, die das Proposal verspricht, prüft sie gegen den Quelltext von `sharedlib` und belegt die Ergebnisse durch eigene Läufe. Wo die Implementierung vom Proposal abweicht, wird die Abweichung bewiesen oder reproduziert und eine Verbesserung vorgeschlagen.

Bemerkung 1.1 (Verhältnis zur Vorarbeit)**.** Das Kapitel zum Laden der Bibliothek in [1] (dort Kapitel 13) wird hier wiederverwendet und erweitert: Fragestellung, Begriffe, die Systematik der Wiederverwendungsformen (Tabelle 3.4) und der Gedanke einer einheitlichen `.sl`-Datei bleiben, die Ausführung dagegen ist neu. Der Wortlaut jenes Kapitels (Datei `p3.tex`

der Vorarbeit) lag bei der Erstellung dieser Fassung nicht vor; die Inhaltsangabe aus Einleitung und Zusammenfassung der Vorarbeit und das vollständige Proposal waren die Grundlage. **[Abgleich mit dem Wortlaut von Kapitel 13 der Vorarbeit offen]**

1.1 Fragestellung

Die Arbeit beantwortet zehn Fragen.

1. Welche Unterschiede zwischen Windows, Linux und macOS betreffen das Laden einer Bibliothek, und welche davon sind Plattformcode ohne Fachlogik? (Kapitel 4)
2. Was lässt sich überhaupt vereinheitlichen, und was beweisbar nicht? (Kapitel 5)
3. Wie lässt sich die standardisierte Ladeprozedur als mathematisches Modell fassen? (Kapitel 6)
4. Welche Garantien folgen für die Namensauflösung und die Suchordnung? (Kapitel 7)
5. Welche Garantien folgen für Container, Metadaten und Versionsbereiche? (Kapitel 8)
6. Welche Garantien folgen für den Loader-Kern: Terminierung, Zykluserkennung, Referenzzählung, Rollback, Lebenszyklus, Fehlerursache, Export und Nebenläufigkeit? (Kapitel 9)
7. Wie weit ist der Kern vom Betriebssystem unabhängig, und was leistet der Plattformadapter? (Kapitel 10)
8. Was leisten Codegenerierung und Build, und welche Aussagen gelten für sie? (Kapitel 11)
9. Was ändert sich in Gen-DB und `subgraph-sat-solver-econ`, wenn sie das Ladeverfahren übernehmen, und was kostet es? (Kapitel 12 und 13)
10. Wo weicht die Implementierung vom Proposal ab, und was folgt daraus? (Kapitel 14 und 15)

1.2 Arten von Aussagen in dieser Arbeit

Wie in der Vorarbeit unterscheidet die Arbeit vier Arten von Aussagen und kennzeichnet sie im Text.

- *Bewiesen:* Sätze, Lemmata und Korollare mit Beweis im mathematischen Modell von Kapitel 6. Ein Beweis gilt für das Modell. Dass das Modell den Quelltext trifft, wird durch die zwei folgenden Arten gesichert.

– *Aus dem Quelltext gelesen:* Eigenschaften, die sich aus dem untersuchten Stand von `sharedlib` ergeben (Archiv `sharedlib-main`, Paketversion 0,1,0, Distribution `sharedlib` 1,0,0). Die Zeilenangaben beziehen sich auf diesen Stand.

– *Geprüft:* Ergebnisse eigener Läufe unter Linux mit Python 3,13 und GCC (Kapitel 13). Sie belegen die geprüften Fälle und Eigenschaften, keine allgemeine Laufzeitaussage. Die Läufe benutzen nur die Standardbibliothek und das Paket `sl` selbst. Die Testsuite des Repositories konnte nicht ausgeführt werden, weil `pytest` in der Arbeitsumgebung nicht installierbar war (Abschnitt 13.2).

– *Offen:* Aussagen, die einen weiteren Lauf, eine andere Plattform oder eine Entscheidung erfordern. Sie sind rot gekennzeichnet, etwa **[offen]**.

Die Trennung ist hier besonders wichtig, weil Beweise und Messungen unterschiedliche Dinge belegen: Ein Satz über die Zustandsmaschine des Loaders gilt für jede Eingabe, sagt aber nichts über die Zeit; eine Messung sagt etwas über die Zeit, aber nur für den gemessenen Rechner.

1.3 Beiträge

1. Ein formales Modell des Proposals (Kapitel 6) in Form von Definitionen für Dateisystem, Umgebung, Suchordnung, Container, Versionsbereiche, Loader-Zustand und Adapter.
2. Über fünfzig bewiesene Aussagen (Beweisindex in Anhang A), darunter die Nicht-Vereinheitlichbarkeit von Maschinencode (Satz 5.1), die exakte Rundreisebedingung der `SL_PATH`-Kodierung (Satz 7.2) und die Invariante der Referenzzählung (Satz 9.3).
3. Ein Abgleich von Proposal und Implementierung (Kapitel 15) mit fünfzehn Befunden (Kapitel 14), die sich teils beweisen, teils reproduzieren lassen.
4. Eine Messung der Kosten des Öffnens und des Aufrufs (Abschnitt 13.3).
5. Ein sehr ausführlicher Ausblick (Kapitel 17), der aus den Befunden, Beweislücken und Entwurfsfragen des Proposals ein Arbeitsprogramm ableitet.

1.4 Aufbau der Arbeit

Die Arbeit hat fünf Teile (Abbildung 1.1).

Teil I legt Begriffe und die formalen Grundlagen fest und ordnet `sharedlib` in die Projektlandschaft und die Wiederverwendungssystematik der Vorarbeit ein. Teil II beschreibt den Ist-Zustand des Ladens auf den drei Betriebssystemen und die Grenzen dessen, was sich vereinheitlichen lässt,

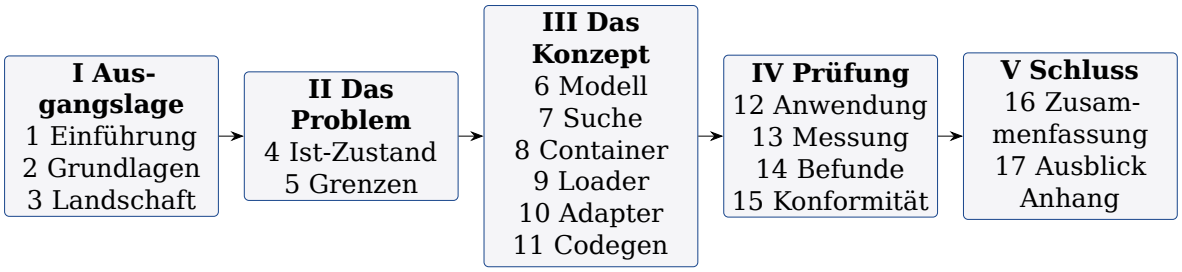


Abbildung 1.1: Gliederung der Arbeit. Teil II benennt das Problem und seine Grenzen, Teil III entwickelt und beweist das Konzept, Teil IV prüft es gegen die Implementierung und die Anwendungen, Teil V fasst zusammen und blickt voraus.

einschließlich eines Beweises, dass Maschinencode sich nicht durch Umbenennen vereinheitlichen lässt. Teil III ist der Kern: ein Modell, in dem die Sätze über Suchordnung, Container, Loader, Adapter und Codegenerierung bewiesen werden. Teil IV wendet das Konzept auf Gen-DB und `subgraph-sat-solver-econ` an, berichtet die Prüfläufe und Messungen, die fünfzehn Befunde und den Abgleich mit dem Proposal. Teil V fasst zusammen und gibt den Ausblick. Anhang A listet alle Sätze mit Quelltextstelle und Prüfung, Anhang B beschreibt die Prüfskripte.

Kapitel 2

Formale Grundlagen

Dieses Kapitel legt die Begriffe fest, auf denen die Beweise der Teile II und III beruhen. Es ist bewusst klein gehalten: Zeichenketten, endliche Abbildungen, endliche gerichtete Graphen, eine Ordnung auf Versionen und ein Ergebnistyp mit Ereignisspur. Alles Weitere (Suchordnung, Container, Loader) wird erst in Kapitel 6 definiert.

2.1 Zeichenketten, Listen und endliche Abbildungen

Sei Σ ein endliches Alphabet, das die in Pfaden, Namen und Umgebungsvariablen vorkommenden Zeichen enthält. Σ^* bezeichnet die endlichen Wörter über Σ, ε das leere Wort, $u \cdot v$ (oder uv) die Verkettung und $|u|$ die Länge. Ein Wort u ist ein *Präfix* von w, wenn $w = uv$ für ein v. Für eine Liste $L = (x_1, \dots, x_k)$ ist $|L| = k$, $L \cdot L'$ die Verkettung und $L[i] = x_i$. Die Abbildung $\mathrm{dedup}(L)$ streicht aus L jedes Element, das weiter links schon vorkommt (die Reihenfolge der ersten Vorkommen bleibt).

Lemma 2.1 (Dedup erhält den ersten Treffer)**.** Sei L eine Liste und P ein Prädikat auf den Elementen. Das erste Element von L, das P erfüllt, ist auch das erste Element von $\mathrm{dedup}(L)$, das P erfüllt (und umgekehrt). Gibt es keines, gilt dasselbe für beide Listen.

Proof. Sei $x = L[i]$ das erste Element mit $P(x)$. Kein früheres Element erfüllt P. x wird nur gestrichen, wenn es links schon vorkommt, dann gäbe es ein früheres Element $L[j] = x$ mit $j < i$, das P erfüllte, im Widerspruch zur Wahl von i. Also bleibt x erhalten. Streichungen betreffen nur Wörter, die links schon stehen, und diese erfüllen P genau dann, wenn ihr erstes Vorkommen es tut; vor x steht daher in $\mathrm{dedup}(L)$ kein Element mit P. Gibt es kein Element mit P, gibt es auch nach dem Streichen keines, weil $\mathrm{dedup}(L)$ eine Teilliste von L ist. □

Eine *partielle Abbildung* $f : A \rightharpoonup B$ hat einen Definitionsbereich $\mathrm{dom}\, f \subseteq A$. Wir schreiben $f(a) = \bot$, wenn $a \notin \mathrm{dom}\, f$.

2.2 Dateisystem und Umgebung

Definition 2.1 (Pfade, Dateisystem, Umgebung)**.** $\mathsf{Path} = \Sigma^*$ ist die Menge der Pfade. Ein *Dateisystem* ist eine endliche partielle Abbildung $\mathsf{FS} : \mathsf{Path} \rightharpoonup \mathsf{Bytes}$ mit $\mathsf{Bytes} = \{0, \ldots, 255\}^*$; $p \in \operatorname{dom}\mathsf{FS}$ heißt: p ist eine Datei. Eine *Umgebung* ist eine partielle Abbildung $E : \mathsf{Var} \rightharpoonup \Sigma^*$. Die Funktion $\mathrm{isabs} : \mathsf{Path} \to \{0,1\}$ unterscheidet absolute und relative Pfade. $\mathrm{join}(d, f)$ ist der Pfad der Datei f im Verzeichnis d. Ein *Arbeitsverzeichnis* $w \in \mathsf{Path}$ ist absolut; es wird nur zur Auflösung relativer Pfade benutzt: Für relatives p ist $\mathsf{FS}(p)$ gleichbedeutend mit $\mathsf{FS}(\mathrm{join}(w, p))$.

Die Modellierung als *unveränderliches* Dateisystem entspricht der Annahme, dass sich während einer Operation des Loaders keine Datei ändert. Sie ist für sequentielle Aussagen nötig; die Nebenläufigkeit behandelt Abschnitt 9.9 getrennt.

Lemma 2.2 (Absolute Auflösung ist arbeitsverzeichnisunabhängig)**.** Ist p absolut, so ist $\mathsf{FS}(p)$ unabhängig von w.

Proof. Nach Definition 2.1 wird w nur bei relativem p benutzt. □

2.3 Gerichtete Graphen und Tiefensuche

Definition 2.2 (Graph, Zyklus, erreichbar)**.** Ein *endlicher gerichteter Graph* ist $G = (V, E)$ mit endlichem V und $E \subseteq V \times V$. Ein *Zyklus* ist eine Folge $v_0, v_1, \ldots, v_k = v_0$ mit $k \geq 1$ und $(v_i, v_{i+1}) \in E$ (Schleifen (v, v) eingeschlossen). $\mathrm{reach}(r)$ ist die Menge der von r aus über Kanten erreichbaren Knoten einschließlich r. Ein Graph ist *azyklisch*, wenn er keinen Zyklus enthält.

Der Loader durchläuft das Abhängigkeitsgraphen als Tiefensuche mit einem Stapel S der gerade im Laden befindlichen Knoten und einer Menge D bereits abgeschlossener Knoten. Das folgende Lemma abstrahiert dieses Verfahren.

Definition 2.3 (Tiefensuche mit Stapel)**.** $\mathrm{DFS}(v)$ arbeitet mit Stapel S (zunächst leer) und Menge D (zunächst eine vorgegebene Menge $D_0 \subseteq V$, *abgeschlossene* Knoten): Ist $v \in S$, wird *Zyklus* gemeldet und abgebrochen. Ist $v \in D$, kehrt der Aufruf sofort zurück. Sonst wird v auf S gelegt, $\mathrm{DFS}(w)$ für alle Nachfolger w von v der Reihe nach aufgerufen, v von S genommen und in D eingefügt.

Lemma 2.3 (Zykluserkennung durch Tiefensuche)**.** Sei G endlich und $D_0 \subseteq V$. Dann terminiert $\mathrm{DFS}(r)$, und es meldet genau dann *Zyklus*, wenn der Graph $G - D_0$ (ohne die Knoten aus D_0) in $\mathrm{reach}_{G-D_0}(r)$ einen Zyklus enthält. Ohne Meldung gilt am Ende $D = D_0 \cup \mathrm{reach}_{G-D_0}(r)$, und jeder Knoten außerhalb von D_0 wird höchstens einmal auf den Stapel gelegt. Ist $r \in D_0$, kehrt der Aufruf sofort zurück.

Proof. Terminierung. Jeder Knoten wird höchstens einmal auf den Stapel gelegt, denn danach liegt er entweder auf S (dann folgt Abbruch) oder in

D (dann kehrt der Aufruf zurück); beides löst keinen weiteren Aufruf aus. Knoten aus D_0 werden nie gelegt. Es gibt also höchstens $|V|$ Stapelvorgänge und höchstens $|E|+1$ Aufrufe von DFS, denn jeder Aufruf außer dem ersten entspricht einer Kante.

Meldung ⇒ Zyklus. Gemeldet wird bei einem Aufruf DFS(w) mit $w \in S$ aus einem Knoten x heraus, der oben auf S liegt. Die Knoten auf S bilden von unten nach oben $s_1, \ldots, s_m = x$ mit $(s_i, s_{i+1}) \in E$, denn jeder wurde beim Durchlaufen der Nachfolger des vorigen gelegt, und keiner liegt in D_0. Mit $w = s_j$ ist $s_j, \ldots, s_m, w$ ein Zyklus in $\text{reach}_{G-D_0}(r)$.

Zyklus ⇒ Meldung. Enthalte $\text{reach}_{G-D_0}(r)$ einen Zyklus C, und die Suche melde nicht. Sei w der Knoten von C, der als erster auf den Stapel gelegt wird (er wird gelegt, weil C in $\text{reach}_{G-D_0}(r)$ liegt und ohne Meldung jeder dort erreichbare Knoten gelegt wird, siehe unten), und $w = u_0, u_1, \ldots, u_k = x, u_{k+1} = w$ der Zyklus ab w. Zum Zeitpunkt, an dem w gelegt wird, ist keiner der u_i schon gelegt worden. Wir zeigen: Jedes u_i wird gelegt, bevor w vom Stapel genommen wird. Für $i = 0$ ist es klar. Wird u_i während der Lebenszeit von w gelegt, so ruft u_i später DFS(u_{i+1}) auf. Ist u_{i+1} dann schon gelegt, war das nach dem Legen von w (nach Wahl von w), also während seiner Lebenszeit; sonst wird es jetzt gelegt. Also wird $x = u_k$ während der Lebenszeit von w gelegt und ruft DFS(w) auf, mit $w \in S$. Das ist eine Meldung, im Widerspruch zur Annahme.

Ohne Meldung. Ohne Meldung wird jeder Nachfolger jedes gelegten Knotens aufgerufen, und jeder aufgerufene Knoten wird gelegt, liegt schon in D oder in D_0. Induktiv über die Weglänge in $G - D_0$ entsteht $D \supseteq D_0 \cup \text{reach}_{G-D_0}(r)$, und $\subseteq$ gilt, weil nur über Kanten aufgerufen wird und nur Knoten außerhalb von D_0 gelegt werden. □

2.4 Versionen

Definition 2.4 (Versionen und Ordnung)**.** $\mathsf{V} = \mathbb{N}^3$ ist die Menge der Versionen (a, b, c) mit der lexikographischen Ordnung $\leq_{\text{lex}}$. $\leq_{\text{lex}}$ ist eine totale Ordnung, die mit dem Tupelvergleich in Python übereinstimmt. Eine Menge $S \subseteq \mathsf{V}$ heißt *konvex*, wenn aus $x, y \in S$ und $x \leq z \leq y$ folgt $z \in S$.

2.5 Ergebnisse, Fehler und Ereignisspuren

Definition 2.5 (Status, Ergebnis, Spur)**.** Die Menge der Fehlerstatus ist $\mathsf{St} = \{\text{NOT_FOUND}, \text{DEPENDENCY_MISSING}, \text{ARCH_MISMATCH}, \text{ABI_MISMATCH}, \text{INVALID_CONTAINER}, \text{SYMBOL_NOT_FOUND}, \text{INIT_FAILED}, \text{CYCLE}, \text{ACCESS_DENIED}, \text{INTERNAL}\}$ (zehn Werte, dazu $\text{OK} = 0$). Ein *Ergebnis* ist $\text{Ok}(x)$ oder $\text{Err}(s, n, \ldots)$ mit $s \in \mathsf{St}$ und Diagnosedaten (Name n, durchsuchte Pfade, native Fehlernummer). Eine *Spur* ist eine endliche Folge von Adapterereignissen $\mathsf{map}(k)$, $\mathsf{init}(k)$, $\mathsf{fini}(k)$, $\mathsf{unmap}(k)$ für Schlüssel k.

Spuren machen die Seiteneffekte des Loaders auf den Adapter beobachtbar. Ein Beweis über die Reihenfolge von Initialisierung und Finalisierung

ist damit ein Beweis über Spuren.

2.6 Annahme zur Hashfunktion

Der Container sichert Payloads mit SHA-256 [8]. Wo ein Satz darauf beruht, steht ausdrücklich die

Definition 2.6 (Annahme H)**.** Zu gegebenem $x \in \mathsf{Bytes}$ findet kein Angreifer ein $x' \neq x$ mit $\mathrm{SHA256}(x') = \mathrm{SHA256}(x)$.

Sie ist eine unbewiesene, aber allgemein geteilte Annahme und im Folgenden die einzige kryptographische Voraussetzung.

2.7 Symbolübersicht

Symbol	Bedeutung
FS, E, w	Dateisystem, Umgebung, Arbeitsverzeichnis (Def. 2.1)
Enc, Dec	Kodierung und Dekodierung von `SL_PATH` (Def. 7.2)
$\mathrm{cand}_C(n, \rho)$	Kandidatenliste der Suche (Def. 7.4)
V, $S(\mathrm{spec})$	Versionen, Erfüllungsmenge eines Bereichs (Def. 8.2)
pack, unpack	Container schreiben und lesen (Def. 8.3)
$\Gamma = (\mathcal{L}, S)$	Zustand des Loaders: geladene Knoten und Ladestapel (Def. 6.3)
$\mathrm{rc}(k)$, $\mathrm{ext}(k)$	Referenzzähler und Zahl externer Referenzen
St	Menge der Fehlerstatus (Def. 2.5)

Tabelle 2.1: Die wichtigsten Symbole der Arbeit.

Kapitel 3

Projektlandschaft und Wiederverwendungsformen

3.1 Die Repositories

Zu den fünf Projekten der Vorarbeit tritt `sharedlib`. Tabelle 3.1 fasst Aufgabe und Rolle zusammen; die letzte Spalte beantwortet, wer den Subgraph Algorithmus *enthält*. Der Quelltext von `gen-db`, `subgraph-sat-solver-econ` und `csubgraph` wurde in dieser Arbeit nicht erneut geprüft; ihre Angaben stammen aus der Vorarbeit und aus dem Abschnitt 2.1 des Proposals und sind entsprechend gekennzeichnet. Geprüft wurde nur `sharedlib`.

Repository	Sprache	Aufgabe	Algorithmus enthalten?
`github.com/naphets86/subgraph`	Python	Referenzimplementierung des Subgraph Algorithmus	ja
`github.com/naphets86/csubgraph`	C++17	statische Bibliothek mit drei Modulen; C-Wrapper für `ctypes`	ja
`github.com/naphets86/gen-db`	Python, C++	Datenbank biologischer Netzwerke, lädt `csubgraph` über einen C-Wrapper (`csubgraph_shim.cpp`, `csubgraph_native.py`)	Relation neu implementiert, Bibliothek geladen
`github.com/naphets86/subgraph-sat-solver-econ`	Python, C++	SAT-Solver mit Bibliothek `wsat_shim`, lädt sie über `native.py`	nein
`github.com/naphets86/subgraph-sat-solver-experiments`	Python	Messframework	nein
`github.com/naphets86/sharedlib`	Python $\geq 3{,}11$	Referenzimplementierung des `.sl`-Formats und der Ladeprozedur	nein

Tabelle 3.1: Die sechs Projekte. `sharedlib` enthält keinen Teil des Algorithmus, sondern die Schicht, über die ihn Python-Projekte laden könnten.

3.2 Was in `sharedlib` steckt

Tabelle 3.2 zeigt die Module des Pakets `sl` mit Zeilenzahl, Aufgabe und der Schicht des Proposals (Abschnitt 5.2 des Proposals). Die Zeilenzahlen sind aus dem Quelltext gezählt (`wc -l`).

Modul	Zeilen	Schicht	Inhalt
`errors.py`	37	Fehlermodell	`SLStatus` (elf Werte), `SLError`
`platform_info.py`	35	Plattformadapter (Namen)	Normalisierung von Plattform und Architektur
`metadata.py`	143	Container	Metadaten, Versionen, Bereiche
`container.py`	98	Container	Layout, `pack`, `unpack`, Hashes
`search.py`	144	Loader-Kern	Namensauflösung, `SL_PATH`, Suchordnung
`loader.py`	178	Loader-Kern	Zustandsmaschine, Referenzzählung
`adapters.py`	100	Plattformadapter	Abbilden, Symbole, Freigeben
`capi.py`, `binding.py`	65, 36	Schnittstelle	`sl_open` usw., Python-Binding
`codegen.py`	145	Werkzeuge	Header, C-Quelle, CMake, Binding
`build.py`, `cli.py`	91, 77	Werkzeuge	Toolchain, Build-Matrix, Kommandozeile

Tabelle 3.2: Module des Pakets `sl` (insgesamt 1 170 Zeilen ohne Tests). Die Testdateien haben weitere 1 150 Zeilen.

Abbildung 3.1 zeigt die Schichtung des Proposals und des Pakets. Entscheidend ist der Satz, dass alles Betriebssystemspezifische im Adapter liegt (Abschnitt 5.1 des Proposals); Kapitel 10 prüft, wie weit das stimmt.

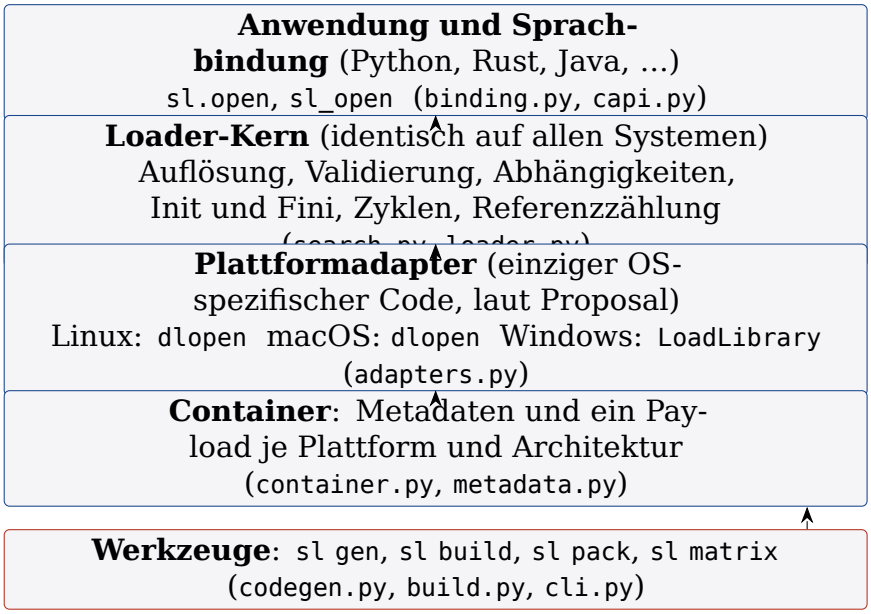


Abbildung 3.1: Schichten von `sharedlib`. Die Werkzeuge erzeugen den Container, die drei oberen Schichten benutzen ihn zur Laufzeit.

3.3 Abhängigkeiten

Abbildung 3.2 zeigt, wo `sharedlib` in die Abhängigkeiten der Vorarbeit eingreifen würde. Durchgezogene Pfeile sind bestehende Laufzeit- oder Bauabhängigkeiten, die gestrichelten Pfeile sind die *vorgeschlagenen* Wege über `sl`: Sie bestehen heute nicht, die Migration ist nicht durchgeführt (Abschnitt 12 des Proposals, Kapitel 12).

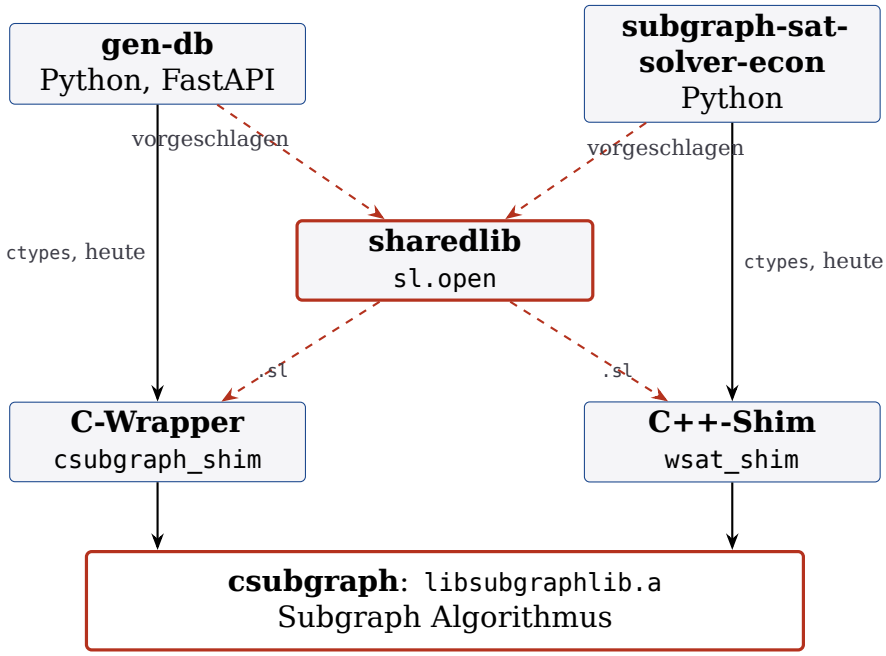


Abbildung 3.2: Abhängigkeiten. Rot gestrichelt: der Weg über `sl`, den das Proposal vorschlägt.

3.4 Befund aus den zwei Projekten

Das Proposal belegt den Bedarf mit sechs Stellen in den zwei Projekten (Abschnitt 2.1 des Proposals). Sie sind in Tabelle 3.3 zusammengefasst. Die Spalte *Plattformcode* ist die Einschätzung des Proposals, dass keine Stelle Fachlogik enthält. **[Quelltext der zwei Projekte in dieser Arbeit nicht neu geprüft]**

3.5 Eine Systematik der Wiederverwendung

Die Vorarbeit unterscheidet sieben Formen der Wiederverwendung: Portierung, Verallgemeinerung, Neuimplementierung als Fallback, Bibliotheksnutzung, Prozessnutzung, Infrastruktur und begriffliche Wiederverwendung [1]. Sie unterscheiden sich darin, welche Änderungen am Original sich fortpflanzen: Portierung, Verallgemeinerung und Neuimplementierung müssen von Hand nachgezogen und geprüft werden, Bibliotheks-

Stelle	Projekt	Inhalt
`library_filename()` in `native.py`	`subgraph-sat-solver-econ`	vier Dateinamen je nach `sys.platform`: `wsat_shim.dll`, `libwsat_shim.dll`, `libwsat_shim.so`, `libwsat_shim.dylib`
`os.add_dll_directory` in `native.py`	`subgraph-sat-solver-econ`	ein Aufruf je `PATH`-Eintrag, nur unter Windows nötig
`build_shim()`	`subgraph-sat-solver-econ`	Sonderfall: sucht zusätzlich `libwsat_shim.dll` für MinGW
Makro `CSUB_EXPORT` in `csubgraph_shim.cpp`	`gen-db`	`extern "C" __declspec(dllexport)` unter Windows, sonst anders
`csubgraph_native.py`	`gen-db`	verlinkt unter Windows die MinGW-Laufzeit statisch, damit `libstdc++-6.dll` nicht geladen werden muss
README	`subgraph-sat-solver-econ`	Build in PowerShell-Syntax, vier mögliche Dateinamen

Tabelle 3.3: Plattformcode in den zwei Projekten nach dem Proposal.

und Prozessnutzung erben jede Änderung beim nächsten Bau, Infrastruktur und Begriffliches bleiben von Änderungen am Algorithmus unberührt.

`sharedlib` fügt der Wiederverwendungskette keine Form hinzu, sondern *verschiebt, was wiederverwendet wird*: Statt eines Algorithmus ist es die Ladeschicht. Tabelle 3.4 ordnet die fünf neuen Wiederverwendungen **W19** bis **W23** den bestehenden Formen zu. Sie sind *Kandidaten*: Das Proposal beschreibt sie, durchgeführt ist die Migration nicht.

Tabelle 3.4: Neue Wiederverwendungen. Sie betreffen nie den Algorithmus und gehören deshalb zu Bibliotheksnutzung, Infrastruktur und begrifflicher Wiederverwendung.

Nr.	Wiederverwendung	Form	Status
W19	`sl.open` ersetzt Namenswahl, Pfadsuche und `add_dll_directory` in `native.py` (`subgraph-sat-solver-econ`)	Bibliotheksnutzung	Kandidat
W20	`sl.open` ersetzt das Laden und die Laufzeitbehandlung in `csubgraph_native.py` (`gen-db`)	Bibliotheksnutzung	Kandidat
W21	erzeugter Header mit `SL_EXPORT` ersetzt `CSUB_EXPORT` und plattformabhängige Exportzeilen	Infrastruktur	Kandidat
W22	`sl build` und `sl pack` erzeugen ein Fat-`.sl` aus den Plattform-Binaries der Shims	Infrastruktur	Kandidat

Nr.	Wiederverwendung	Form	Status
W23	Ladeprozedur als Spezifikation (Suchordnung, Fehlermodell, Lebenszyklus) für alle Sprachbindungen	begriffliche Wiederverwendung	Proposal liegt vor

Aus Tabelle 3.4 folgt die praktische Konsequenz, die die Vorarbeit für die Formen zieht: Wer den Subgraph Algorithmus ändert, muss an den Wiederverwendungen **W19** bis **W23** nichts nachziehen, denn keine von ihnen enthält Algorithmuscode. Wer dagegen die *Ladeprozedur* ändert, etwa die Suchordnung oder den Fehlercode, ändert das Verhalten aller Projekte, die über `sl` laden, ohne dass sie ihren Quelltext ändern. Das ist der Gewinn der Standardisierung und zugleich ihr Risiko; die Kapitel 7 bis 10 beweisen, welche Eigenschaften dabei erhalten bleiben.

Teil II

Das Problem

Kapitel 4

Der Ist-Zustand des Ladens

Dasselbe Konzept heißt unter Windows `.dll`, unter Linux `.so` und unter macOS `.dylib`. Dahinter stehen drei Binärformate (PE, ELF, Mach-O) [4, 5], drei Lader und drei Regeln dafür, welche Funktionen nach außen sichtbar sind. Dieses Kapitel stellt die Unterschiede zusammen, die das *Laden* betreffen, trennt Plattformcode von Fachlogik und führt ein einfaches Kostenmodell ein, mit dem sich der Gewinn einer gemeinsamen Schicht begründen lässt. Es ist die Ausgangslage von Abschnitt 2 und 3 des Proposals.

4.1 Unterschiede der drei Systeme

Tabelle 4.1 stellt die Aspekte gegenüber, die in den zwei Projekten tatsächlich Quelltext erzeugt haben (Abschnitt 3.4).

Auffällig ist, dass die Unterschiede in der *Schnittstelle* (Lade-API, Fehlermeldung) kleiner sind als in den *Regeln* (Sichtbarkeit, Abhängigkeitssuche). Die Schnittstelle lässt sich durch einen Aufruf kapseln, die Regeln nicht, weil sie das Verhalten bestimmen: Dieselbe Zeile `dlopen("x")` findet je nach `LD_LIBRARY_PATH` und Arbeitsverzeichnis etwas anderes. Das Proposal setzt deshalb bei den Regeln an und spezifiziert das *Verhalten* statt des Mechanismus (Abschnitt 5.1 des Proposals).

4.2 Fünf Fehlerklassen

Das Proposal nennt fünf vermeidbare Fehlerklassen (Abschnitt 3.2 des Proposals). Tabelle 4.2 ordnet jeder Klasse den Mechanismus des Proposals und den Satz dieser Arbeit zu, der ihn im Modell absichert oder in seiner Reichweite begrenzt. Die Zuordnung ist zugleich eine Vorschau auf Teil III.

4.3 Ein Kostenmodell

Das Proposal begründet den Nutzen mit zwei Argumenten: Die Arbeit entfällt pro Projekt, und die Testmatrix wird kleiner (Abschnitt 3.1 und 7 des Proposals). Das zweite Argument lässt sich genau fassen.

Aspekt	Windows	Linux	macOS
Dateiendung	`.dll`	`.so`, oft `.so.1.2.3`	`.dylib`, für Plug-ins `.so`
Namenspräfix	keines (MSVC) oder `lib` (MinGW)	`lib`	`lib`
Binärformat	PE/COFF	ELF	Mach-O
Lade-API	`LoadLibrary`	`dlopen`	`dlopen`
Symbolsuche	`GetProcAddress`	`dlsym`	`dlsym`
Sichtbarkeit standard-mäßig	nichts exportiert ohne `__declspec(dllexport)`	alles sichtbar, außer versteckt	alles sichtbar, außer versteckt
Abhängigkeiten finden	DLL-Suchpfad, `PATH`, `add_dll_directory`	`RPATH`, `RUNPATH`, `LD_LIBRARY_PATH`	`@rpath`, `@loader_path`, `DYLD_*`
Zusatzdatei beim Binden	Importbibliothek `.lib`	die `.so` selbst	die `.dylib` selbst
Versionierung	Manifest, Dateiversion	SONAME, Symbolversionen	`install_name`
Init-Haken	`DllMain`	Konstruktor, `.init_array`	Konstruktor, `__mod_init_func`
Fehlermeldung	`GetLastError`, Zahl	`dlerror`, Zeichenkette	`dlerror`, Zeichenkette
Entladen	`FreeLibrary`	`dlclose`	`dlclose`

Tabelle 4.1: Unterschiede beim Laden einer dynamischen Bibliothek (nach Abschnitt 2 des Proposals; Zeilen *Lade-API* und *Entladen* zeigen, dass Linux und macOS dieselbe Schnittstelle haben, aber ein anderes Binärformat).

Definition 4.1 (Testfälle)**.** Es gebe $n \geq 1$ Projekte, die je eine native Bibliothek laden, und $m \geq 1$ Kombinationen aus Betriebssystem und Toolchain. *Ohne gemeinsame Schicht* muss jedes Projekt sein eigenes Ladeverhalten auf jeder Kombination prüfen: $T_{\text{ohne}} = n \cdot m$ Fälle. *Mit gemeinsamer Schicht* wird das Verhalten einmal je Kombination durch eine Konformitätssuite geprüft (m Fälle), und jedes Projekt prüft nur noch einmal, dass es die Schicht richtig aufruft (n Fälle): $T_{\text{mit}} = n + m$.

Lemma 4.1 (Summe statt Produkt)**.** Es gilt $T_{\text{ohne}} - T_{\text{mit}} = (n-1)(m-1) - 1$. Daraus folgt: $T_{\text{mit}} \leq T_{\text{ohne}}$ genau dann, wenn $n \geq 2$ und $m \geq 2$; $T_{\text{mit}} < T_{\text{ohne}}$ genau dann, wenn $(n-1)(m-1) \geq 2$; für $n = 1$ oder $m = 1$ ist $T_{\text{mit}} = T_{\text{ohne}} + 1$. Für festes $m \geq 2$ wächst die Differenz linear in n mit Steigung $m - 1$.

Proof. $(n-1)(m-1) = nm - n - m + 1$, also $nm - n - m = (n-1)(m-1) - 1$. Ist $n = 1$ oder $m = 1$, so ist $(n-1)(m-1) = 0$ und die Differenz -1, das heißt $T_{\text{mit}} = T_{\text{ohne}} + 1$ (die Schicht kostet einen Fall mehr). Sind $n, m \geq 2$, so ist $(n-1)(m-1) \geq 1$ und die Differenz ≥ 0; sie ist genau 0 bei $(n-1)(m-1) = 1$, also $n = m = 2$, und echt positiv genau bei $(n-1)(m-1) \geq 2$. Die Steigung in n ist $m - 1$, weil $(n-1)(m-1) - 1$ linear in n mit diesem Koeffizienten ist. □

Das Lemma korrigiert eine zu glatte Formulierung: Eine gemeinsame Schicht lohnt sich *nicht* für ein einzelnes Projekt (Fall $n = 1$ kostet einen Fall

Fehlerklasse	Mechanismus des Proposals	Aussage dieser Arbeit
Funktion nur auf Linux sichtbar, unter Windows nicht exportiert	ein Exportmakro, Exportliste im Container	Satz 9.8: nur Exportierte sind erreichbar; Einschränkung: keine Sicherheitsgrenze (**B12**)
fehlende abhängige DLL, Meldung nennt nur die angeforderte Datei	Auflösung der Abhängigkeiten vor dem Abbilden, Feld `failed_name`	Satz 9.7: `failed_name` nennt die fehlende Bibliothek; Einschränkung: gilt für die `.sl`-Ebene, nicht für native Importe (**B7**)
MinGW- und MSVC-Bibliotheken gemischt, Laufzeiten passen nicht	Feld `runtime` im Container	nicht gelöst: das Feld hat im Loader keine Semantik (**B9**)
gleicher Dateiname, verschiedene Bedeutung (`.so` auf macOS)	eine Endung `.sl`, keine Plattformpräfixe	Satz 7.1: Namensauflösung ist plattformunabhängig und liefert genau einen Dateinamen
Suchpfade verschieden in Einzelheiten (Arbeitsverzeichnis, `PATH`)	feste Suchordnung, kein Arbeitsverzeichnis, kein `PATH`	Sätze 7.3, 7.4 und 7.5; Einschränkung: `SL_PATH` (Satz 7.2, **B1**, **B2**)

Tabelle 4.2: Fehlerklassen, Mechanismen und die Sätze, die sie absichern.

mehr) und nicht für eine einzige Plattform; sie lohnt sich ab drei Projekten auf zwei Kombinationen oder zwei Projekten auf drei Kombinationen. Mit den zwei Projekten der Vorarbeit und den drei Betriebssystemen ($n = 2$, $m \geq 3$) ist $T_{\text{ohne}} = 2m \geq 6$ gegen $T_{\text{mit}} = m + 2 \geq 5$ und die Differenz $(m-1) - 1 = m - 2 \geq 1$. Rechnet man MSVC und MinGW unter Windows getrennt ($m = 4$), beträgt die Differenz 2. **[Ob die Fälle in der Praxis gleich teuer sind, ist nicht untersucht]**

Bemerkung 4.1 (Grenzen des Modells)**.** Das Modell zählt Fälle, nicht Aufwand. Es nimmt an, dass sich jedes Projekt mit einem Fall auf die richtige Benutzung der Schicht prüfen lässt und dass die Konformitätssuite die Plattformunterschiede vollständig abdeckt. Beides ist eine Forderung an die Suite (Abschnitt 5.9 des Proposals), die das Repository bisher nur teilweise erfüllt (Kapitel 15).

4.4 Was Plattformcode ist und was nicht

Das Proposal behauptet: *Keine der Stellen enthält Fachlogik*. Eine genaue Fassung dieser Behauptung unterscheidet drei Arten von Code in einem Projekt, das eine native Bibliothek lädt.

Definition 4.2 (Arten von Code)**.** *Fachlogik* ist Code, dessen Ergebnis

sich nicht ändert, wenn man das Betriebssystem tauscht (der Algorithmus selbst). *Plattformcode* ist Code, der abhängig vom Betriebssystem verzweigt oder eine betriebssystemabhängige Schnittstelle aufruft, ohne dass das Ergebnis der Anwendung davon abhängt. *Kopplungscode* bindet die Fachlogik an den Plattformcode (Argumente, Rückgabetypen).

Alle sechs Stellen von Tabelle 3.3 sind Plattformcode in diesem Sinn. Das Ziel von `sharedlib` ist, ihn aus den Projekten in *eine* Schicht zu verschieben, in der er nur einmal geschrieben, getestet und dokumentiert ist; der Kopplungscode wird durch den erzeugten Python-Binding-Text ersetzt (Satz 11.2). Das ist keine Aussage darüber, ob der Plattformcode *kleiner* wird, sondern darüber, dass er *einmal* existiert.

Kapitel 5

Grenzen der Vereinheitlichung

Ein Vorschlag, der drei Systeme vereinheitlichen will, muss sagen, was er *nicht* vereinheitlicht. Dieses Kapitel zieht die Grenze mathematisch: Der erste Satz zeigt, dass sich Maschinencode nicht durch Umbenennen vereinheitlichen lässt, der zweite, dass sich auch die Bedeutung eines C-Typs von System zu System ändert. Daraus folgt, was das Proposal sinnvollerweise verspricht. Abschnitt 5.5 bewertet die Einwände des Proposals (Abschnitt 8 des Proposals).

5.1 Ziele und Nichtziele

Das Proposal legt sieben Ziele und drei Nichtziele fest (Abschnitt 4 des Proposals).

Z1 Ein Dateiname `<name>.sl` auf allen Systemen.

Z2 *Ein* standardisiertes, betriebssystemunabhängiges Ladeverfahren, festgelegt durch eine Spezifikation, nicht durch das Verhalten des darunterliegenden Laders.

Z3 Ein Exportmodell: ausdrücklich, opt-in, auf allen Systemen gleich.

Z4 Ein Metadatenblock mit ABI-Version, Name, Version und Abhängigkeiten.

Z5 Eine Suchpfadregel mit festgelegter Rangfolge.

Z6 Ein Fehlermodell mit maschinenlesbaren Codes und Klartext, das fehlende Abhängigkeiten benennt.

Z7 Koexistenz mit bestehenden Bibliotheken.

N1 Identischer Maschinencode auf jedem Betriebssystem und jeder CPU.

N2 Kurzfristiger Ersatz von PE, ELF oder Mach-O.

N3 Standardisierung von C++-ABIs; die stabile Grenze ist die C-ABI.

Das erste Nichtziel **N1** wird gewöhnlich mit dem Einwand angegriffen, gleiche CPU-Architektur bedeute gleiche Bibliothek. Der folgende Satz zeigt, dass dieser Einwand zutrifft, soweit es um Maschinencode geht, und dass N1 daher kein Mangel des Vorschlags, sondern eine Notwendigkeit ist.

5.2 Aufrufkonventionen

Definition 5.1 (Maschinenmodell)**.** Ein *Registerzustand* ist eine Abbildung $\sigma : \mathsf{Reg} \to \mathsf{W}$ mit der Registermenge $\mathsf{Reg} \supseteq \{\mathrm{rax}, \mathrm{rcx}, \mathrm{rdi}\}$ und Wörtern $\mathsf{W} = \{0, \ldots, 2^{64} - 1\}$. Ein *Programmtext* (Folge von Maschinenbefehlen) wird durch seine Wirkung $p : (\mathsf{Reg} \to \mathsf{W}) \to (\mathsf{Reg} \to \mathsf{W})$ beschrieben: der Registerzustand bei der Rückkehr als Funktion des Registerzustands beim Einsprung. Die Wirkung hängt nur von den Bytes des Programmtexts und von der CPU ab, nicht vom Betriebssystem.

Das Modell lässt Speicher, Sprünge und weitere Register weg. Es ist ein Teilmodell: Ein Beweis, dass schon die Registerwirkung nicht übereinstimmen kann, gilt erst recht für das volle Modell.

Definition 5.2 (Aufrufkonvention, Korrektheit)**.** Für Funktionen $g : \mathsf{W} \to \mathsf{W}$ (ein ganzzahliges Argument, ein ganzzahliges Ergebnis) legt eine Aufrufkonvention κ ein Register $\mathrm{arg}_\kappa \in \mathsf{Reg}$ für das Argument fest und rax für das Ergebnis. Ein Programm p ist *κ-korrekt für g*, wenn für jeden Registerzustand σ gilt $p(\sigma)(\mathrm{rax}) = g(\sigma(\mathrm{arg}_\kappa))$. Die beiden Konventionen von x86-64 sind κ_{Win64} mit $\mathrm{arg} = \mathrm{rcx}$ und κ_{SysV} mit $\mathrm{arg} = \mathrm{rdi}$.

Die Forderung “für jeden Registerzustand” ist berechtigt: Vor einem Aufruf kann der Aufrufer in anderen Registern als dem Argumentregister beliebige Werte haben (bei Windows ist `rdi` ein gesichertes Register des Aufrufers, bei System V ist `rcx` das vierte Argumentregister), und jede Belegung der Argumentregister ist ein zulässiger Aufruf.

Satz 5.1 (Maschinencode ist nicht vereinheitlichbar)**.** Sei $g : \mathsf{W} \to \mathsf{W}$ nicht konstant. Dann gibt es kein Programm p, das sowohl κ_{Win64}-korrekt als auch κ_{SysV}-korrekt für g ist.

Proof. Angenommen, p sei beides. Da g nicht konstant ist, gibt es $a, b \in \mathsf{W}$ mit $g(a) \neq g(b)$. Sei σ ein Registerzustand mit $\sigma(\mathrm{rcx}) = a$ und $\sigma(\mathrm{rdi}) = b$ (die beiden Register sind verschieden, die Belegung ist also möglich). κ_{Win64}-Korrektheit liefert $p(\sigma)(\mathrm{rax}) = g(\sigma(\mathrm{rcx})) = g(a)$, κ_{SysV}-Korrektheit liefert $p(\sigma)(\mathrm{rax}) = g(\sigma(\mathrm{rdi})) = g(b)$. Also $g(a) = g(b)$, Widerspruch. □

Korollar 5.1 (Ein Container braucht mehrere Payloads)**.** Exportiert eine Bibliothek eine nicht konstante Funktion mit ganzzahligem Argument, so müssen die Payloads für Windows x86-64 und Linux x86-64 verschiedene Bytefolgen sein. Ein Container für beide Ziele enthält daher mindestens zwei Payloads.

Proof. Wären die Bytefolgen gleich, wäre ihre Wirkung p gleich, und p wäre für beide Konventionen korrekt, im Widerspruch zu Satz 5.1. □

Das Korollar begründet das Layout des Containers: Eine Tabelle mit je einem Eintrag für Plattform und Architektur (Kapitel 8) ist nicht Entwurfsvorliebe, sondern Folge der Mathematik. Das Proposal nennt das “fat binary” (Abschnitt 6.3 des Proposals) und verweist auf Universal Binaries als Vorbild.

Bemerkung 5.1. Der Satz gilt für Ganzzahlargumente. Für das erste Gleitkommaargument benutzen beide Konventionen `xmm0`; Bibliotheken, die nur solche Funktionen exportieren, können binär übereinstimmen. Die Aussage des Satzes ist die Existenz eines Gegenbeispiels, nicht die Behauptung, dass alle Funktionen verschieden übersetzt werden müssen. Hinzu kommt, dass auch das Containerformat (PE, ELF, Mach-O), die Systemaufrufe und die C-Laufzeit verschieden sind (Abschnitt 8.1 des Proposals); die Registerbetrachtung ist die einfachste Begründung, nicht die einzige.

5.3 Datenmodelle

Definition 5.3 (Datenmodell)**.** Ein *Datenmodell* ordnet den Grundtypen `int`, `long`, `long long` und Zeigern Größen in Bytes zu. LP64 (Linux, macOS auf 64 Bit): $(4, 8, 8, 8)$. LLP64 (Windows auf 64 Bit): $(4, 4, 8, 8)$. ILP32 (32-Bit-Systeme): $(4, 4, 8, 4)$. Die Typen mit fester Breite `int`N`_t` und `uint`N`_t` haben in jedem Datenmodell $N/8$ Bytes.

Satz 5.2 (Typbreite)**.** (a) Die Größe von `long` unterscheidet sich zwischen LP64 und LLP64; es gibt einen Wert, der in LP64 als `long` darstellbar ist, in LLP64 nicht. (b) Für Typen fester Breite sind Größe und Wertebereich in allen drei Datenmodellen gleich.

Proof. (a) sizeof(`long`) $= 8$ in LP64 und 4 in LLP64. Der Wert 2^{31} liegt im Wertebereich $[-2^{63}, 2^{63} - 1]$ von LP64 und nicht in $[-2^{31}, 2^{31} - 1]$ von LLP64. (b) Der Wertebereich eines Typs der Breite N ist durch N bestimmt, nicht durch das Datenmodell; die Definition legt die Größe $N/8$ in allen Modellen fest. □

Der Satz ist die Rechtfertigung der Forderung des Proposals, die exportierte Schnittstelle benutze Typen fester Breite (Abschnitt 5.8 des Proposals). Er ist zugleich der Maßstab für einen Befund: Der Codegenerator akzeptiert `long` in Signaturen ohne Warnung (**B13**).

5.4 Was bleibt

Satz 5.1 und Satz 5.2 ziehen die Grenze. Vereinheitlichbar sind genau die Dinge, die *oberhalb* von Maschinencode und Datenmodell liegen, und alle diese Dinge betreffen das *Verfahren*, nicht das *Ergebnis* der Übersetzung (Tabelle 5.1).

Gegenstand	vereinheitlichbar?	Begründung
Dateiname und Endung	ja	Konvention (Satz 7.1)
Suchordnung und Fehlermodell	ja	Spezifikation (Kapitel 7, 9)
Exportmodell	ja, auf API-Ebene	Metadaten und Prüfung (Satz 9.8)
Container und Metadaten	ja	Format (Kapitel 8)
Lebenszyklus (Init, Fini, Zählung)	ja	Zustandsmaschine (Kapitel 9)
Maschinencode je Ziel	nein	Satz 5.1, Korollar 5.1
Größe von C-Typen	nein, nur mit festen Breiten	Satz 5.2
C++-ABI	nein, Grenze ist C	Nichtziel N3

Tabelle 5.1: Was sich vereinheitlichen lässt und was nicht.

5.5 Die Einwände des Proposals

Das Proposal bewertet fünf Einwände (Abschnitt 8 des Proposals). Hier die Einordnung mit den Mitteln dieser Arbeit.

Gleiche Architektur, gleiche Bibliothek. Der Einwand trifft für Maschinencode zu (Satz 5.1). Das Proposal zieht daraus die richtige Folgerung: Es vereinheitlicht Schnittstelle, Container und Verfahren.

Das Problem sei durch Abstraktionen gelöst. Abstraktionen wie `ctypes.CDLL` sind Implementierungen, keine Spezifikation. Zwei Abstraktionen können verschiedene Suchordnungen haben. Ein Beweis, dass sie es tatsächlich haben, liegt nicht vor, weil dafür die Suchordnungen von `ctypes` unter allen drei Systemen zu modellieren wären. **[Vergleich der Suchordnungen von `ctypes` und `sl` nicht formalisiert]** Das Proposal formuliert deshalb zu Recht, dass die Abstraktion existiert, aber nicht standardisiert ist.

Altlasten. Der Weg über einen Legacy-Modus (gewöhnliche `.dll`, `.so`, `.dylib` über dieselbe API) ist im Repository nicht umgesetzt: `sl` öffnet nur Container (Befund **B15** und Abschnitt 15.1).

Wer entscheidet? Eine Standardisierung setzt Beteiligung voraus; die Stufen 1 und 2 des Proposals brauchen sie nicht. Dieser Einwand ist organisatorisch und wird im Ausblick aufgegriffen (Abschnitt 17.12).

Neuer Lader, neue Angriffsfläche. Berechtigt. Kapitel 9 und 14 zeigen, dass die Härtung im Repository nur teilweise besteht: `hardened` verwirft `SL_PATH` und schreibbare Verzeichnisse, prüft aber weder Eigentümer noch Signaturen (**B4**, **B10**).

Teil III

Das Konzept: Modell und Beweise

Kapitel 6

Das formale Modell der Ladeprozedur

Dieses Kapitel macht aus der Beschreibung des Proposals (Abschnitt 5.3: neun Schritte) und dem Quelltext von `loader.py` ein mathematisches Objekt. Es legt fest, welche Größen die Prozedur liest (Dateisystem, Umgebung, Konfiguration), welchen Zustand sie hält (geladene Knoten, Ladestapel) und welche Operationen sie anbietet. Die Kapitel 7 bis 10 beweisen darin Eigenschaften; Abschnitt 6.5 gibt an, an welcher Quelltextstelle jede Modellgröße steht, damit die Beweise auf den Quelltext zurückführbar bleiben.

6.1 Konfiguration und Adapter

Definition 6.1 (Adapter)**.** Ein *Plattformadapter* $\mathfrak{A}$ besteht aus vier Operationen:

- $D_{\mathfrak{A}}(E)$: eine endliche Liste von Verzeichnispfaden (*Systemverzeichnisse*), abhängig von der Umgebung E;
- $\mathrm{map}_{\mathfrak{A}}(x, n)$: bildet einen Payload $x \in \mathsf{Bytes}$ der Bibliothek n ab; liefert eine *native Referenz* ν oder löst einen Betriebssystemfehler (OSError) mit Nummer $e \geq 0$ aus;
- $\mathrm{lookup}_{\mathfrak{A}}(\nu, s)$: liefert zu einem Symbolnamen s eine aufrufbare Funktion oder den Fehler SYMBOL_NOT_FOUND;
- $\mathrm{unmap}_{\mathfrak{A}}(\nu)$: gibt die native Referenz frei.

Ein Adapter *erfüllt den Vertrag*, wenn lookup bei festem (ν, s) stets dasselbe Ergebnis liefert, unmap höchstens einmal je ν aufgerufen wird und kein Aufruf von lookup nach unmap erfolgt.

Der Vertrag fasst zusammen, was der Loader von einem Adapter voraussetzt. Dass der Loader ihn einhält, ist Gegenstand von Satz 9.10 (im sequentiellen Fall gilt es) und von Befund **B6** (nebenläufig nicht).

Definition 6.2 (Konfiguration)**.** Eine *Konfiguration* $C = (\pi, \alpha, E, r, a, h, [\ell, u], \mathfrak{A})$ besteht aus Plattform $\pi \in \{\mathrm{linux}, \mathrm{macos}, \mathrm{windows}\}$, Architektur $\alpha \in$

$\{\text{x86_64}, \text{aarch64}, \text{x86}, \text{arm}\}$, Umgebung E, Aufruferverzeichnis $r \in \mathsf{Path} \cup \{\bot\}$, Anwendungsverzeichnis $a \in \mathsf{Path} \cup \{\bot\}$, dem Schalter $h \in \{0, 1\}$ (*hardened*), einem ABI-Bereich $[\ell, u] \subseteq \mathbb{N}$ (Voreinstellung $[1, 1]$) und einem Adapter $\mathfrak{A}$.

Die Konfiguration ist der Teil, den ein Aufrufer wählt; `Loader.__init__` nimmt genau diese Parameter (Abschnitt 6.5). Der Rest der Welt ist FS (Definition 2.1).

6.2 Zustand und Operationen

Definition 6.3 (Knoten, Zustand)**.** Sei $\mathsf{Key} = \mathsf{Path}$ die Menge der Schlüssel (der aufgelösten, absoluten Dateipfade). Ein *Knoten* ist $\mathsf{nd} = (\mu, d, \mathrm{rc}, \nu)$ mit Metadaten μ, der Liste $d \in \mathsf{Key}^*$ seiner Abhängigkeiten (mit Wiederholungen), einem Zähler $\mathrm{rc} \geq 1$ und einer nativen Referenz ν. Ein *Zustand* des Loaders ist $\Gamma = (\mathcal{L}, S)$ mit einer endlichen partiellen Abbildung $\mathcal{L} : \mathsf{Key} \rightharpoonup \mathsf{Node}$ (*geladen*) und einem *Ladestapel* $S \in \mathsf{Key}^*$. Der Anfangszustand ist $\Gamma_0 = (\emptyset, \varepsilon)$.

Ein Client hält *Handles*. Wir modellieren die Handles, die Clients gerade halten, als Multimenge H von Schlüsseln; $\mathrm{ext}(k)$ ist die Vielfachheit von k in H. Das entspricht dem Quelltext, in dem `open` bei bereits geladenem Schlüssel dasselbe Objekt noch einmal zurückgibt (Befund **B8**).

Definition 6.4 (Operationen)**.** Die Operationen $\mathrm{open}(n)$, $\mathrm{close}(k)$, $\mathrm{symbol}(k, s)$ und $\mathrm{info}(k)$ sind die Operationen des Loaders. Jede liefert ein Ergebnis (Definition 2.5), verändert (Γ, H) und hängt Ereignisse an die Spur an. Abbildung 6.1 gibt open und seine Hilfsoperationen an.

Zwei Beobachtungen zur Transkription sind für die Beweise wichtig.

Bemerkung 6.1 (Merkmale der Transkription)**.** (a) Der Ladestapel S wird im `finally`-Zweig immer um genau den Schlüssel verkürzt, den `open` gelegt hat; S ist nach jedem Aufruf von open auf der obersten Ebene leer. (b) Der Eintrag $\mathcal{L}[k]$ wird erst *nach* erfolgreichem `LOAD` gesetzt; während der Abhängigkeiten geladen werden, ist k in S, aber noch nicht in $\mathrm{dom}(\mathcal{L})$. Es gilt daher stets $S \cap \mathrm{dom}(\mathcal{L}) = \emptyset$. (c) Die Abhängigkeiten eines Metadatensatzes werden in der Ordnung der Namen durchlaufen (Lemma 8.1).

6.3 Annahmen

Die Sätze der Kapitel 7 bis 10 gelten unter drei ausdrücklichen Annahmen.

Definition 6.5 (Annahmen)**. A1 (statisches Dateisystem).** FS und E ändern sich während einer Operation nicht. **A2 (Adaptervertrag).** Der Adapter erfüllt den Vertrag von Definition 6.1; außerdem terminieren seine Operationen. **A3 (sequentielle Ausführung).** Operationen laufen nacheinander (nicht verschränkt); die Verschränkung untersucht Abschnitt 9.9 getrennt.

```
OPEN(n, spec, rho): # loader.py _open
  (f, explicit) <- RESOLVE_NAME(n) # NOT_FOUND bei ungueltigem Namen
  c <- LOCATE(f, explicit, rho) # NOT_FOUND | ACCESS_DENIED
  k <- REALPATH(c)
  if k in S: raise CYCLE(n)
  if k in dom(L):
     CHECK_RANGE(mu(L[k]), spec) # DEPENDENCY_MISSING
     rc(L[k]) += 1; return k
  push k on S
  try: nd <- LOAD(k, spec)
  finally: pop S
  L[k] <- nd; return k

LOAD(k, spec): # loader.py _load
  x <- READ(k) # ACCESS_DENIED | INTERNAL
  ct <- UNPACK(x) # INVALID_CONTAINER
  if abi(ct) not in [l, u]: raise ABI_MISMATCH
  p <- PAYLOAD_FOR(ct, pi, alpha) # ARCH_MISMATCH | INVALID_CONTAINER
  CHECK_RANGE(mu(ct), spec) # DEPENDENCY_MISSING
  opened <- []; nu <- none
  try:
    for d in DEPS(mu(ct)) in Namensordnung:
       opened.append( OPEN_DEP(d, k) ) # NOT_FOUND wird DEPENDENCY_MISSING
    nu <- map(p, name(mu(ct))) # OSError wird INTERNAL
    if init(mu(ct)) defined: RUN_INIT # INIT_FAILED bei Fehler oder Wert !=
        ↪ 0
    return (mu(ct), opened, rc = 1, nu)
  on SLError:
    if nu != none: unmap(nu)
    for h in opened: RELEASE(h)
    re-raise

RELEASE(k): # loader.py _release
  rc(L[k]) -= 1; if rc(L[k]) > 0: return
  nd <- L[k]; remove k from dom(L)
  if fini(mu(nd)) defined: FINI (fehlendes Symbol wird ignoriert)
  unmap(nu(nd))
  for h in d(nd): RELEASE(h)

CLOSE(k): REQUIRE_OPEN(k); RELEASE(k)
SYMBOL(k,s): REQUIRE_OPEN(k); if s not in exports(mu(L[k])): raise
    ↪ SYMBOL_NOT_FOUND
          return lookup(nu(L[k]), s)
```

Abbildung 6.1: Modell von `open`, `close` und `symbol` mit ihren Hilfsoperationen. Die Kommentare nennen die Fehler in der Reihenfolge, in der der Quelltext sie prüft (Satz 9.6).

Ohne A3 ist Satz 9.3 falsch, nicht weil die Sperre fehlt (Operationen `open` und `close` sind durch eine Sperre geschützt, Satz 9.10), sondern weil `symbol`

und `info` es nicht sind.

6.4 Der Ablauf von `sl_open`

Abbildung 6.2 zeigt Abbildung 6.1 als Ablauf mit den Fehlern am Rand: die neun Schritte des Proposals (Abschnitt 5.3 und Anhang B des Proposals) und wo der Quelltext sie ausführt.

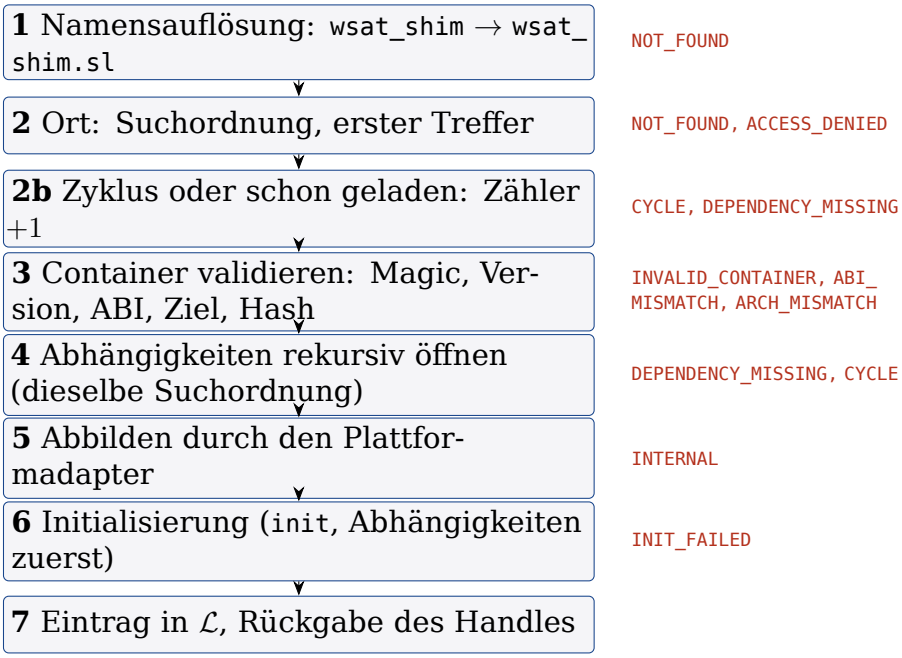


Abbildung 6.2: Ablauf von `open`. Rechts stehen die Fehler, die der jeweilige Schritt auslösen kann. Das Proposal nennt acht Schritte plus Freigabe (Abschnitt 5.3 des Proposals); der Quelltext führt Zyklus- und Zählerprüfung (2b) vor der Containerprüfung aus.

6.5 Zuordnung von Modell und Quelltext

Tabelle 6.1 ordnet die Größen des Modells den Quelltextstellen zu. Sie ist die Rückführung der Beweise auf den Quelltext (Art *aus dem Quelltext gelesen*).

Tabelle 6.1: Modell und Quelltext. Zeilennummern beziehen sich auf den untersuchten Stand.

Modellgröße	Quelltext	Bemerkung
Konfiguration C (Def. 6.2)	`loader.py`, Z. 30–44	Parameter von `Loader.__init__`
Zustand $\Gamma = (\mathcal{L}, S)$	Z. 41–42	`_loaded`, `_loading`
Knoten $(\mu, d, \mathrm{rc}, \nu)$	Z. 17–26	Klasse `Handle`

Modellgröße	Quelltext	Bemerkung
Open	Z. 83–102	`_open`
Load	Z. 114–142	`_load`
Release	Z. 165–178	`_release`
OpenDep	Z. 144–152	NOT_FOUND → DEPENDENCY_MISSING
RunInit	Z. 154–163	`init` liefert 0 bei Erfolg
Adapter (Def. 6.1)	`adapters.py`	`system_dirs`, `map_payload`, `lookup`, `unmap`
Enc, Dec, cand	`search.py`, Z. 14–110	`split_sl_path`, `candidates`
Locate	`search.py`, Z. 130–145	`locate`
pack, unpack	`container.py`, Z. 54–98	
Versionsbereich	`metadata.py`, Z. 13–52	`satisfies`

Kapitel 7

Namensauflösung und Suchordnung

Die Schritte 1 und 2 der Ladeprozedur bestimmen, *welche Datei* gemeint ist. Das Proposal verlangt dafür eine feste Reihenfolge und schließt das Arbeitsverzeichnis sowie `PATH`, `LD_LIBRARY_PATH` und `DYLD_LIBRARY_PATH` ausdrücklich aus (Abschnitt 5.3 und 5.4 des Proposals). Dieses Kapitel beweist, was daraus folgt, und beweist an einer Stelle, dass die Implementierung hinter dem Proposal zurückbleibt: der Kodierung von `SL_PATH`.

7.1 Namensauflösung

Definition 7.1 (Logischer Name, Auflösung)**.** LName sei die Menge der Wörter b mit $b \in$ [A-Za-z_][A-Za-z0-9_]* (Bezeichner). Die Funktion resolve $: \Sigma^* \to (\mathsf{Path} \times \{0,1\}) \cup \{\text{NOT_FOUND}\}$ ist definiert durch: Enthält n das Zeichen `/` oder `\`, so ist resolve$(n) = (n', 1)$ mit $n' = n$, falls n auf `.sl` endet, sonst $n' = n \cdot$ `.sl` (*explizit*). Sonst sei $b = n$ ohne ein etwaiges Endstück `.sl`; ist $b \in$ LName, so ist resolve$(n) = (b \cdot$ `.sl`$, 0)$ (*logisch*), andernfalls NOT_FOUND.

Satz 7.1 (Namensauflösung)**.** (a) resolve ist total. (b) Für logische Namen gilt resolve$(n) = (b \cdot$ `.sl`$, 0)$ mit $b \in$ LName; die Datei enthält also außer dem Endstück `.sl` keinen Punkt und kein Pfadtrennzeichen. (c) resolve$(b) =$ resolve$(b \cdot$ `.sl`$)$ für alle $b \in$ LName. (d) $b \mapsto b \cdot$ `.sl` ist injektiv auf LName: verschiedene logische Namen bezeichnen verschiedene Dateien. (e) Jeder Name, der außer einem Endstück `.sl` einen Punkt enthält, etwa `libwsat.so`, `wsat.dll` oder `wsat.1.sl`, wird mit NOT_FOUND abgewiesen, sofern er kein Pfadtrennzeichen enthält.

Proof. (a) Die Definition unterscheidet zwei vollständige Fälle (Trennzeichen enthalten oder nicht) und im zweiten Fall die Zugehörigkeit zu LName; jeder Zweig liefert ein Ergebnis oder NOT_FOUND. (b) Ein Wort aus LName enthält nur Buchstaben, Ziffern und _, also weder Punkt noch Trennzeichen. (c) Hat b kein Endstück `.sl`, so ergibt b selbst $b \cdot$ `.sl`, und $b \cdot$ `.sl` ergibt nach Streichen des Endstücks wieder b. Endet $b \in$ LName auf `.sl`, enthielte es einen Punkt, im Widerspruch zu $b \in$ LName; der Fall tritt nicht auf. (d) $b \cdot$ `.sl` $= b' \cdot$ `.sl` gilt nur für $b = b'$ (Streichen desselben Endstücks). (e) Enthält

n einen Punkt außerhalb eines Endstücks `.sl`, so enthält b (nach Streichen des Endstücks) einen Punkt, also $b \notin \mathsf{LName}$. □

Satz 7.1 ist die formale Fassung von Ziel Z1 und der Aussage des Proposals, ein Name enthalte weder Plattformpräfixe (`lib`) noch Suffixe (`.dll`, `.so.1`) (Abschnitt 5.3, Schritt 1). Der Quelltext lehnt `libwsat.so` ab, nimmt aber `libwsat` an: Ein logischer Name `libwsat` ist erlaubt, weil er ein Bezeichner ist; er bezeichnet die Datei `libwsat.sl`, nicht eine Plattformbibliothek. Der Bindestrich, wie in `wsat-shim`, ist dagegen *kein* Bezeichnerzeichen; der Name wird abgewiesen. Ein solcher Name ist in der Praxis häufig, das Proposal erwähnt ihn nicht.

7.2 Die Kodierung von `SL_PATH`

Das Proposal verlangt, dass die Trennung der Pfadliste auf allen Systemen gleich ist, und nennt zwei Möglichkeiten: ein einziges Trennzeichen mit festgelegter Maskierung oder eine Listen-API (Abschnitt 5.4 und 14 des Proposals). Der Quelltext (`split_sl_path`) verfolgt einen dritten Weg.

Definition 7.2 (Kodierung und Dekodierung)**.** Für einen Eintrag e sei $\mathrm{esc}(e)$ das Wort, das aus e durch Ersetzen jedes `\` durch `\\` und jedes `:` durch `\:` entsteht. Für eine Liste $L = (e_1, \dots, e_k)$ von Einträgen ist

$$\mathrm{Enc}(L) = \mathrm{esc}(e_1) \cdot \texttt{:} \cdot \mathrm{esc}(e_2) \cdot \texttt{:} \cdots \texttt{:} \cdot \mathrm{esc}(e_k).$$

$\mathrm{Dec} : \Sigma^* \to (\Sigma^*)^*$ ist die Funktion von `split_sl_path`: Für $w = \varepsilon$ ist $\mathrm{Dec}(w) = ()$. Enthält w ein `;`, so ist $\mathrm{Dec}(w)$ die Liste der nichtleeren Teilwörter zwischen den `;` (ohne jede Maskierung). Sonst wird w von links nach rechts gelesen, mit Position i (von 0 an), und dabei ein Eintrag aufgebaut: (i) Ist $i = 1$, $w[0]$ ein Buchstabe, $w[1] =$ `:` und ($|w| = 2$ oder $w[2] \in \{\texttt{/}, \texttt{\}\}$), so wird das `:` an den Eintrag angehängt (*Laufwerkspräfix*). (ii) `\:` liefert `:`, `\\` liefert `\` (zwei Zeichen werden gelesen). (iii) Ein `:` beendet den Eintrag; ein leerer Eintrag wird verworfen. (iv) Jedes andere Zeichen, auch ein einzelnes `\`, wird übernommen.

Das Kodierungsschema entspricht dem des Proposals, soweit es POSIX-Pfade betrifft (Doppelpunkt, Maskierung mit `\`). Es kommt ein zweiter Modus (Semikolon) und eine Heuristik (Laufwerkspräfix) hinzu. Satz 7.2 sagt exakt, wann das gut geht und wann nicht.

Definition 7.3 (Zulässige Listen, Laufwerksfall)**.** $\mathcal{L}_{\mathrm{ok}}$ sei die Menge der Listen nichtleerer Einträge ohne `;`. Für $L \in \mathcal{L}_{\mathrm{ok}}$ gelte $\mathrm{Drive}(L)$, wenn $|L| \geq 2$, e_1 aus genau einem Buchstaben besteht und e_2 mit `/`, `\` oder `:` beginnt.

Satz 7.2 (Rundreise der `SL_PATH`-Kodierung)**.** (a) Für $L \in \mathcal{L}_{\mathrm{ok}}$ gilt $\mathrm{Dec}(\mathrm{Enc}(L)) = L$ genau dann, wenn nicht $\mathrm{Drive}(L)$. (b) Jedes Wort w hat ein Bild $\mathrm{Dec}(w)$ aus nichtleeren Einträgen, und kein Eintrag im Bild von Dec enthält ein `;`. Eine Liste mit einem Eintrag, der `;` enthält, ist also nicht als `SL_PATH` darstellbar. (c) Das Laufwerkspräfix gilt nur für den ersten Eintrag: $\mathrm{Dec}(\texttt{C:\x:D:\y}) = (\texttt{C:\x}, \texttt{D}, \texttt{\y})$. (d) Enthält w ein `;`, so wird `\` nicht als Maskierung gelesen: $\mathrm{Dec}(\texttt{/a:b;/c}) = (\texttt{/a:b}, \texttt{/c})$.

Proof. (a) *Schritt 1: ohne Sonderregeln.* Sei Dec_0 die Dekodierung ohne Regel (i) und ohne Semikolon-Modus. In $\mathrm{Enc}(L)$ steht jedes `\` entweder in einem Paar `\\` (für ein `\` des Eintrags) oder in einem Paar `\:` (für ein `:`); ein `:` ohne vorangehendes `\` ist stets ein Trenner. Die Lesung von links nach rechts nimmt an jeder Stelle, an der `\` steht, das folgende Zeichen mit (zwei Zeichen), und stellt dadurch dieselbe Zerlegung in Zeichen und Trenner her, aus der Enc das Wort gebaut hat. Genauer: Induktion über $|L|$. Für $|L| = 1$ liefert $\mathrm{Dec}_0(\mathrm{esc}(e_1)) = e_1$, weil jedes Zeichen von e_1 als ein Block (`\\`, `\:` oder das Zeichen selbst) kodiert ist und die Blöcke von links nach rechts eindeutig gelesen werden (kein Block ist echtes Präfix eines anderen, und jedes `\` beginnt einen Zwei-Zeichen-Block). Für $|L| > 1$ endet die Lesung des ersten Eintrags am ersten unmaskierten `:`, danach liest sie $\mathrm{Enc}((e_2, \ldots, e_k))$, und die Induktionsannahme greift. Die Einträge sind nichtleer, werden also nicht verworfen.

Schritt 2: Regel (i). In $\mathrm{Enc}(L)$ mit $L \in \mathcal{L}_{\mathrm{ok}}$ ist kein `;` vorhanden, der Semikolon-Modus greift nicht. Regel (i) greift genau dann, wenn $w[0]$ ein Buchstabe ist, $w[1] =$ `:` und ($|w| = 2$ oder $w[2] \in \{$`/`, `\`$\}$). Ist $w[0]$ ein Buchstabe, so ist $w[0]$ kein Maskierungszeichen, also steht $w[1]$ an einer Blockgrenze; $w[1] =$ `:` bedeutet dann einen unmaskierten Doppelpunkt, das heißt einen Trenner, und $e_1 = w[0]$ besteht aus genau einem Buchstaben; es gibt also $|L| \geq 2$. Dass $|w| = 2$ ist, ist ausgeschlossen, weil e_2 nichtleer ist. $w[2]$ ist das erste Zeichen von $\mathrm{esc}(e_2)$; es ist `/`, wenn e_2 mit `/` beginnt, und `\`, wenn e_2 mit `\` oder `:` beginnt (kodiert als `\\` beziehungsweise `\:`). Anderenfalls ist $w[2]$ ein anderes Zeichen. Also greift (i) genau bei $\mathrm{Drive}(L)$.

Schluss. Ohne $\mathrm{Drive}(L)$ stimmen Dec und Dec_0 auf $\mathrm{Enc}(L)$ überein, und Schritt 1 liefert $\mathrm{Dec}(\mathrm{Enc}(L)) = L$. Gilt $\mathrm{Drive}(L)$, so hängt (i) das `:` an den ersten Eintrag an, und der erste Eintrag von $\mathrm{Dec}(\mathrm{Enc}(L))$ beginnt mit $e_1 \cdot$ `:` und ist länger als e_1, also $\neq e_1$.

(b) Ist `;` $\in w$, sind die Einträge Teilwörter zwischen den `;` und enthalten kein `;`; leere werden verworfen. Ist `;` $\notin w$, besteht jeder Eintrag aus Zeichen von w oder, nach Regel (ii), aus `:` oder `\`, enthält also kein `;`. (c) und (d) sind Auswertung der Definition. (c): Position 1 ist das `:` nach `C` mit $w[2] =$ `\`, Regel (i) greift; das `:` nach `x` ist ein Trenner, ebenso das `:` nach `D`, denn (i) greift nur bei $i = 1$. (d) w enthält `;`, also wird nur an `;` getrennt. □

Korollar 7.1 (Befunde **B1** und **B2**)**.** (a) Es gibt Listen $L \in \mathcal{L}_{\mathrm{ok}}$ mit $\mathrm{Dec}(\mathrm{Enc}(L)) \neq L$, etwa $L =$ (`a`, `/opt/x`): $\mathrm{Dec}($`a:/opt/x`$) =$ (`a:/opt/x`). In diesem Fall geht der Eintrag `/opt/x` verloren. (b) Kein Wort w ergibt einen Eintrag mit `;`. (c) Die Kodierung benutzt für die Liste zwei verschiedene Syntaxen mit verschiedener Maskierung.

Proof. (a) $\mathrm{Drive}(L)$ gilt, also scheitert die Rundreise nach Satz 7.2(a). (b) Satz 7.2(b). (c) Satz 7.2(d) gegenüber der Maskierung in (a). □

Die praktische Tragweite ist gering, aber die Ursache ist grundsätzlich: Die Heuristik des Laufwerkspräfixes entscheidet über die Struktur der Liste aufgrund der Form der Einträge. Eine Kodierung, deren Rundreise von der Form der Einträge abhängt, ist keine Kodierung im Sinne des Proposals (“single separator, with a defined escaping rule”). Abschnitt 14.1 gibt eine Verbesserung an, die die Rundreise für *alle* Listen herstellt.

7.3 Die Kandidatenliste

Definition 7.4 (Kandidatenliste)**.** Zu Konfiguration C, aufgelöstem Namen (f, x) und Aufruferverzeichnis $\rho \in \mathsf{Path} \cup \{\bot\}$ ist $\mathrm{cand}_C(f, x, \rho)$ die Liste

$$\begin{cases} (f) & \text{falls } x = 1, \\ \mathrm{dedup}\big(K_2 \cdot K_3 \cdot K_4 \cdot K_5\big) & \text{sonst,} \end{cases}$$

mit $K_2 = (\mathrm{join}(d, f) \mid d \in \mathrm{Dec}(E[\texttt{SL_PATH}]),\ \mathrm{isabs}(d))$ für $h = 0$ und $K_2 = ()$ für $h = 1$; $K_3 = (\mathrm{join}(\rho, f), \mathrm{join}(\rho, \texttt{sl}, f))$ falls $\rho \neq \bot$ (sonst $()$); $K_4 = (\mathrm{join}(a, f))$ falls $a \neq \bot$ (sonst $()$); $K_5 = (\mathrm{join}(d, f) \mid d \in D_{\mathfrak{A}}(E))$. Dabei ist $E[\texttt{SL_PATH}] = \varepsilon$, wenn `SL_PATH` nicht gesetzt ist. Die fünf Klassen heißen *explizit* (K_1), *Umgebung* (K_2), *Aufrufer* (K_3), *Anwendung* (K_4), *System* (K_5).

Definition 7.5 (Locate)**.** $\mathrm{locate}_C(n, \rho)$ ist für $\mathrm{resolve}(n) = (f, x)$ und $\mathrm{cand}_C(f, x, \rho) = (c_1, \ldots, c_m)$ wie folgt definiert: Ist i minimal mit $c_i \in \mathrm{dom}\,\mathsf{FS}$, so ist das Ergebnis, sofern $h = 0$ oder das Verzeichnis von c_i nicht weltbeschreibbar ist, das Paar $(c_i, (c_1, \ldots, c_i))$; ist $h = 1$ und das Verzeichnis weltbeschreibbar, so ist es $\mathrm{Err}(\mathrm{ACCESS_DENIED})$ mit durchsuchten Pfaden $(c_1, \ldots, c_i)$. Gibt es kein solches i, ist es $\mathrm{Err}(\mathrm{NOT_FOUND})$ mit durchsuchten Pfaden $(c_1, \ldots, c_m)$. Ist $\mathrm{resolve}(n) = \mathrm{NOT_FOUND}$, ist es $\mathrm{Err}(\mathrm{NOT_FOUND})$.

Satz 7.3 (Erster Treffer)**.** Sei $h = 0$. (a) $\mathrm{locate}_C(n, \rho)$ ist eine Funktion von $(\mathsf{FS}, E, C, n, \rho)$ allein. (b) Ist ein Kandidat eine Datei, so liefert locate den ersten solchen Kandidaten der Liste, und die Liste der durchsuchten Pfade ist das Präfix der Kandidatenliste bis zu ihm. (c) Ist kein Kandidat eine Datei, liefert locate *NOT_FOUND* und die vollständige Kandidatenliste. (d) Liegt ein Treffer in der Klasse K_j, so liegt in keiner Klasse K_i mit $i < j$ ein Treffer. (e) Die Streichung gleicher Kandidaten ändert den Treffer nicht.

Proof. (a) Alle Bestandteile der Definition (Dekodierung, Verbindung, Systemverzeichnisse, Dateitest) sind Funktionen dieser Größen. (b) und (c) sind die Definition 7.5 mit minimalem i. (d) Die Liste ist die Verkettung $K_2 \cdot K_3 \cdot K_4 \cdot K_5$; ein Treffer in K_j ist der erste Treffer, also gibt es vorher in der Verkettung keinen, insbesondere in keiner Klasse mit kleinerem Index. (e) Lemma 2.1 mit $P(c) = (c \in \mathrm{dom}\,\mathsf{FS})$. □

Korollar 7.2 (Stabilität)**.** Sei c_i der erste Treffer. Ändert man FS so, dass sich nur Kandidaten nach c_i ändern (Dateien hinzufügen oder entfernen), bleibt das Ergebnis von locate gleich.

Proof. Satz 7.3(b): Das Ergebnis hängt nur vom Präfix $(c_1, \ldots, c_i)$ ab. □

Satz 7.4 (Kein implizites Arbeitsverzeichnis)**.** Sei n ein logischer Name. Sind r und a absolut oder $\bot$ und ist jedes Element von $D_{\mathfrak{A}}(E)$ absolut, so ist jeder Kandidat von $\mathrm{cand}_C(f, 0, \rho)$ absolut (für $\rho \in \{r\} \cup \{\text{dirname eines aufgelösten Pfads}\}$), und $\mathrm{locate}_C(n, \rho)$ ist unabhängig vom Arbeitsverzeichnis w. Die Variablen `PATH`, `LD_LIBRARY_PATH` und `DYLD_LIBRARY_PATH` gehen in keine Kandidatenliste ein; von E gehen nur `SL_PATH` (Kern) und, im Windows-Adapter, `ProgramData` und `ProgramFiles` ein.

Proof. Jeder Kandidat hat die Form $\mathrm{join}(d, f)$ oder $\mathrm{join}(d, \texttt{sl}, f)$ mit einem Verzeichnis d aus K_2 (absolut nach Definition, wegen des Filters isabs), K_3 ($d = \rho$), K_4 ($d = a$) oder K_5 ($d \in D_{\mathfrak{A}}(E)$, absolut nach Voraussetzung). Der Name f ist logisch, enthält also kein Pfadtrennzeichen (Satz 7.1(b)); die Verbindung join hängt ihn an d an und bleibt unter d, ein absolutes d ergibt also einen absoluten Pfad. Nach Lemma 2.2 ist FS auf absoluten Pfaden unabhängig von w. Der Rest ist Lesen des Quelltexts: In `search.py` wird E nur in `env.get("SL_PATH")` gelesen und an den Adapter weitergegeben, im Windows-Adapter nur `ProgramData` und `ProgramFiles`. □

Bemerkung 7.1 (Notwendigkeit der Voraussetzung)**.** Die Voraussetzung ist nötig. Ist r oder a relativ, so ist der Kandidat relativ, und locate hängt von w ab. Die Prüfung zeigt: `candidates("x.sl", False, requester_dir="req", app_dir="app")` liefert `req/x.sl` und `app/x.sl` als Kandidaten (geprüft, Abschnitt 13.2). Auch ist der Aufruf mit explizitem relativem Pfad per Definition abhängig vom Arbeitsverzeichnis: `sl.open("lib/x")` ist ein expliziter Pfad. Das Proposal sagt, das Arbeitsverzeichnis werde *implizit* nie durchsucht; der Satz gibt die genaue Bedingung an. Der Quelltext verlangt Absolutheit nur für `SL_PATH`-Einträge, nicht für r und a; eine Prüfung dort wäre eine einfache Verbesserung (Abschnitt 14.1).

Satz 7.5 (Gehärteter Modus)**.** Sei $h = 1$. (a) cand_C ist unabhängig von $E[\texttt{SL_PATH}]$. (b) Liefert locate einen Pfad c, so ist das Verzeichnis von c nicht weltbeschreibbar (POSIX: Bit `S_IWOTH` des Verzeichnisses nicht gesetzt). (c) locate *bricht ab*, statt weiterzusuchen: Ist der erste Treffer in einem weltbeschreibbaren Verzeichnis, ist das Ergebnis ACCESS_DENIED, auch wenn ein späterer Kandidat zulässig wäre.

Proof. (a) Für $h = 1$ ist $K_2 = ()$ nach Definition 7.4. (b) und (c) folgen aus Definition 7.5: Nur im Fall “nicht weltbeschreibbar” wird $(c_i, \ldots)$ geliefert, sonst ACCESS_DENIED, und zwar für den *ersten* Treffer, ohne Fortsetzung der Suche. □

Die Eigenschaft (c) ist die Forderung “fail closed” des Proposals (Abschnitt 10 des Proposals). Der Satz sagt auch, was der gehärtete Modus *nicht* prüft: das Eigentum am Verzeichnis, die Rechte der Datei selbst, die Rechte der übergeordneten Verzeichnisse und, unter Windows, überhaupt etwas (Befund **B4**).

Bemerkung 7.2 (Eine Lücke der Härtung)**.** Liegt die Datei in $/a/b/x.sl$, das Verzeichnis $/a/b$ ist nicht weltbeschreibbar, aber $/a$ ist es (ohne Sticky-Bit), so kann ein Angreifer mit Schreibrecht auf $/a$ das Verzeichnis $/a/b$ umbenennen und ein eigenes Verzeichnis gleichen Namens mit eigener `x.sl` anlegen; dieses ist nicht weltbeschreibbar, gehört aber dem Angreifer. Der gehärtete Modus lässt die Datei zu. Zur Verhinderung genügt Eigentümer- und Rechtsprüfung entlang der gesamten Pfadkette bis zur Wurzel.

Kapitel 8

Container, Metadaten und Versionsbereiche

Der Container ist der Teil von `.sl`, der *Daten* festlegt: ein Metadatenkopf und je Plattform und Architektur ein Payload (Abschnitt 6 des Proposals). Dieses Kapitel beweist, dass Schreiben und Lesen zueinander passen, dass die Hashes Veränderungen der Payloads erkennen, und dass sie Veränderungen des Kopfes *nicht* erkennen. Es beweist ferner, dass Versionsbereiche konvexe Mengen sind und sich ihre Verträglichkeit linear entscheiden lässt.

8.1 Metadaten

Definition 8.1 (Gültige Metadaten)**.** Ein Metadatensatz $\mu = (n, v, \mathrm{abi}, T, \Delta, X, \Theta, \mathrm{rt}, \iota, \phi)$ besteht aus Name $n \in \mathsf{LName}$, Version $v \in \mathsf{V}$ (als Text `x.y.z`), ABI-Version $\mathrm{abi} \in \mathbb{N}_{\geq 1}$, einer nichtleeren Liste T paarweise verschiedener Ziele (π, α) mit π, α aus den bekannten Mengen, einer Abbildung Δ von Namen aus LName auf Versionsbereiche (Abhängigkeiten), einer Liste X paarweise verschiedener Bezeichner (Exporte), einer Abbildung Θ von Exportnamen auf C-Deklarationen (Signaturen; $\mathrm{dom}\,\Theta \subseteq X$), $\mathrm{rt} \in \{\mathrm{embedded}, \mathrm{external}\}$ und optionalen Bezeichnern ι, ϕ (`init`, `fini`). Die Prüfungen sind die von `Metadata.__post_init__`.

Das Proposal fordert als Felder `magic`, `container_version`, `name`, `version`, `abi_version`, `targets`, `dependencies`, `exports`, `runtime` und `init`/`fini` (Abschnitt 6.2). Die Implementierung hat dieselben Felder und zusätzlich `signatures`; `magic` und `container_version` stehen im Kopf des Containers statt in den Metadaten.

Lemma 8.1 (Ordnung der Abhängigkeiten)**.** Für jeden Metadatensatz, der aus einem Container gelesen wurde, durchläuft `dependencies.items()` die Namen in aufsteigender Ordnung der Zeichenketten (nach Codepunkten).

Proof. `to_dict` bildet `dict(sorted(self.dependencies.items()))`, also eine Abbildung mit aufsteigend sortierten Schlüsseln; `json.dumps(…, sort_keys=True)` sortiert alle Schlüssel erneut, die der Abhängigkeiten ebenso. `json.loads` erzeugt Abbildungen in der Reihenfolge des Textes, und `from_`

`dict` kopiert sie mit `dict(…)`, was die Reihenfolge erhält. Die Namen sind eindeutig, die Sortierung nach Paaren (Name, Bereich) ist daher eine Sortierung nach Namen. □

Das Lemma macht die Ladereihenfolge unabhängig von der Reihenfolge, in der der Autor die Abhängigkeiten aufschrieb. Sie ist deterministisch und alphabetisch; Satz 9.5 baut darauf auf.

8.2 Versionsbereiche

Definition 8.2 (Bereich)**.** Eine *Beschränkung* ist ein Paar $(\bowtie, v)$ mit $\bowtie \in \{\geq, \leq, =, >, <\}$ (im Text `>=`, `<=`, `==`, `>`, `<`) und $v \in \mathsf{V}$. Ein *Bereich* spec ist eine Liste von Beschränkungen (Text: durch Kommata getrennt, jeweils `op x.y.z`); der Text $*$ oder der leere Text ist die leere Liste. Die *Erfüllungsmenge* ist

$$S(\text{spec}) = \{x \in \mathsf{V} \mid \forall (\bowtie, v) \in \text{spec} : \; x \bowtie v\}.$$

Dabei ist die Versionsangabe stets dreiteilig (`x.y.z`); jede andere Form ist ein Fehler vom Status *INVALID_CONTAINER*.

Beispiel 8.1 (Das Proposal ist strenger als es klingt)**.** Anhang A des Proposals nennt `version = ">=1.2,<2"`. Die Implementierung lehnt diesen Bereich ab, weil `1.2` und `2` keine dreiteiligen Versionen sind (`satisfies("1.5.0", ">=1.2,<2")` wirft INVALID_CONTAINER, geprüft). Gleiches gilt für `1.0.0` ohne Operator und `>=1.0`. Das ist der Befund **B3**.

Satz 8.1 (Konvexität)**.** Für jeden Bereich ist $S(\text{spec})$ konvex in $(\mathsf{V}, \leq_{\text{lex}})$.

Proof. Jede einzelne Menge $S(\bowtie, v)$ ist konvex: Seien $x, y \in S(\bowtie, v)$ und $x \leq z \leq y$. Für $\bowtie = \geq$: $z \geq x \geq v$. Für $\leq$: $z \leq y \leq v$. Für $>$: $z \geq x > v$. Für $<$: $z \leq y < v$. Für $=$: $x = y = v$, also $v \leq z \leq v$ und $z = v$. $S(\text{spec})$ ist der Durchschnitt endlich vieler konvexer Mengen (für die leere Liste ist es V), und der Durchschnitt konvexer Mengen ist konvex: Sind x, y in jeder, so liegt z in jeder. □

Die Konvexität ist die Begründung, warum eine Liste von Beschränkungen als *Intervall* zu lesen ist. Sie schließt aus, dass ein Bereich eine Lücke hat; deshalb gibt es kein „$\neq$“ (der Quelltext unterstützt es nicht). Sie hat noch eine praktische Folge für die Verträglichkeit.

Satz 8.2 (Verträglichkeit und kleinste Version)**.** (a) $S(\text{spec}_1 \cdot \text{spec}_2) = S(\text{spec}_1) \cap S(\text{spec}_2)$. (b) Sei $\text{lo}(\text{spec})$ das Maximum der Werte v der Beschränkungen $(\geq, v)$ und $(=, v)$ sowie der Werte $v^+ = (a, b, c+1)$ der Beschränkungen $(>, v)$ mit $v = (a, b, c)$ (ist keine solche Beschränkung vorhanden, sei $\text{lo} = (0, 0, 0)$). Dann ist $S(\text{spec}) \neq \emptyset$ genau dann, wenn $\text{lo}(\text{spec}) \in S(\text{spec})$, und dann ist $\text{lo}(\text{spec}) = \min S(\text{spec})$. (c) Die Verträglichkeit zweier Bereiche, also $S(\text{spec}_1) \cap S(\text{spec}_2) \neq \emptyset$, ist in Zeit $O(|\text{spec}_1| + |\text{spec}_2|)$ entscheidbar.

Proof. (a) Die Bedingung “alle Beschränkungen gelten” ist für die verkettete Liste die Konjunktion beider.

(b) *Der Wert* $v^+ = (a, b, c+1)$ *ist der unmittelbare Nachfolger von* $v = (a, b, c)$: Ein x mit $v < x < v^+$ hätte erste Komponenten (a, b) (sonst wäre $x \le v$ oder $x \ge v^+$ nach der lexikographischen Ordnung) und dritte Komponente $c < x_3 < c+1$, was in $\mathbb{N}$ nicht möglich ist. Also gilt $x > v \iff x \ge v^+$.

Jede Beschränkung der Form $(\ge, v)$, $(=, v)$ oder $(>, v)$ verlangt $x \ge \mathrm{lo}_j$ mit $\mathrm{lo}_j \in \{v, v^+\}$; zusammen verlangen sie $x \ge \mathrm{lo}(\mathrm{spec})$. Die übrigen Beschränkungen (nach oben) sind abwärts abgeschlossen: Gilt $y \le x$ und erfüllt x sie, so erfüllt y sie ($(\le, v)$: $y \le x \le v$; $(<, v)$: $y \le x < v$; $(=, v)$ verlangt auch $x \le v$ und ist in beiden Rollen enthalten). Ist $S \neq \emptyset$, so gibt es $x \in S$, und $x \ge \mathrm{lo}$. Weil $\mathrm{lo} \le x$ und die Beschränkungen nach oben abwärts abgeschlossen sind, erfüllt lo sie; die Beschränkungen nach unten erfüllt lo nach Konstruktion. Also $\mathrm{lo} \in S$. Umgekehrt folgt aus $\mathrm{lo} \in S$ sofort $S \neq \emptyset$. lo ist kleinstes Element: Jedes $x \in S$ erfüllt $x \ge \mathrm{lo}$.

(c) Nach (a) ist die Verträglichkeit die Nichtleerheit von $S(\mathrm{spec}_1 \cdot \mathrm{spec}_2)$. Nach (b) genügt die Berechnung von lo (ein Durchlauf) und die Prüfung, ob lo jede Beschränkung erfüllt (ein zweiter). □

Der Satz erlaubt es, einen Konflikt *vor* dem Laden zu entscheiden, etwa wenn zwei Abhängigkeitskanten auf dieselbe Bibliothek unvereinbare Bereiche verlangen. Der Loader tut das nicht von sich aus; er prüft jede Kante beim Öffnen (Satz 8.8). Die Verbindung zur SAT-basierten Auflösung beschreibt der Ausblick (Abschnitt 17.8).

8.3 Das Containerformat

Definition 8.3 (Container)**.** Ein *Container* für Metadaten μ und eine Abbildung $P : T \to \mathsf{Bytes}$ ist die Bytefolge $\mathrm{pack}(\mu, P) = \mathsf{H} \cdot \mathsf{M} \cdot \mathsf{T} \cdot \mathsf{B}$ mit:

- Kopf H (18 Bytes): Magic `89 53 4C 49 42 0D 0A 1A` (acht Bytes), Containerversion 1 (`u16`), Länge $|\mathsf{M}|$ (`u32`), Anzahl $|T|$ (`u32`), alle Zahlen kleinendig ohne Auffüllung.
- Metadaten M: der JSON-Text von μ [7] mit sortierten Schlüsseln, als ASCII.
- Tabelle T: je Ziel in aufsteigender Ordnung ein Eintrag von 80 Bytes: Plattform (16 Bytes, mit Nullen aufgefüllt), Architektur (16), Offset (`u64`), Länge (`u64`), SHA-256 des Payloads (32).
- Daten B: die Payloads, hintereinander, in der Ordnung der Tabelle.

pack ist definiert, wenn $\mathrm{dom}\, P$ gleich der Menge der Ziele von μ ist. unpack : $\mathsf{Bytes} \rightharpoonup (\mu, \mathsf{Tab}, \mathsf{B})$ prüft der Reihe nach: Länge ≥ 18; Magic; Containerversion; $18 + |\mathsf{M}| + 80 \cdot \text{Anzahl} \le |x|$; lesbares JSON; gültige Metadaten nach Definition 8.1; für jeden Tabelleneintrag ein gültiges Ziel und Offset + Länge $\le |\mathsf{B}|$; die Menge der Ziele der Tabelle stimmt mit der der Metadaten überein. Jeder Fehler ist ein Fehler vom Status INVALID_CONTAINER. *Payload_for*

(t) liefert die Bytes des Eintrags für t nach Prüfung des SHA-256, oder den Fehler ARCH_MISMATCH (kein Eintrag) beziehungsweise INVALID_CONTAINER (Hash falsch).

Satz 8.3 (Rundreise)**.** Sei μ gültig und $\operatorname{dom} P = T(\mu)$. Dann gelingt unpack(pack(μ, P)), die gelesenen Metadaten sind gleich μ (einschließlich der Reihenfolge von Zielen und Exporten), und payload_for$(t) = P(t)$ für jedes $t \in T(\mu)$.

Proof. Metadaten. to_dict(μ) hat die Schlüssel `name`, `version`, `abi_version`, `targets` (Liste von Paaren in der Reihenfolge von T), `dependencies`, `exports` (in der Reihenfolge von X), `signatures`, `runtime`, `init`, `fini`; alle Werte sind JSON-Werte (Zahlen, Zeichenketten, Listen, Abbildungen, `null`), und `json.loads`$\circ$`json.dumps` ist auf ihnen die Identität. `from_dict` akzeptiert genau diese Schlüssel und baut μ' komponentenweise nach: T als Tupel der Ziele in derselben Reihenfolge, Δ und Θ als Abbildungen gleichen Inhalts (Gleichheit von Abbildungen ist unabhängig von der Reihenfolge), X als Tupel in derselben Reihenfolge, `null` als `None`. Da μ gültig ist, besteht μ' die Prüfungen, und $\mu' = \mu$.

Kopf und Tabelle. Die Länge von M passt in `u32`, Plattform- und Architekturnamen (höchstens sieben Zeichen: `windows`) in 16 Bytes, und beim Lesen werden die aufgefüllten Nullen entfernt. Daher ist $18 + |\mathsf{M}| + 80|T| \le |x|$, und jedes Ziel der Tabelle ist gültig und gleich einem Ziel von μ.

Offsets. Die Offsets sind die Teilsummen der Längen der in der Ordnung der Ziele vorangehenden Payloads, die Summe aller Längen ist $|\mathsf{B}|$, also Offset + Länge $\le |\mathsf{B}|$ für jeden Eintrag. *Zielmengen.* Die Tabelle enthält genau $\operatorname{dom} P = T(\mu)$. *Payloads.* $\mathsf{B}[\text{Offset}, \text{Offset} + \text{Länge}) = P(t)$, und der Hash stimmt, weil er aus genau diesen Bytes berechnet wurde. □

Korollar 8.1 (Bestimmtheit und Injektivität)**.** pack ist eine Funktion, und sie ist injektiv: pack$(\mu, P) =$ pack(μ', P') impliziert $\mu = \mu'$ und $P = P'$. Insbesondere ist der Container bei gleichen Payload-Bytes reproduzierbar, das heißt byte-identisch.

Proof. pack benutzt nur sortierte Daten (`sort_keys`, `sorted(payloads)`) und deterministische Funktionen (SHA-256, JSON-Text), ist also eine Funktion. Aus Satz 8.3 folgt, dass aus dem Bild μ und P zurückgewonnen werden; eine Funktion mit Linksinverser ist injektiv. □

Satz 8.4 (Integrität der Payloads)**.** Sei $x =$ pack(μ, P) und $x' \neq x$ eine Bytefolge gleicher Länge, die sich von x nur im Bereich des Payloads für t unterscheidet. Dann gelingt unpack(x'), aber payload_for(t) schlägt auf x' mit INVALID_CONTAINER fehl, es sei denn, es wurde eine SHA-256-Kollision gefunden (Annahme H).

Proof. Kopf, Metadaten und Tabelle sind unverändert, also gelingt unpack wie auf x. payload_for(t) berechnet SHA-256 des Bereichs von x' und vergleicht mit dem Eintrag der Tabelle, der der Hash des Bereichs von x ist. Gleichheit hieße SHA256$(x'|_t) =$ SHA256$(x|_t)$ bei $x'|_t \neq x|_t$, eine Kollision. □

Satz 8.5 (Kopf und Metadaten sind nicht geschützt)**.** Es gibt $x = \mathrm{pack}(\mu, P)$ und $x' \neq x$, die sich nur im Metadatenbereich unterscheiden, sodass $\mathrm{unpack}(x')$ gelingt, die Metadaten $\mu' \neq \mu$ liefert und $\mathrm{payload_for}(t)$ für alle t gelingt.

Proof. Sei μ mit Version `1.2.3`. Ersetze im JSON-Text die Ziffer `3` der Version durch `4`. Länge, Kopf, Tabelle und Daten bleiben gleich; M' ist gültiges JSON und liefert gültige Metadaten μ' mit Version $1.2.4$. Die Hashes beziehen sich nur auf Payloads und bleiben richtig. □

Das ist keine theoretische Spielerei: Wer die Datei verändern kann, kann auf diese Weise ABI-Version, Exportliste, Abhängigkeiten und Namen von `init` und `fini` ändern, ohne dass der Loader etwas bemerkt (**B10**). Eine Prüfung gegen Angreifer braucht eine Signatur über Kopf, Metadaten und Tabelle, nicht nur Hashes der Payloads (Ausblick, Abschnitt 17.7). Die Messung bestätigt es: Von 439 Einzelbitänderungen im Bereich von Kopf, Metadaten und Tabelle eines kleinen Containers wurden 369 beim Lesen zurückgewiesen, 66 beim Prüfen des Hashes erkannt (sie lagen in der Hashtabelle) und 4 als Container mit *anderen* Metadaten akzeptiert; alle 80 Einzelbitänderungen im Datenbereich wurden erkannt (Abschnitt 13.3).

Satz 8.6 (Totalität des Lesens)**.** Für jede Bytefolge x gilt: $\mathrm{unpack}(x)$ liefert einen Container oder löst einen Fehler vom Status INVALID_CONTAINER aus, mit der einen Ausnahme, dass hinreichend tief geschachteltes JSON in den Metadaten eine `RecursionError` der Laufzeit auslöst.

Proof. Fallunterscheidung über die Operationen von `unpack`. (1) Die Länge wird vor dem Entpacken des Kopfes geprüft; `struct.unpack_from` ist dann definiert. (2) Magic und Version werden verglichen. (3) $\mathrm{table_end} \leq |x|$ wird geprüft, bevor Tabelleneinträge gelesen werden, also sind `unpack_from`-Aufrufe in Grenzen. (4) `json.loads` löst `ValueError` (Unterklassen `JSONDecodeError`, `UnicodeDecodeError`) aus, und beide werden zu INVALID_CONTAINER; eine `RecursionError` ist keine `ValueError`. (5) `from_dict` prüft, dass das Ergebnis eine Abbildung ist, die Schlüssel bekannt sind und die Pflichtschlüssel vorhanden; Typfehler, Schlüsselfehler und Wertfehler beim Bau der Metadaten werden zu INVALID_CONTAINER. (6) Jeder Tabelleneintrag wird auf ASCII-Dekodierbarkeit, gültiges Ziel und Grenzen geprüft. (7) Zielmengen werden verglichen. Außer in (4) kann keine Operation eine andere Ausnahme auslösen. Dass die Ausnahme in (4) auftritt, zeigt die Messung: $100\,000$ geöffnete Klammern als Metadaten erzeugen `RecursionError`; $20\,000$ zufällige Mutationen eines gültigen Containers lösten dagegen nur Fehler vom Status INVALID_CONTAINER oder gar keinen Fehler aus (Abschnitt 13.3). □

Satz 8.7 (Das Lesen ist nicht kanonisch)**.** Es gibt $x \neq x'$ mit $\mathrm{unpack}(x) = \mathrm{unpack}(x')$ und gültigen Container: Angehängte Bytes ändern das Ergebnis nicht. Ferner akzeptiert unpack überlappende Payload-Bereiche und mehrfache Einträge für dasselbe Ziel.

Proof. B ist als Rest ab `table_end` definiert, ohne dass sein Ende geprüft wird; Offsets und Längen werden nur gegen $|\mathsf{B}|$ geprüft. Hängt man `JUNK` an einen

gültigen Container, vergrößert sich $|\mathsf{B}|$, alle bisherigen Einträge bleiben gültig und liefern dieselben Bytes (geprüft: `payload_for` liefert auf dem erweiterten Container dieselben Bytes). Überlappungen: Die Prüfung ist nur Offset + Länge $\leq |\mathsf{B}|$. Mehrfache Einträge: Die Einträge werden in einer Abbildung nach Ziel gesammelt, ein späterer überschreibt den früheren; die Zielmengenprüfung vergleicht Mengen, nicht Listen. □

Satz 8.7 widerspricht nicht der Rundreise (sie gilt für das Schreiben), aber der Forderung "fail closed" des Proposals bei Mehrdeutigkeit (Abschnitt 10 des Proposals): Ein Container, der Bytes hinter den Payloads hat, wird nicht als verdächtig behandelt. Als Folge ist der Hash der *Datei* keine Identität des Containers.

8.4 Der Satz zu Kanten und Versionen

Satz 8.8 (Alle Kanten erfüllen ihren Bereich)**.** Gelingt open (im Sinn von Abbildung 6.1) und ist u ein in $\mathcal{L}$ geladener Knoten mit Abhängigkeit $(d, \text{spec}) \in \Delta(\mu_u)$, die zu einem Knoten k aufgelöst wurde, so gilt $v(\mu_k) \in S(\text{spec})$.

Proof. Die Abhängigkeitskante wird beim Laden von u durch $\text{OPEN}(d, \text{spec}, \cdot)$ geöffnet. Ist k dort schon geladen, ruft OPEN $\text{CHECK_RANGE}(\mu_k, \text{spec})$ auf, bevor der Zähler erhöht wird; scheitert die Prüfung, bricht es ab, und u wird nicht geladen. Ist k nicht geladen, ruft LOAD nach der Containerprüfung $\text{CHECK_RANGE}(\mu_k, \text{spec})$ auf, bevor ein Zustand verändert wird. In beiden Fällen ist $v(\mu_k) \in S(\text{spec})$ eine Vorbedingung dafür, dass u in $\mathcal{L}$ eingetragen wird, und Knoten werden nie nachträglich verändert. □

Kapitel 9

Der Loader-Kern

Dieses Kapitel enthält die Hauptergebnisse der Arbeit: Beweise für das Verhalten der Zustandsmaschine aus Abbildung 6.1. Die Reihenfolge folgt den Versprechen des Proposals: Das Laden terminiert (Abschnitt 9.1), Zyklen werden erkannt (Abschnitt 9.2), Referenzzählung und Freigabe sind konsistent (Abschnitt 9.3), ein gescheitertes Öffnen hinterlässt keinen Zustand (Abschnitt 9.4), Initialisierung und Finalisierung folgen der Abhängigkeitsordnung (Abschnitt 9.5), Fehler benennen ihre Ursache (Abschnitt 9.6), nur Exportiertes ist erreichbar (Abschnitt 9.7) und die Nebenläufigkeit (Abschnitt 9.9). Abschnitt 9.8 zeigt, wo das Versprechen "dieselbe Bibliothek ergibt dieselbe Instanz" endet. Alle Sätze gelten unter den Annahmen A1 bis A3, sofern nichts anderes dasteht.

9.1 Der aufgelöste Graph und die Terminierung

Definition 9.1 (Aufgelöster Graph)**.** Unter A1 ist für einen Knoten u (Schlüssel einer gültigen Container-Datei) und jede Abhängigkeit $d \in \operatorname{dom}\Delta(\mu_u)$ das Ergebnis $\operatorname{res}(u,d) = \mathrm{REALPATH}(\mathrm{locate}_C(d, \mathrm{dirname}(u)))$ eine feste Größe (ein Schlüssel oder ein Fehler). Der *aufgelöste Graph* $G_C = (V, E)$ hat als Knoten die Schlüssel gültiger Dateien und als Kanten $(u, \operatorname{res}(u,d))$ für jedes d, dessen Auflösung gelingt. Die Auflösung hängt von u nur über $\mathrm{dirname}(u)$ ab, die Kanten sind also eine Funktion des Dateisystems.

Für $r = \mathrm{REALPATH}(\mathrm{locate}_C(n, \rho))$ sei $V_r = \operatorname{reach}_{G - \operatorname{dom}\mathcal{L}}(r)$ die über nicht geladene Knoten erreichbare Menge und E_r die Menge der Kanten von Knoten aus V_r.

Satz 9.1 (Terminierung und Aufwand)**.** Jeder Aufruf $\mathrm{open}(n)$ terminiert. Er ruft LOAD höchstens $|V_r|$-mal und OPEN höchstens $(|E_r|+1)$-mal auf, und die Rekursionstiefe überschreitet $|V_r|+1$ nicht. Wurde r schon geladen, besteht der Aufruf aus einer Zählererhöhung.

Proof. OPEN verhält sich auf dem Graphen wie die Tiefensuche von Definition 2.3 mit $D_0 = \operatorname{dom}\mathcal{L}$: Ein Schlüssel auf S bricht mit CYCLE ab, ein Schlüssel in $\operatorname{dom}\mathcal{L}$ kehrt nach einer Zählererhöhung zurück, ohne seine

Abhängigkeiten zu besuchen, jeder andere wird auf S gelegt, seine Nachfolger werden der Reihe nach geöffnet, danach wird er von S genommen und in $\mathcal{L}$ eingetragen (Bemerkung 6.1(b): Eintrag in $\mathcal{L}$ entspricht dem Eintrag in D). Fehler beenden die Aufrufe nur früher. Lemma 2.3 liefert Terminierung, höchstens $|V_r|$ Stapelvorgänge (jeder Stapelvorgang ist ein LOAD) und höchstens $|E_r| + 1$ Aufrufe, und die Tiefe ist durch die Stapelhöhe $\leq |V_r|$ beschränkt. Alle weiteren Operationen von OPEN und LOAD (Lesen, Entpacken, Adapter) terminieren nach A2 und Satz 8.6. □

Der Satz bestätigt, dass die Kosten des Auflösens linear in der Größe des erreichbaren Abhängigkeitsgraphen sind. Das Proposal macht dazu keine Aussage; die Bewertung der Zeit je Bibliothek steht in Abschnitt 13.3.

9.2 Zykluserkennung

Definition 9.2 (Defektfrei)**.** Eine Anfrage open(n) heißt *defektfrei bis auf Zyklen*, wenn für jeden Knoten aus V_r: die Datei lesbar und ein gültiger Container ist, die ABI im Bereich $[\ell, u]$ liegt, ein Payload für (π, α) vorhanden und der Hash richtig ist, alle Bereichsprüfungen der Kanten bestanden sind, jede Abhängigkeit aufgelöst wird, das Abbilden gelingt und `init` den Wert 0 liefert.

Satz 9.2 (Zykluserkennung ist korrekt und vollständig)**.** Sei open(n) defektfrei bis auf Zyklen. Dann endet open(n) genau dann mit dem Fehler CYCLE, wenn der Graph $G_C - \operatorname{dom}\mathcal{L}$ in V_r einen Zyklus enthält; sonst gelingt es. Insbesondere meldet eine Bibliothek, die von sich selbst abhängt, CYCLE.

Proof. Lemma 2.3 mit $D_0 = \operatorname{dom}\mathcal{L}$ und der Zuordnung aus dem Beweis von Satz 9.1. Ohne Defekte ist der einzige mögliche Abbruch die Meldung. Sie erfolgt genau dann, wenn $G_C - \operatorname{dom}\mathcal{L}$ in reach(r) einen Zyklus hat. Ohne Meldung gelingt jeder Aufruf. Eine Schleife (u, u) ist ein Zyklus der Länge 1: res$(u, d) = u$ trifft auf $u \in S$. □

Der Satz setzt *keine* Gleichheit der Namen voraus: Zyklen werden zwischen *Dateien* (Schlüsseln) erkannt, auch wenn sie über verschiedene Namen laufen, die dieselbe Datei meinen (etwa über einen symbolischen Verweis). Das ist stärker als nötig und deshalb vorteilhaft. Ein Zyklus, der nur über Dateien läuft, die nicht gleichzeitig Kanten desselben Graphen sind, bleibt dagegen unerkannt, wenn die Auflösung vom Aufrufer abhängt; Abschnitt 9.8 beschreibt, wie das aussehen kann.

9.3 Die Invariante der Referenzzählung

Dieser Abschnitt enthält den umfangreichsten Beweis der Arbeit. Er zeigt, dass die Zähler stets genau die Zahl der Verwender abbilden und dass dadurch nie eine noch benutzte Bibliothek entladen wird.

Definition 9.3 (Vielfachheiten und Buchführung)**.** Für $u \in \operatorname{dom}\mathcal{L}$ und einen Schlüssel k sei $m(u,k)$ die Anzahl der Vorkommen von k in der Liste $d(u)$. Eine *Buchführung* ist eine Multimenge F von Schlüsseln (*unterwegs befindliche Verweise*): Verweise, die OPEN zurückgegeben hat und die noch weder einem Client noch der Abhängigkeitsliste eines Knotens zugeordnet wurden. Der *erweiterte Zustand* ist (Γ, H, F).

Definition 9.4 (Invariante)**.** $\mathrm{Inv}(\Gamma, H, F)$ bedeute:

(I1) Für alle $k \in \operatorname{dom}\mathcal{L}$: $\mathrm{rc}(k) = \mathrm{ext}_H(k) + F(k) + \sum_{u \in \operatorname{dom}\mathcal{L}} m(u,k)$.

(I2) Für alle $u \in \operatorname{dom}\mathcal{L}$: $d(u) \subseteq \operatorname{dom}\mathcal{L}$.

(I3) Für alle $k \in \operatorname{dom}\mathcal{L}$: $\mathrm{rc}(k) \geq 1$, und für alle $u \in \operatorname{dom}\mathcal{L}$: $u \notin d(u)$.

(I4) Alle Schlüssel in H und in F liegen in $\operatorname{dom}\mathcal{L}$.

Die Invariante sagt: Der Zähler ist die Summe aus Clients, Unterwegs-Verweisen und Verwendern in der Abhängigkeitsliste (I1); Abhängigkeiten sind geladen (I2); es gibt keine toten und keine in sich zyklischen Knoten (I3); nichts Gehaltenes ist entladen (I4). Der Anfangszustand $((\emptyset, \varepsilon), \emptyset, \emptyset)$ erfüllt sie trivial.

Lemma 9.1 (Freigabe)**.** Erfüllt (Γ, H, F) die Invariante Inv und gilt $F(k) \geq 1$, so liefert RELEASE(k) einen Zustand $(\Gamma', H, F - \{k\})$, der Inv erfüllt. Es terminiert, und die Adapterereignisse sind $\mathsf{fini}(x) \cdot \mathsf{unmap}(x)$ für jeden dabei aus $\mathcal{L}$ entfernten Knoten x.

Proof. Induktion über $|\operatorname{dom}\mathcal{L}|$. Nach (I4) ist $k \in \operatorname{dom}\mathcal{L}$, nach (I3) $\mathrm{rc}(k) \geq 1$. RELEASE verringert $\mathrm{rc}(k)$ um 1; das passt zu $F - \{k\}$ in (I1) für k. *Fall* $\mathrm{rc}'(k) > 0$*:* Der Knoten bleibt, (I1) bis (I4) gelten für alle Knoten unverändert (für k siehe oben, für die übrigen hat sich nichts geändert).

Fall $\mathrm{rc}'(k) = 0$. Dann gilt in (I1) für den neuen Zustand $\mathrm{ext}_H(k) = 0$, $F'(k) = 0$ und $m(u,k) = 0$ für alle $u \in \operatorname{dom}\mathcal{L}$; mit (I3) gibt es also weder einen Client noch einen Verwender. k wird aus $\mathcal{L}$ entfernt, `fini` (falls deklariert) aufgerufen, unmap aufgerufen. Für jeden Schlüssel $x \in d(k)$ (mit Vielfachheit) gilt nach Entfernung des Knotens k aus der Summe in (I1) zunächst eine Lücke von $m(k,x)$: In (Γ'', H, F'') mit $F'' = F' + \{x \mid x \in d(k)\}$ (jeder Eintrag der Liste $d(k)$ wird zu einem Unterwegs-Verweis) gilt (I1) weiterhin, denn der Beitrag $m(k,x)$ der Summe wandert in $F''(x)$. (I2) gilt weiter, weil $\operatorname{dom}\mathcal{L}'' = \operatorname{dom}\mathcal{L} \setminus \{k\}$ und gilt: Gilt $k \in d(u)$ für ein $u \in \operatorname{dom}\mathcal{L}''$, so wäre $m(u,k) \geq 1$ im Widerspruch zu $m(u,k) = 0$; daher bleibt (I2) wahr. (I4): H enthält k nicht ($\mathrm{ext}_H(k) = 0$), F'' enthält keine Schlüssel außerhalb von $\operatorname{dom}\mathcal{L}''$, weil $x \in d(k) \subseteq \operatorname{dom}\mathcal{L}$ nach (I2) und $x \neq k$ nach (I3). Nun wird RELEASE(x) für jeden Eintrag x von $d(k)$ der Reihe nach aufgerufen, jeweils mit $F''(x) \geq 1$ und kleinerem $|\operatorname{dom}\mathcal{L}''|$; nach Induktionsannahme erhält jeder Aufruf die Invariante und verbraucht genau einen Unterwegs-Verweis. Am Ende ist F'' auf $F' = F - \{k\}$ geschrumpft und die Invariante gilt. Dass jeder Aufruf definiert ist, folgt aus (I4): Der Schlüssel liegt in $\operatorname{dom}\mathcal{L}$ (sonst gäbe es den Fehler `KeyError` beim Entfernen; (I4) schließt ihn aus). □

Lemma 9.2 (Öffnen)**.** Erfüllt (Γ, H, F) die Invariante und gilt $S \cap \operatorname{dom} \mathcal{L} = \emptyset$, so gilt für OPEN: Entweder wird ein Fehler ausgelöst, und (Γ, H, F) und S sind *unverändert*; oder OPEN liefert k, und der Folgezustand $(\Gamma', H, F + \{k\})$ erfüllt die Invariante, es ist $\operatorname{dom} \mathcal{L} \subseteq \operatorname{dom} \mathcal{L}'$, die Listen $d(u)$ und die Metadaten der alten Knoten sind unverändert, und S ist unverändert.

Proof. Induktion über $|\mathsf{Key} \setminus S|$ (endlich, weil nur Schlüssel gültiger Dateien betrachtet werden; jeder rekursive Aufruf hat einen Schlüssel mehr auf S).

Fehler vor jeder Zustandsänderung. Namensauflösung, Suche, Zyklus- und Bereichsprüfung bei geladenem Knoten, Lesen, Entpacken, ABI-, Plattform- und Bereichsprüfung bei neuem Knoten verändern weder $\mathcal{L}$ noch Zähler noch S (Abbildung 6.1: Bereichsprüfung vor dem Erhöhen).

Geladener Knoten $k \in \operatorname{dom} \mathcal{L}$, Prüfung bestanden. $\mathrm{rc}(k)$ wächst um 1, F um $\{k\}$: (I1) bleibt wahr. (I2) bis (I4) sind nicht berührt.

Neuer Knoten. k wird auf S gelegt; nach Induktionsannahme kann jeder rekursive Aufruf OPEN_DEP für eine Abhängigkeit d_i nur entweder den Zustand unverändert lassen und einen Fehler auslösen, oder k_i liefern und F um $\{k_i\}$ erweitern. Gelingen alle Abhängigkeiten, hält der Aufruf die Verweise $F + \{k_1, \ldots, k_j\}$, bildet ν mit `map`, führt `init` aus und legt den Knoten $\mathsf{nd} = (\mu, (k_1, \ldots, k_j), 1, \nu)$ an. Nach dem Eintrag $\mathcal{L}' = \mathcal{L}[k \mapsto \mathsf{nd}]$ gilt: $F' = F + \{k\}$ (die k_i sind jetzt Kanten, k ist der zurückgegebene Verweis). Prüfung der Invariante. (I1) für k: $\mathrm{rc}(k) = 1 = 0 + F'(k) + 0$, denn kein Knoten hängt von k ab (k ist neu) und $\mathrm{ext}_H(k) = 0$. (I1) für k_i: Die Summe hat einen neuen Summanden $m(k, k_i)$, dafür fehlt in F' genau ein Verweis je Eintrag k_i; die rechte Seite ist unverändert. (I2): $d(k) = (k_1, \ldots, k_j) \subseteq \operatorname{dom} \mathcal{L}'$ nach der Induktionsannahme ($k_i \in \operatorname{dom} \mathcal{L}$ nach (I4) im erweiterten Zustand). (I3): $k \notin d(k)$, weil ein Aufruf mit $k \in d(k)$ wegen $k \in S$ mit CYCLE geendet hätte; $\mathrm{rc}(k) = 1 \geq 1$. (I4): F' enthält $k \in \operatorname{dom} \mathcal{L}'$.

Fehler nach Zustandsänderungen. Ein Fehler bei der Abhängigkeit d_i erzeugt keinen Zustand nach Induktionsannahme, die bereits gewonnenen Verweise $k_1, \ldots, k_{i-1}$ werden mit RELEASE freigegeben; ebenso bei Fehlern von `map` (dann zusätzlich nichts abzubilden) und `init` (dann zusätzlich $\mathrm{unmap}(\nu)$). Dass (Γ, H, F) danach wiederhergestellt ist, beweist Lemma 9.3. □

Der Beweis von Lemma 9.2 benutzt Lemma 9.3 (“Rückgängigmachen”), das seinerseits Lemma 9.2 für kleinere $|\mathsf{Key} \setminus S|$ braucht; die Induktion ist daher gemeinsam über beide zu führen, was die Kopplung der beiden zu einem Induktionsbeweis über $|\mathsf{Key} \setminus S|$ macht.

Lemma 9.3 (Rückgängigmachen)**.** Sei (Γ_0, H, F_0) ein Zustand mit Inv. Erfolgen nacheinander erfolgreiche Aufrufe OPEN, die $k_1, \ldots, k_j$ liefern (jeder im Folgezustand des vorigen), und werden danach $\mathrm{RELEASE}(k_1), \ldots, \mathrm{RELEASE}(k_j)$ in *beliebiger* Reihenfolge aufgerufen, so ist der Endzustand (Γ_0, H, F_0): $\mathcal{L}$ und alle Zähler sind wie am Anfang.

Proof. Nach Lemma 9.2 und Lemma 9.1 gilt Inv im Endzustand mit $F = F_0$ und H unverändert. Sei $N = \operatorname{dom} \mathcal{L}_{\text{end}} \setminus \operatorname{dom} \mathcal{L}_0$ die Menge der Knoten, die neu geladen und nicht entfernt wurden, und angenommen $N \neq \emptyset$. Neue Knoten haben $\mathrm{ext}_H = 0$ und $F_0(x) = 0$ (nach (I4) für Γ_0 liegt F_0 in $\operatorname{dom} \mathcal{L}_0$). Die

Kanten der alten Knoten haben sich nicht verändert und enden in $\operatorname{dom}\mathcal{L}_0$ (I2), also kommt kein Verwender von $x \in N$ unter den alten Knoten vor. Eine Abhängigkeit $u \to x$ mit neuem u entsteht nur, wenn x vor dem Abschluss von u geladen wurde; ordnet man die neuen Knoten nach dem Zeitpunkt ihres Eintrags in $\mathcal{L}$, so gehen alle Kanten von später eingetragenen zu früher eingetragenen. Sei $x \in N$ der zuletzt eingetragene. Jeder Verwender von x in $\operatorname{dom}\mathcal{L}_{\text{end}}$ müsste später eingetragen sein; es gibt keinen. Nach (I1) ist $\mathrm{rc}(x) = 0 + 0 + 0 = 0$, im Widerspruch zu (I3). Also $N = \emptyset$. Für einen alten Knoten y gilt dann nach (I1) im Endzustand $\mathrm{rc}(y) = \mathrm{ext}_H(y) + F_0(y) + \sum_{u \in \operatorname{dom}\mathcal{L}_0} m(u, y)$, das ist die rechte Seite von (I1) im Anfangszustand, also $\mathrm{rc}(y)$ wie am Anfang. □

Satz 9.3 (Invariante der Referenzzählung)**.** Sei H die Multimenge der von Clients gehaltenen Schlüssel, und jeder Client rufe $\mathrm{close}(k)$ höchstens so oft auf, wie er k erfolgreich geöffnet hat (Vorbedingung $\mathrm{ext}_H(k) \geq 1$). Dann erfüllt der Zustand nach jeder Operation $\mathrm{Inv}(\Gamma, H, \emptyset)$ mit $S = \varepsilon$. Insbesondere gilt: (a) $\mathrm{unmap}(\nu_k)$ wird nur aufgerufen, wenn $\mathrm{ext}(k) = 0$ und kein geladener Knoten von k abhängt; (b) jeder offene Handle verweist auf einen geladenen Knoten; (c) $\mathrm{rc}(k) \geq 1$ für jeden geladenen Knoten.

Proof. Induktion über die Operationen. Der Anfangszustand erfüllt die Invariante. *open:* Lemma 9.2 mit $F = \emptyset$: Bei Fehler ist der Zustand unverändert; bei Erfolg gilt $\mathrm{Inv}(\Gamma', H, \{k\})$, und das Hinzufügen von k zu H (der Client hält den Verweis) bei gleichzeitigem Wegnehmen aus F erhält (I1), (I4). S ist nach der Operation leer (Bemerkung 6.1(a)). *close(k):* Wegen $\mathrm{ext}_H(k) \geq 1$ und (I1), (I3) ist $k \in \operatorname{dom}\mathcal{L}$; der Handle gilt als offen. Man verschiebe den Verweis aus H nach F und wende Lemma 9.1 an. *symbol, info:* verändern (Γ, H, F) nicht. (a) Nach dem Beweis von Lemma 9.1 wird nur im Fall $\mathrm{rc}'(k) = 0$ abgebildet, und dort ist $\mathrm{ext}(k) = 0$ und $m(u, k) = 0$. (b) ist (I4). (c) ist (I3). □

Bemerkung 9.1 (Die Vorbedingung ist nötig)**.** Der Loader prüft die Vorbedingung nicht: `close` verlangt nur, dass der Handle nicht schon als geschlossen markiert ist (der Handle ist erst geschlossen, wenn der Zähler 0 erreicht). Ist $\mathrm{ext}(k) = 0$, aber $\mathrm{rc}(k) \geq 1$ wegen eines Verwenders, so wird ein weiteres `close` angenommen. Es entzieht dem Verwender seinen Verweis, bis zum Entladen. Beispiel: $\mathrm{open}(\texttt{base})$ liefert h_1, $\mathrm{open}(\texttt{top})$ lädt `top` mit der Abhängigkeit `base` ($\mathrm{rc}(\texttt{base}) = 2$), $\mathrm{close}(h_1)$ senkt auf 1, ein zweites $\mathrm{close}(h_1)$ wird akzeptiert, entlädt `base` und lässt `top` ohne seine Abhängigkeit zurück. Geprüft: $\mathrm{unmap}(\texttt{base})$ erfolgt, `top` bleibt geladen (Abschnitt 13.3). Das ist Befund **B8**.

9.4 Atomarität: gescheitertes Öffnen hinterlässt nichts

Satz 9.4 (Rollback)**.** Löst $\mathrm{open}(n)$ einen Fehler aus, so sind $\mathcal{L}$, alle Zähler und S nach dem Aufruf wie vor dem Aufruf. Die Adapterereignisse des Aufrufs sind *ausgeglichen*: Jedem $\mathsf{map}(x)$ folgt ein $\mathsf{unmap}(x)$ und jedem

erfolgreichen $\mathsf{init}(x)$ ein $\mathsf{fini}(x)$. Das *fehlgeschlagene* `init` der betroffenen Bibliothek hat kein `fini`.

Proof. Der erste Teil ist die Fehleraussage von Lemma 9.2 für $F = \emptyset$ (mit Lemma 9.3 für den Fall, dass vor dem Fehler Abhängigkeiten geladen wurden). Für die Ereignisse: Jeder Knoten, der im Verlauf neu in $\mathcal{L}$ eingetragen wurde, wurde nach Lemma 9.3 wieder entfernt, und beim Entfernen gelten nach Lemma 9.1 die Ereignisse $\mathsf{fini} \cdot \mathsf{unmap}$ (falls `fini` deklariert ist). Ein Knoten, dessen `map` gelungen ist, dessen `init` aber scheitert, wird im Fehlerzweig von LOAD mit `unmap` freigegeben, ohne `fini`. Ein Knoten ohne `init` oder `fini` erzeugt die entsprechenden Ereignisse nicht, die Aussage gilt dann leer. □

Das ist die Eigenschaft, auf der die Fehlerbehandlung in den Anwendungen aufbaut: Ein Aufrufer kann nach einem Fehler dieselbe Anfrage wiederholen, ohne dass ein halb geladener Zustand zurückbleibt. Sie ist die formale Fassung der Zeile “Dependencies are resolved *before* the library itself is mapped” des Proposals (Schritt 4): Ein Fehler in einer Abhängigkeit kann kein Abbild der anfragenden Bibliothek hinterlassen, weil abgebildet erst nach den Abhängigkeiten wird.

9.5 Lebenszyklus: Initialisierung und Finalisierung

Definition 9.5 (Lebensdauer)**.** Eine *Lebensdauer* des Schlüssels k ist ein maximales Teilstück einer Spur zwischen einem $\mathsf{map}(k)$ und dem zugehörigen $\mathsf{unmap}(k)$ (oder dem Ende der Spur).

Satz 9.5 (Lebenszyklus)**.** Seien die Spuren aller Operationen unter A1 bis A3 gegeben. (a) Die Ereignisse eines Schlüssels k mit deklariertem `init` und `fini` bilden Wörter aus $(\mathsf{map} \cdot \mathsf{init} \cdot \mathsf{fini} \cdot \mathsf{unmap} \mid \mathsf{map} \cdot \mathsf{init}_{\text{fehl}} \cdot \mathsf{unmap})^*$ (ein vollständiges init je Lebensdauer; Präfix am Spurende). (b) Für jede Kante $u \to v$ von G_C und jede Lebensdauer von u gibt es eine Lebensdauer von v, die sie umfasst; innerhalb davon gilt $\mathsf{init}(v) < \mathsf{init}(u)$ und $\mathsf{fini}(u) < \mathsf{fini}(v)$ (mit $<$ der Reihenfolge in der Spur), sofern beide deklariert sind. (c) Die Reihenfolge der Finalisierung ist im Allgemeinen *nicht* die Umkehrung der Initialisierung.

Proof. (a) Ein Knoten wird nur in OPEN, Fall “neu”, abgebildet (genau ein map je Eintritt in $\mathcal{L}$ und im Fehlerzweig); danach folgt in LOAD `init` (genau einmal); bei Erfolg wird der Knoten eingetragen und bleibt, bis der Zähler 0 wird; dann folgen in RELEASE fini und unmap (genau einmal je Entfernung). Bei fehlgeschlagenem `init` folgt ein unmap ohne fini. Mehrfaches Öffnen eines geladenen Knotens erzeugt keine Ereignisse (nur Zähleränderung), daher bleibt es bei einem init je Lebensdauer.

(b) Nach (I2) liegt v in $\operatorname{dom}\mathcal{L}$, solange u in $\operatorname{dom}\mathcal{L}$ liegt, also umfasst die Lebensdauer von v die von u. Bei der Beschaffung von u in LOAD werden zuerst alle Abhängigkeiten geöffnet (einschließlich map und init von

v, falls v neu ist, oder früherer Beschaffung, falls v schon lag), dann folgt erst $\mathsf{map}(u)$ und $\mathsf{init}(u)$; also $\mathsf{init}(v) < \mathsf{init}(u)$. Bei der Freigabe von u in RELEASE folgen $\mathsf{fini}(u)$ und $\mathsf{unmap}(u)$, und *danach* die Verringerung der Zähler der Abhängigkeiten. Ein $\mathsf{fini}(v)$ setzt $\mathrm{rc}(v) = 0$ voraus, also nach (I1), dass $m(u, v) = 0$ für alle geladenen u; die Verringerung wegen u findet erst nach $\mathsf{fini}(u)$ statt, so dass $\mathsf{fini}(u) < \mathsf{fini}(v)$.

(c) Gegenbeispiel: r hat die voneinander unabhängigen Abhängigkeiten B und C (keine weiteren Kanten), alle drei haben `init` und `fini`. Nach Lemma 8.1 werden B und C in dieser Reihenfolge geöffnet: $\mathsf{init}(B)\,\mathsf{init}(C)\,\mathsf{init}(r)$. Bei close$(r)$ folgt $\mathsf{fini}(r)$, dann RELEASE der Liste $d(r) = (B, C)$ in dieser Ordnung: $\mathsf{fini}(B)\,\mathsf{fini}(C)$. Die Umkehrung der Initialisierung wäre $\mathsf{fini}(r)\,\mathsf{fini}(C)\,\mathsf{fini}(B)$. □

Satz 9.5(b) erfüllt die Forderung des Proposals, dass Abhängigkeiten zuerst initialisiert und zuletzt finalisiert werden (Abschnitt 5.3, Schritte 6 und 9: “in reverse dependency order”), in der Lesart “Abhängige vor Abhängigkeiten” (umgekehrte topologische Ordnung). In der Lesart “Umkehrung der Initialisierungsreihenfolge” (LIFO) ist es nicht erfüllt (c). Das Proposal sagt nicht, welche Lesart gemeint ist; eine Präzisierung ist Teil des Ausblicks (Abschnitt 17.4). Die Modelläufe bestätigen (b): in $3\,000$ Läufen mit $12\,181$ erfolgreichen und $4\,621$ gescheiterten Öffnungen und $5\,757$ Schließungen wurde die Reihenfolge der Ereignisse in jedem Lauf geprüft, ohne Verletzung (Abschnitt 13.3).

9.6 Fehler: Priorität und Ursache

Satz 9.6 (Fehlerpriorität)**.** Hat eine Anfrage mehrere Defekte an *demselben* Knoten, so wird der Fehler der in Tabelle 9.1 zuerst genannten Prüfung gemeldet. Die Reihenfolge ist fest und von FS unabhängig.

Proof. Die Prüfungen stehen im Quelltext von `_open` und `_load` in der Reihenfolge von Tabelle 9.1; jeder Fehler beendet den Aufruf durch eine Ausnahme, spätere Prüfungen werden nicht ausgeführt. □

Satz 9.7 (Ursache eines Fehlers)**.** Scheitert open(r) in einer Abhängigkeit, weil für deren Namen d vom Knoten u aus keine Datei gefunden wird (Status NOT_FOUND an dieser Stelle), so ist der gemeldete Fehler DEPENDENCY_MISSING mit `failed_name`$= d$ und `searched` gleich der Kandidatenliste von d vom Verzeichnis von u aus. Das gilt unabhängig von der Tiefe von u im Abhängigkeitsbaum: Fehler anderer Art oder bereits umbenannte Fehler werden von den umgebenden Aufrufen unverändert weitergereicht.

Proof. OPEN_DEP fängt genau Fehler vom Status NOT_FOUND aus dem Öffnen der Abhängigkeit und ersetzt sie durch DEPENDENCY_MISSING mit dem Namen d der Abhängigkeit und den durchsuchten Pfaden des ursprünglichen Fehlers; alle anderen Status werden unverändert weitergereicht (Zeilen 146–152). DEPENDENCY_MISSING ist nicht NOT_FOUND und

Nr.	Prüfung	Fehlerstatus	Quelltext
1	Name gültig	NOT_FOUND	`search.py` 74–76
2	Datei gefunden, im gehärteten Modus sicher	NOT_FOUND, ACCESS_DENIED	`search.py` 130–145
3	Schlüssel nicht auf dem Stapel	CYCLE	`loader.py` 88–90
4	bei geladenem Knoten: Versionsbereich	DEPENDENCY_MISSING	`loader.py` 92–93
5	Datei lesbar	ACCESS_DENIED, INTERNAL	`loader.py` 104–112
6	Container gültig	INVALID_CONTAINER	`loader.py` 115
7	ABI im Bereich	ABI_MISMATCH	`loader.py` 117–121
8	Payload für Plattform und Architektur, Hash	ARCH_MISMATCH, INVALID_CONTAINER	`loader.py` 122
9	Versionsbereich bei neuem Knoten	DEPENDENCY_MISSING	`loader.py` 123
10	Abhängigkeiten (rekursiv)	DEPENDENCY_MISSING und alle Fehler der Abhängigkeit	`loader.py` 126–127
11	Abbilden	INTERNAL	`loader.py` 128–132
12	`init`	INIT_FAILED	`loader.py` 134–135

Tabelle 9.1: Fehlerpriorität in der Reihenfolge der Prüfungen.

wird daher von OPEN_DEP des umgebenden Aufrufs unverändert weitergereicht. Ein NOT_FOUND des Wurzelaufrufs selbst bleibt NOT_FOUND, weil dort kein OPEN_DEP vermittelt. □

Das erfüllt die Forderung des Proposals, `failed_name` nenne den eigentlichen Verursacher und nicht nur die angeforderte Bibliothek (Abschnitt 5.6 des Proposals), *für fehlende Abhängigkeiten*. Für andere Fehler gilt es nicht: Container- und Metadatenfehler (INVALID_CONTAINER) tragen keinen Namen, auch nicht der Hashfehler beim Prüfen eines Payloads; geprüft wurde, dass `failed_name=''` für eine fehlerhafte Datei einer Abhängigkeit und für einen Hashfehler (Abschnitt 13.3). Ein Aufrufer sieht dann nur die Meldung, nicht die schuldige Datei (Befund **B10**).

9.7 Export

Satz 9.8 (Exportdurchsetzung)**.** symbol(k, s) gelingt nur, wenn $s \in X(\mu_k)$ und lookup(ν_k, s) gelingt. Für $s \notin X(\mu_k)$ ist das Ergebnis SYMBOL_NOT_FOUND, unabhängig davon, ob das native Binärformat s exportiert. Die Namen von

`init` und `fini` sind für Clients nur erreichbar, wenn sie in X stehen, obwohl der Loader sie selbst über den Adapter aufruft.

Proof. SYMBOL prüft $s \in X(\mu_k)$ vor jedem Zugriff auf den Adapter und löst sonst den Fehler aus (Zeilen 50–56). RUN_INIT und FINI rufen lookup unmittelbar auf und umgehen die Prüfung; Clients haben darauf keinen Zugriff. □

Bemerkung 9.2 (Reichweite)**.** Die Durchsetzung betrifft nur die API von `sl`. Sie ist *keine* Sicherheitsgrenze: Die native Referenz ν_k ist ein Attribut des Handles, und nativer Code, der in denselben Prozess geladen ist, kann Symbole über andere Wege (`dlsym` auf einem eigenen Handle, Adressen) auflösen. Das Proposal behauptet an dieser Stelle, die erreichbare Oberfläche der Bibliothek werde auf die deklarierte Schnittstelle begrenzt (Abschnitt 10, “Export enforcement”); das gilt für Aufrufer, die über `sl` gehen. Außerdem prüft der Loader beim Laden nicht, dass die Exporte tatsächlich im Payload vorhanden sind: ein exportiertes, aber fehlendes Symbol ergibt erst beim Nachschlagen SYMBOL_NOT_FOUND (Befund **B12**).

9.8 Identität: wann ist dieselbe Bibliothek dieselbe Instanz?

Das Proposal verspricht: “Handles are reference-counted; opening the same library twice returns the same underlying instance” (Abschnitt 5.3, Schritt 8). Der Quelltext schlüsselt Instanzen nach dem *aufgelösten Dateipfad*, nicht nach dem logischen Namen.

Satz 9.9 (Instanzidentität)**.** (a) Zwei Aufrufe open, die zum selben Schlüssel k führen, liefern dasselbe Objekt, und der Zähler wächst um 1. (b) Zwei Aufrufe mit demselben logischen Namen, aber verschiedenen Aufruferverzeichnissen $\rho \neq \rho'$ können zu *verschiedenen* Schlüsseln führen, also zu zwei Instanzen mit je einem `init`. (c) Ist $h = 0$ und liegt im Verzeichnis eines Eintrags von `SL_PATH` eine Datei `<name>.sl`, so führen alle Aufrufe mit diesem Namen zum selben Schlüssel, unabhängig von ρ.

Proof. (a) Zweiter Fall von OPEN. (c) Die Kandidatenliste beginnt mit K_2 und diese hängt von ρ nicht ab (Definition 7.4); der erste Treffer liegt also in K_2 und ist für alle ρ derselbe (Satz 7.3(b)). (b) Gegenbeispiel: SL_PATH enthält weder X noch Y und keine Datei `d.sl`. Die Bibliothek b liegt in X, die Bibliothek c in Y, beide haben die Abhängigkeit `d`; `d.sl` liegt nur in $X/$`sl` und in $Y/$`sl` (als verschiedene Dateien mit gleichem Inhalt). Beim Laden von b wird `d` von X aus aufgelöst: Treffer $X/$`sl/d.sl`; beim Laden von c von Y aus: $Y/$`sl/d.sl`. Das sind zwei Schlüssel. Geprüft: `_loaded` enthielt beide Dateien, und `init` lief zweimal (Abschnitt 13.3). □

Das Ergebnis ist ein Befund (**B8**): Die Aussage des Proposals gilt für Dateien, nicht für logische Namen. Für Bibliotheken mit Zustand, etwa ein `static`-Zähler in `d`, bedeutet es zwei getrennte Zustände in einem Prozess.

Teil (c) zeigt eine Gegenmaßnahme: Wer die Bibliothek über `SL_PATH` auffindbar macht, erhält eindeutige Auflösung. Der Preis ist der Verlust der Lokalität (Abschnitt 5.4 des Proposals nennt das Aufruferverzeichnis *nach* `SL_PATH`).

9.9 Nebenläufigkeit

Das Proposal verlangt (Abschnitt 5.7): `sl_open` und `sl_close` sind threadsicher; gleichzeitiges Öffnen derselben Bibliothek ergibt eine Instanz; die Initialisierung läuft höchstens einmal und *nie, während ein Lader-interner Lock gehalten wird, den Benutzercode erneut betreten könnte*; Initialisierung darf `sl_open` nicht auf eine Bibliothek aufrufen, die gerade initialisiert wird (CYCLE). Die Referenzimplementierung hält ein `threading.RLock`; das README gibt an, der Lader halte es während des Initialisierens (“Status und Grenzen”).

Satz 9.10 (Serialisierung von open und close)**.** (a) open und close eines Loaders schließen einander aus (gegenseitiger Ausschluss über denselben `RLock`); jede nebenläufige Ausführung von Öffnungen und Schließungen ist äquivalent zu einer sequentiellen in der Reihenfolge, in der die Sperre erworben wird. Die Sätze der Abschnitte 9.3 bis 9.5 gelten daher für sie. (b) Derselbe Thread kann innerhalb von `init` erneut `open` aufrufen; eine Bibliothek, die gerade geladen wird, ergibt dann CYCLE. (c) Läuft `init` unter der Sperre, kann ein anderer Thread, auf den `init` wartet, nicht `open` aufrufen: *Verklemmung*. (d) symbol und info sind nicht durch die Sperre geschützt; sie können mit close so verschränkt ablaufen, dass `lookup` auf eine entladene Bibliothek zugreift.

Proof. (a) `open` und `close` erwerben die Sperre für die Dauer des ganzen Aufrufs. Alle in Abschnitt 9.3 bis 9.5 behandelten Zustandsänderungen geschehen unter dieser Sperre, also sequentiell; die Annahme A3 gilt damit für sie. (b) `RLock` ist wiedereintrittsfähig; der innere `open` sieht S mit dem Schlüssel der ladenden Bibliothek, und Satz 9.2 gilt (die innere Anfrage ist eine Kante des Graphen). (c) Sei T_1 im `open` einer Bibliothek A mit `init`, und `init` startet einen Thread T_2, der `open`(B) am selben Loader aufruft, und wartet auf dessen Ende (`join`). T_1 hält die Sperre und wartet auf T_2; T_2 wartet auf die Sperre, die T_1 hält. Im Wartegraph ($T_1 \to T_2 \to T_1$) gibt es einen Zyklus: Keiner der beiden kommt voran. (d) `symbol` prüft zuerst, dass der Handle nicht geschlossen ist (Zeilen 71–74), ruft dann den Adapter auf. Zwischen beiden Schritten kann ein anderer Thread `close` ausführen, der den Zähler auf 0 senkt und unmap aufruft; danach ruft der erste Thread `lookup` auf der freigegebenen nativen Referenz, ein Verstoß gegen den Adaptervertrag (Definition 6.1). □

Die Punkte (c) und (d) sind die Befunde **B5** und **B6**. Punkt (c) wurde reproduziert: Ein Adapter, dessen `init` einen Thread startet, der `open` aufruft, und höchstens zwei Sekunden darauf wartet, ließ den Thread nach Ablauf der Zeit weiter blockiert (Abschnitt 13.3); mit unbegrenztem `join` wäre das

die Verklemmung. Punkt (d) wurde aus dem Quelltext gelesen und nicht dynamisch reproduziert, weil das Zeitfenster zwischen den beiden Zeilen von Hand nicht verlässlich zu treffen ist. **[Dynamische Reproduktion von (d) mit kontrollierter Verschränkung offen]**

Kapitel 10

Plattformadapter, Binding und Schnittstelle

Das Proposal behauptet: Alles Betriebssystemspezifische steckt im Plattformadapter; alles darüber ist auf jedem System identisch und kann mit *einer* Konformitätssuite getestet werden (Abschnitt 5.2 des Proposals). Dieses Kapitel prüft die Behauptung in drei Teilen. Der erste beweist, was an ihr *stimmt*: Der Kern greift auf den Adapter nur über vier Operationen zu, und zwei Adapter mit gleichem Verhalten sind austauschbar. Der zweite zeigt, was *nicht* stimmt: Der Kern selbst enthält Plattformabhängigkeit. Der dritte beweist Eigenschaften der Abbildung (Entpacken und Laden), des Bindings und der C-artigen Schnittstelle.

10.1 Die drei Adapter

Die Adapter unterscheiden sich im Quelltext nur in zwei Punkten (Tabelle 10.1); alles andere, einschließlich des Abbildens, steht in der gemeinsamen Basisklasse `PlatformAdapter`.

Adapter	Systemverzeichnisse $D_{\mathfrak{A}}(E)$	Freigabe
Linux	`/usr/local/lib/sl`, `/usr/lib/sl`	`_ctypes.dlclose`
macOS	`/usr/local/lib/sl`, `/opt/homebrew/lib/sl`	`_ctypes.dlclose`
Windows	`%ProgramData%\sl`, `%ProgramFiles%\sl` (nur gesetzte Variablen)	`_ctypes.FreeLibrary`

Tabelle 10.1: Was die drei Adapter unterscheidet. Das Laden selbst ist in der Basisklasse: Payload in eine Datei schreiben, mit `ctypes.CDLL` laden.

Die Adapter benutzen also nicht die nativen Laderschnittstellen unmittelbar (`LoadLibraryEx`, `dlopen`), sondern `ctypes.CDLL`, das sie je Plattform aufruft.

Das ist Stufe 1 des Proposals (“internally an ordinary DLL, .so, or .dylib”, Abschnitt 9).

10.2 Der Kern ist adapterparametrisch

Satz 10.1 (Parametrizität)**.** Der Quelltext der Module `loader.py` und `search.py` greift auf den Adapter nur über die Operationen `system_dirs`, `map_payload`, `lookup` und `unmap` zu. Die Plattform- und Architekturnamen π und α sind Parameter der Konfiguration, nicht des Adapters.

Proof. Eine statische Analyse des Syntaxbaums beider Module sammelt alle Attributzugriffe der Form `adapter.x` und `self.adapter.x`. Die Menge ist $\{\texttt{lookup}, \texttt{map_payload}, \texttt{system_dirs}, \texttt{unmap}\}$ (geprüft, Abschnitt 13.3). Attribute wie `platform` und `native_suffix` des Adapters werden vom Kern nicht gelesen; π und α sind Parameter des Loaders (Definition 6.2). □

Definition 10.1 (Beobachtungsgleiche Adapter)**.** Zwei Adapter $\mathfrak{A}_1, \mathfrak{A}_2$ heißen *beobachtungsgleich*, wenn (i) $D_{\mathfrak{A}_1}(E) = D_{\mathfrak{A}_2}(E)$ für alle E, (ii) map auf gleichen Eingaben entweder beide den gleichen OSError oder beide eine native Referenz liefern, (iii) lookup auf einander entsprechenden Referenzen für jedes Symbol beide denselben Fehler oder beide Funktionen mit gleichem Verhalten liefern, und (iv) unmap bei beiden keinen beobachtbaren Effekt außer dem Ereignis hat.

Satz 10.2 (Austauschbarkeit)**.** Sind $\mathfrak{A}_1$ und $\mathfrak{A}_2$ beobachtungsgleich, so liefern open, close, symbol und info des Loaders mit $\mathfrak{A}_1$ und mit $\mathfrak{A}_2$ bei gleichem FS, E und gleicher Konfiguration (außer dem Adapter) *dieselben Ergebnisse, dieselben Fehlerstatus und dieselben Spuren* (bis auf die Namen der nativen Referenzen).

Proof. Nach Satz 10.1 ist jeder Schritt des Loaders eine Funktion von FS, E, der Konfiguration, dem Zustand Γ und den Ergebnissen der vier Adapteroperationen. Induktion über die Schritte der Ausführung: Nach (i) sind die Kandidatenlisten gleich, nach (ii) und (iii) sind die Ergebnisse der Adapteraufrufe gleich, nach (iv) hat unmap keinen weiteren Einfluss. Daher bleiben Zustand und Ergebnis in jedem Schritt gleich. □

Der Satz ist die Grundlage der Testmethode des Repositories: Die Tests der Loader-Logik laufen gegen einen Fake-Adapter (`FakeAdapter` in `conftest.py`), und der Satz erlaubt, ihre Ergebnisse auf den echten Adapter zu übertragen, *soweit* der Fake die Bedingungen (i) bis (iv) erfüllt. Der Fake hat $D = \emptyset$, also gilt (i) nicht für echte Adapter; die Suchordnung in den Tests von `search.py` wird deshalb zusätzlich mit echten Adaptern geprüft.

10.3 Wie weit der Kern plattformfrei ist

Der Kern ist *adapterparametrisch*, aber nicht *hostunabhängig*.

Satz 10.3 (Hostabhängigkeit des Kerns)**.** Es gibt (FS, E, C), sodass locate_C auf verschiedenen Hostsystemen verschiedene Ergebnisse liefert, obwohl alle Argumente gleich sind. Ursachen: (a) die Prüfung auf absolute Pfade benutzt `os.path.isabs` des Hosts; (b) die Verbindung von Pfaden benutzt `os.path.join` des Hosts für nicht mit `/` beginnende Verzeichnisse; (c) die Prüfung auf weltbeschreibbare Verzeichnisse enthält einen Zweig `os.name == "nt"`.

Proof. (a) Sei $E[\texttt{SL_PATH}] = \texttt{C:\lib;D:\lib}$ und $h = 0$. Auf einem Windows-Host ist `ntpath.isabs("C:\\lib")` wahr, die Einträge werden Kandidaten; auf einem POSIX-Host ist `posixpath.isabs` falsch, die Einträge werden verworfen (geprüft: `ntpath.isabs` liefert `True`, `posixpath.isabs` `False`; die Kandidatenliste unter Linux enthält nur die Systemverzeichnisse). Das ist der Fall der Konfiguration $\pi =$ Windows auf Linux-Host. (b) und (c) sind in den Zeilen 87–94 und 113–127 gelesen. (c) Im Zweig für `os.name == "nt"` wird ein Verzeichnis genau dann für weltbeschreibbar erklärt, wenn sein Name `open` ist, und sonst nie; auf POSIX-Hosts wird das Bit `S_IWOTH` geprüft. □

Der Satz widerlegt die Lesart, der Kern sei hostunabhängig; er ist nur adapterunabhängig im Sinn von Satz 10.2. Die Folgen: Die Aussagen des Proposals zur Reproduzierbarkeit (“one behavior everywhere”) gelten nur, wenn auch `search.py` hostunabhängig gemacht wird (Verbesserungsvorschlag in Abschnitt 14.1). Am schwersten wiegt (c): Der Zweig ist erkennbar ein Überbleibsel aus einem Test (der Kommentar nennt ihn), und im gehärteten Modus prüft die Referenzimplementierung unter Windows damit *nichts*.

10.4 Das Abbilden: Entpacken und Laden

Definition 10.2 (Abbilden durch Entpacken)**.** $\mathrm{map}(x, n)$ der Basisklasse erzeugt ein privates temporäres Verzeichnis (`mkdtemp`, Präfix `sl-`), schreibt x als Datei $n \cdot \mathrm{suffix}(\pi)$ hinein und lädt sie mit `ctypes.CDLL`. unmap gibt das native Handle frei und löscht das Verzeichnis.

Satz 10.4 (Aufwand des Abbildens)**.** Das Öffnen eines neuen Knotens mit Payload der Größe s benötigt Zeit $\Theta(s)$ für Lesen, Hashen und Schreiben, zuzüglich einer von s unabhängigen Konstante für Suche, Entpacken des Kopfes und `dlopen`; der Speicherbedarf beträgt mindestens $3s$ (die gelesene Datei, die Kopie des Datenbereichs in `unpack`, die Kopie des Payloads in `payload_for`). Der Aufruf einer gebundenen Funktion hat keinen Zusatzaufwand, weil `lookup` die Funktion des Adapters liefert.

Proof. Aus dem Quelltext: `Path.read_bytes()` liest die ganze Datei ($\geq s$ Bytes); `unpack` bildet `blob = data[table_end:]` (Kopie, $\geq s$); `payload_for` bildet `data = self._blob[a:b]` (Kopie, s) und hasht sie (zeitlich $\Theta(s)$); `map_payload` schreibt s Bytes. Alle Kopien sind gleichzeitig lebendig, wenn `payload_for` zurückkehrt (die erste hält `_read`, die zweite der `Container`, die dritte die Variable `payload` in `_load`). Der Funktionszeiger aus `getattr(cdll, symbol)`

ist eine `ctypes`-Funktion; der Aufruf geht ohne Zwischenschicht an die native Funktion. Die Messungen (Abschnitt 13.3) bestätigen den Anstieg: in drei Läufen $1{,}0$ bis $3{,}4$ μs je KiB bei Achsenabschnitten von $0{,}16$ bis $0{,}39$ ms (Abschnitt 13.5). □

Satz 10.5 (Native Abhängigkeiten werden nicht aufgelöst)**.** Sei u eine Bibliothek, die laut Metadaten von v abhängt, und der Payload von u enthalte einen undefinierten Verweis auf ein Symbol, das v definiert. Dann scheitert map(u) mit dem Fehler INTERNAL (OSError "undefined symbol"), *obwohl* v vor u geladen wurde. Die Metadatenabhängigkeit bestimmt Ladereihenfolge, Lebensdauer und Versionsprüfung, aber keine Symbolbindung.

Proof. CDLL lädt mit dem Standardmodus `RTLD_LOCAL`; nach POSIX stehen die Symbole eines so geladenen Objekts nicht zur Auflösung von Verweisen später geladener Objekte zur Verfügung [6]. `ctypes` fordert sofortige Bindung (`RTLD_NOW`), also wird der undefinierte Verweis beim Laden von u aufgelöst und schlägt fehl. Reproduktion: Zwei Bibliotheken `a` (definiert `a_func`) und `b` (ruft `a_func`) mit `dependencies={a: *}: open("b")` liefert `SL_ERR_INTERNAL: mapping 'b' failed: …/b.so: undefined symbol: a_func`, und der Zustand ist danach leer (geprüft, Abschnitt 13.3). Satz 9.4 sorgt dafür, dass `a` wieder entladen wird. □

Das ist der gewichtigste inhaltliche Befund (**B7**). Die Abhängigkeit im Metadatenkopf wirkt wie ein Versprechen (Proposal, Schritt 4: "All dependencies are resolved *before* the library itself is mapped, so that a missing dependency produces an error that names the dependency"); das Versprechen gilt, soweit es die *Auffindbarkeit* betrifft (Satz 9.7). Eine Bibliothek, die Symbole einer anderen *benutzt*, kann sie auf diesem Weg nicht binden. Die vorhandenen Tests prüfen nur, dass eine Abhängigkeit geladen wird, und rufen eine Funktion auf, die kein Symbol der Abhängigkeit benutzt. Der Ausblick nennt drei Lösungen (Abschnitt 17.5).

Satz 10.6 (Aufräumen)**.** (a) Bei jedem Rückgang des Zählers auf 0 wird das temporäre Verzeichnis gelöscht. (b) Endet der Prozess, ohne dass ein geladener Knoten freigegeben wurde, bleibt das Verzeichnis zurück. (c) Das Verzeichnis ist nur für den Eigentümer zugänglich.

Proof. (a) unmap ruft `shutil.rmtree`. (b) Es gibt keinen `atexit`-Haken und keinen Finalizer; die Reproduktion (ein Prozess öffnet eine Bibliothek und endet) hinterließ ein Verzeichnis `sl-*` (Abschnitt 13.3). (c) `tempfile.mkdtemp` erzeugt das Verzeichnis mit den Rechten `0700`. □

10.5 Die C-artige Schnittstelle und das Python-Binding

Die Schnittstelle `capi.py` bildet jede Operation auf ein Statusergebnis ab: `sl_open` $\mapsto$ (status, wert, fehlerinfo), entsprechend `sl_symbol`, `sl_info` und, ohne Wert, `sl_close`. Sie entspricht Abschnitt 5.5 des Proposals, übersetzt aber den Zeiger-Parameter `sl_error *err` in eine Ergebniskomponente.

Satz 10.7 (Totalität der Statusschnittstelle)**.** Für jede der vier Funktionen gilt: Löst die zugrunde liegende Operation einen `SLError` aus, liefert die Schnittstelle den Status dieses Fehlers und eine Fehlerinfo mit `message`, `failed_name`, `searched` (Pfade durch Zeilenumbrüche getrennt, wie in Abschnitt 5.5 des Proposals) und `native_error`; sonst liefert sie den Status 0 (`OK`) und eine leere Fehlerinfo. Andere Ausnahmen werden nicht abgefangen.

Proof. Die Funktionen sind `try`-Blöcke mit genau einem `except SLError`, der den Status und `_error(exc)` zurückgibt. □

Das letzte ist eine Einschränkung: Eine `RecursionError` aus `unpack` (Satz 8.6) erreicht den Aufrufer als Ausnahme, nicht als Status.

Satz 10.8 (Das Binding teilt Funktionsobjekte)**.** (a) Zwei Aufrufe `lib.symbol(s)` für dasselbe s auf derselben Instanz liefern *dasselbe* Objekt; das Setzen von `argtypes` oder `restype` durch einen Aufruf ändert es für alle. (b) Ein exportierter Bezeichner der Form `__x__` ist nicht nachschlagbar.

Proof. `PlatformAdapter.lookup` benutzt `getattr(native.cdll, symbol)`. `ctypes.CDLL.__getattr__` legt die gefundene Funktion mit `setattr` im Objekt ab, ein zweites `getattr` findet sie dort; (b) `__getattr__` löst `AttributeError` für Namen aus, die mit zwei Unterstrichen beginnen und enden, noch bevor das Symbol gesucht wird. Reproduktion: `f1 is f2` liefert `True`; nach `f1.argtypes=[c_int]; f2.argtypes=[c_char_p]` ist `f1.argtypes` gleich `[c_char_p]`; `L.__x__` löst `AttributeError` aus, `L["__x__"]()` liefert den Wert (Abschnitt 13.3). Das Proposal erlaubt Bezeichner nach dem üblichen C-Muster, also auch `__x__`. □

Beide Punkte haben dieselbe Ursache und dieselbe Abhilfe: `native.cdll[symbol]` (Elementzugriff) erzeugt bei jedem Aufruf ein neues Funktionsobjekt und kennt keine Ausnahme für Namen mit doppelten Unterstrichen (Befund **B14**).

Kapitel 11

Codegenerierung und Build

Der Loader löst die Hälfte des Problems der Projekte: das Finden und Laden. Die andere Hälfte, das *Bauen* einer Bibliothek mit gleichem Export und das *Binden* ihrer Funktionen in Python, übernehmen die Werkzeuge `sl gen`, `sl build` und `sl pack`. Dieses Kapitel beweist die Eigenschaften, die sich beweisen lassen (Korrektheit des Signaturparsers, Typbreiten, Bestimmtheit und Zielmengen), und benennt, wo die erzeugten Dateien hinter dem Proposal zurückbleiben.

11.1 Was erzeugt wird

Aus einem Metadatensatz erzeugt `generate_all` sechs Dateien (Tabelle 11.1).

Datei	Inhalt
`<name>.h`	Include-Guard, `SL_EXPORT` (`__declspec(dllexport)` unter Windows, sonst `visibility("default")`), `extern "C"`, Konstante `<NAME>_ABI_VERSION`, Deklarationen von `<name>_abi_version`, `<name>_sl_metadata`, `init`, `fini` und aller Exporte
`<name>_sl.c`	Definitionen von `<name>_abi_version` und `<name>_sl_metadata` (das JSON der Metadaten als Zeichenkette), Exporttabelle
`<name>.cmake`	CMake-Funktion `<name>_add_sl_library`, die eine `SHARED`-Bibliothek ohne Präfix erzeugt und nach dem Bauen `sl pack` aufruft
`<name>_binding.py`	Python-Modul mit `SIGNATURES` (Argumenttypen, Rückgabetyp) und `load()`
`<name>.json`	die Metadaten
`build_matrix.json`	je Ziel Ausgabedatei, Compiler und Flags

Tabelle 11.1: Erzeugte Dateien. Das Proposal verlangt Header, Build-Werkzeug und Binding (Abschnitte 5.8 und 9).

Satz 11.1 (Bestimmtheit der Erzeugung)**.** `generate_all` ist eine Funktion der Metadaten: gleiche Metadaten ergeben byte-identische Dateien. Die Ausgabe ändert sich nicht, wenn die Reihenfolge der Schlüssel von `dependencies` oder `signatures` in der Eingabe geändert wird.

Proof. Die Texte werden aus den Metadaten durch Verkettung gebildet; das JSON entsteht mit `sort_keys=True` beziehungsweise aus `to_dict`, das Abhängigkeiten und Signaturen sortiert. Die Reihenfolge der Exporte ist Teil der Metadaten (Liste), nicht Eingabeartefakt; Zeit, Pfade oder Zufallswerte gehen nirgends ein. □

11.2 Der Signaturparser

Der Python-Code des Bindings entsteht aus den Signaturen der Metadaten (Zeichenketten in C-Syntax). Dafür gibt es `parse_signature` und `ctype_expr`. Satz 11.2 beweist ihre Korrektheit für eine genau beschriebene Sprache; die Grenzen der Sprache sind ein Befund (**B13**).

Definition 11.1 (Signatursprache)**.** Sei $\mathcal{T}$ die Menge der Typnamen aus Tabelle 11.2 (ohne Zeiger) und $\mathcal{W}$ die Menge der Wörter, die `parse_signature` als Typwörter erkennt (`void`, `char`, `short`, `int`, `long`, `float`, `double`, `unsigned`, `signed`, `size_t`, `bool`, `const` und alle einwortigen Namen aus Tabelle 11.2). Eine *Basis* ist T oder `const` T mit $T \in \mathcal{T}$. Die Signatursprache $\mathcal{S}$ besteht aus den Wörtern

$$B\ \sigma^*\ \ f\ \texttt{(}\ \textit{args}\ \texttt{)}\quad\text{mit}\quad \textit{args} \in \{\varepsilon,\ \texttt{void}\} \cup \{a_1, \ldots, a_k\},$$

wobei B eine Basis ist, σ^* eine Folge von Sternen (mit beliebigen Leerzeichen dazwischen), f ein Bezeichner und jedes a_i die Form $B_i\,\sigma_i^*\,[\,p_i\,]$ hat mit einem optionalen Parameternamen p_i, der ein Bezeichner ist und nicht in $\mathcal{W}$ liegt; ein Parameter `void` ohne Stern kommt nur als alleiniges Argument vor. Leerzeichen dürfen zwischen allen Teilen stehen, wo die Trennung der Wörter es verlangt, und am Ende darf ein Semikolon stehen.

Satz 11.2 (Korrektheit des Signaturparsers)**.** Für $s \in \mathcal{S}$ mit Funktionsname f liefert `parse_signature`(s, f) ein Paar $(\rho, (\tau_1, \ldots, \tau_k))$ mit: ρ und τ_i stimmen nach Normalisierung (Sterne durch Leerzeichen getrennt, Leerzeichen auf eines verdichtet) mit Rückgabetyp $B\sigma^*$ und Parametertypen $B_i\sigma_i^*$ überein; für $\textit{args} \in \{\varepsilon, \texttt{void}\}$ ist $k = 0$. Ist $f' \neq f$, wird ein `ValueError` ausgelöst.

Proof. Zerlegung. Das Muster ist `^(?P<ret>.*?[\s*])(?P<name>[A-Za-z_]\w*)\s*\((?P<args>.*)\)\s*;?\s*$`. Ein Wort $s \in \mathcal{S}$ enthält genau eine öffnende und genau eine schließende Klammer (die Typen und Parameter enthalten keine), die erste Klammer steht also unmittelbar hinter dem Namen f (gegebenenfalls nach Leerzeichen), und die letzte schließt die Argumentliste. Jede Übereinstimmung des Musters muss deshalb `args` gleich dem Wort zwischen den beiden Klammern wählen. Der Name wird durch $[A\text{-}Za\text{-}z_]\backslash w^*$ erkannt und muss vor `\s*\(` enden; sein Anfang liegt hinter einem Zeichen aus `[\s*]` (dem Ende von `ret`). Das Zeichen vor dem Anfang von f im Wort ist ein Leerzeichen oder ein Stern, und das Zeichen davor ist Teil von B oder σ^*; ein früherer Beginn des Namens würde ein Wort `ret` verlangen, das auf $[\backslash s*]$ endet und dessen Rest ein einziges Bezeichnerwort vor der Klammer ist; Leerzeichen sind im Namen nicht erlaubt, also beginnt der

C-Typ	`ctypes`-Typ	Bemerkung
`void` (Rückgabe)	`None`	
`char`	`c_char`	ein Byte
`short`, `int`	`c_short`, `c_int`	Breite des Hosts, in allen Datenmodellen 2 und 4 Bytes
`unsigned`, `unsigned int`	`c_uint`	
`long`, `unsigned long`	`c_long`, `c_ulong`	**Breite abhängig vom Datenmodell**
`long long`, `unsigned long long`	`c_longlong`, `c_ulonglong`	8 Bytes
`float`, `double`	`c_float`, `c_double`	
`size_t`, `bool`	`c_size_t`, `c_bool`	`size_t`: Zeigerbreite
`int8_t`…`uint64_t`	`c_int8`…`c_uint64`	feste Breite
`char *`, `const char *`	`c_char_p`	Bytes, nullterminiert
jeder andere Zeiger	`c_void_p`	
alles andere (`unsigned char`, `signed int`, `struct`…)	*Fehler* (`ValueError`)	

Tabelle 11.2: Abbildung der C-Typen auf `ctypes`-Typen in `ctype_expr`.

Name genau bei f. Damit ist `ret` $= B\sigma^*$ (bis auf angrenzende Leerzeichen, die `strip()` entfernt) und `name` $= f$.

Argumente. Für *args* $\in \{\varepsilon, \texttt{void}\}$ liefert der Code die leere Liste. Sonst wird an den Kommas getrennt; in $\mathcal{S}$ kommen Kommas nur zwischen den Parametern vor. Für einen Parameter $a = B_i\sigma_i^*[p_i]$ entstehen aus der Ersetzung von $*$ durch ` * ` und dem Zerlegen an Leerzeichen die Wörter von B_i, danach die Sterne, danach, falls vorhanden, p_i. Das letzte Wort wird entfernt, wenn es mehr als ein Wort gibt, kein Stern ist und nicht in $\mathcal{W}$ liegt. Ist p_i vorhanden, so ist es das letzte Wort, nicht Stern, nicht in $\mathcal{W}$ (Voraussetzung), und es gibt mehr als ein Wort (B_i hat mindestens eins): es wird entfernt. Fehlt p_i, ist das letzte Wort entweder ein Stern oder das letzte Wort von B_i, und das liegt in $\mathcal{W}$ (jedes Typwort in $\mathcal{T}$ ist in $\mathcal{W}$): es wird nicht entfernt. Die verbleibenden Wörter sind genau die normalisierte Form von $B_i\sigma_i^*$.

Fehler. Stimmt f' nicht mit `name` überein, wird der Fehler ausgelöst (Zeile 29). □

Zur Bestätigung wurden 100 000 Wörter von $\mathcal{S}$ zufällig erzeugt (Typen aus Tabelle 11.2 mit und ohne `const`, null bis zwei Sterne, null bis drei Parameter, Namen `x`, `count`, `p_1` oder keine). Es gab keine Abweichung zwischen `parse_signature` und der Erzeugung (Abschnitt 13.3). Der Rückgabetyp wird vom Parser nicht normalisiert (`unsigned long*`), die Argumenttypen schon; die Normalisierung geschieht in `ctype_expr`.

Bemerkung 11.1 (Die Voraussetzung über Parameternamen)**.** Ein Parameter, der *wie ein Typwort* heißt, wird für den Typ gehalten: `int count` ist in Ordnung, aber ein Parameter namens `long` wäre in C ohnehin kein Bezeichner. Bei Bezeichnern, die einem der einwortigen Namen aus $\mathcal{W}$ entsprechen

(`size_t`, `bool`), tritt der Fall in C ebenfalls nicht auf. Die Voraussetzung $p_i \notin \mathcal{W}$ schließt in der Praxis nur verbotene Namen aus.

11.3 Typbreiten im Binding

Satz 11.3 (Typabbildung)**.** (a) Für Typen mit fester Breite N (`int`N`_t`, `uint`N`_t`), für `int`, `short`, `char` und `long long` hat der gewählte `ctypes`-Typ in allen Datenmodellen von Definition 5.3 (LP64, LLP64, ILP32) dieselbe Breite wie der C-Typ. (b) Für `long`, `unsigned long` und `size_t` hat der `ctypes`-Typ die Breite des *Hosts*, nicht eine feste: In LP64 sind `long` 8 Bytes, in LLP64 4 Bytes; das Binding gibt daher auf zwei Systemen denselben Quelltext mit verschiedenem Wertebereich. (c) Jeder Nicht-Zeigertyp, der nicht in Tabelle 11.2 steht, löst einen Fehler aus.

Proof. (a) Nach Satz 5.2(b) ist die Breite der Typen fester Breite konstant; `c_int8` bis `c_uint64` haben die Breiten $1, 2, 4, 8$ in `ctypes`. Die Größen von `short`, `int` und `long long` in den drei Datenmodellen sind gleich $(2,\ 4,\ 8)$, und `c_short`, `c_int`, `c_longlong` entsprechen dem C-Typ des Hosts. (b) `c_long` entspricht dem `long` der C-Laufzeit des Hosts (Dokumentation von `ctypes`), nach Satz 5.2(a) ist das nicht plattformunabhängig. (c) `ctype_expr` löst `ValueError` für Basen außerhalb der Tabelle aus, sofern kein Stern vorkommt (geprüft: `unsigned char` und `signed int` lösen den Fehler aus; `struct s *` wird als beliebiger Zeiger zu `c_void_p`). □

Aus (b) folgt der Befund **B13**: Das Proposal verlangt feste Breiten in der exportierten Schnittstelle (Abschnitt 5.8), aber weder der Parser noch die Metadatenprüfung weisen ein `long` zurück. Außerdem fehlen `unsigned char` und `signed` gänzlich, obwohl sie in C-Schnittstellen häufig vorkommen.

11.4 Build-Matrix, Fat-Container und CMake

Satz 11.4 (Build-Matrix)**.** Für Metadaten mit $|T|$ Zielen hat `build_matrix` genau $|T|$ Einträge; der Eintrag für (π, α) hat die Flags flags(π, α) (Linux: `-shared -fPIC -fvisibility=hidden`; macOS: `-dynamiclib -fvisibility=hidden -arch <a>` mit $a \in \{\text{x86_64}, \text{arm64}\}$, für andere Architekturen `ValueError`; Windows: `-shared`) und die Ausgabedatei $n \cdot \text{suffix}(\pi)$. Die Ausgabedateien sind genau dann paarweise verschieden, wenn die Plattformen der Ziele paarweise verschieden sind.

Proof. Die Matrix ist eine Listenkomprehension über die Ziele. Der Dateiname hängt nur von n und π ab, nicht von α; zwei Ziele mit gleicher Plattform haben also gleiche Namen, zwei mit verschiedener nicht (die Endungen `.so`, `.dylib`, `.dll` sind verschieden). □

Das Beispiel $T = \{\text{linux:x86_64}, \text{linux:aarch64}\}$ ergibt zweimal `wsat.so` (geprüft); baut man beide Ziele in dasselbe Verzeichnis, überschreibt das zweite das erste. Das Proposal nennt als möglichen Namen `wsat_shim-x86_64.sl`, “only if explicitly desired for distribution” (Abschnitt 6.1); die Matrix kennt das nicht (**B13**).

Satz 11.5 (Fat-Container)**.** Für jedes durch `pack_files` oder `build_sl` erzeugte `.sl` ist die Zielmenge der Metadaten im Container gleich der Menge der tatsächlich gepackten Payloads; die im Eingabe-JSON *deklarierte* Zielmenge geht nicht ein. Bei `build_sl` gewinnt der Payload des Hosts bei Gleichheit mit einem Eintrag aus `extra_files`.

Proof. `pack_files` ruft `pack(metadata.with_targets(payloads), payloads)` auf; `with_targets` ersetzt die Ziele durch die sortierten Schlüssel von `payloads`. `build_sl` bildet `files = {**extra_files, host: lib}`, in dem der spätere Schlüssel gewinnt. Reproduktion: Deklariert sind `linux:x86_64` und `linux:aarch64`, gebaut wurde nur der Host; der Container enthält `linux:x86_64` (geprüft). □

Satz 11.5 ist beabsichtigt (das Ergebnis ist in sich konsistent: Tabelle und Metadaten stimmen überein, Satz 8.3), aber die deklarierte Zielliste im JSON ist damit keine Zusage; wer ein Ziel vergisst, bekommt keine Meldung. Eine Warnung bei Abweichung wäre billig (**B13**).

Bemerkung 11.2 (Weitere Beobachtungen aus dem Quelltext)**.** (a) Das erzeugte CMake-Modul setzt nur `C_VISIBILITY_PRESET hidden`, nicht `CXX_VISIBILITY_PRESET`; eine C++-Shim-Bibliothek wie in Gen-DB bleibt damit in ihren Symbolen sichtbar. (b) Es benutzt die Variablen `SL_TARGET_PLATFORM` und `SL_TARGET_ARCH`, die nirgends gesetzt werden (geprüft: der Text enthält kein `set(SL_TARGET_PLATFORM)`; ohne sie ist der Aufruf von `sl pack` fehlerhaft. (c) Die erzeugte C-Quelle definiert die Tabelle `<name>_sl_exports`, die nicht benutzt wird; GCC meldet mit `-Wall -Wextra -Wunused-const-variable` (geprüft). Der vorhandene Test übersetzt nur den Header streng (`-fsyntax-only`). (d) Die Definition `<NAME>_SL_BUILDING` wird gesetzt, aber vom Header nicht verwendet. (e) Für Windows gibt die Build-Matrix nur `-shared`, ohne `-static-libgcc` oder `-static-libstdc++`; das Feld `runtime` hat also auch im Build keine Wirkung (**B9**).

Teil IV

Prüfung und Anwendung

Kapitel 12

Anwendung auf Gen-DB und Subgraph-SAT-Solver

Dieses Kapitel beantwortet die Frage, was sich in den zwei Projekten ändert, wenn sie das Ladeverfahren von `sharedlib` übernehmen, und unter welchen Bedingungen das Verhalten gleich bleibt. Der Quelltext der Projekte lag nicht vor; die Angaben stützen sich auf Abschnitt 2.1 und 12 des Proposals und auf die Vorarbeit und sind so gekennzeichnet (Tabelle 3.3). Die Aussagen über `sl` selbst sind die der Kapitel 7 bis 11.

12.1 Vorher und nachher

Das Proposal nennt, was in beiden Projekten bei Stufe 1 entfallen oder einfacher würde (Abschnitt 12 des Proposals). Tabelle 12.1 gibt es mit den Sätzen dieser Arbeit wieder, die ersetzen, was bisher Quelltext war.

Nicht aus dem Quelltext der Projekte zu entnehmen ist, *wie viele Zeilen* entfallen. Das Proposal schlägt vor, genau das beim Migrieren zu messen (Abschnitt 13, Punkt 5). **[Zahl der entfallenden Plattformzeilen in beiden Projekten nicht gemessen]**

12.2 Wann bleibt das Verhalten gleich?

Eine Migration ersetzt `ctypes.CDLL`(p) für einen Pfad p durch `sl.open`(n). Die Frage ist, wann beide dasselbe tun.

Definition 12.1 (Migrationsbedingungen)**.** Die Migration einer Bibliothek b mit nativer Datei p_b zu einer Datei n`.sl` erfüllt:

M1 Der Payload für das Ziel des Hosts in n`.sl` ist byteweise gleich p_b.

M2 Der Payload hängt nicht von seinem eigenen Ablageort ab (keine Suche nach Dateien relativ zur Bibliothek, kein `$ORIGIN` im `RUNPATH`, kein `@loader_path`).

M3 Der Payload braucht von anderen Bibliotheken nur solche, die das System des Hosts findet (Standardpfade), nicht die der Metadatenabhängigkeiten (Satz 10.5).

Bisher	Mit `sl`	Was den Ersatz trägt
`library_filename()`, vier Dateinamen	`sl.open("wsat_shim")`	Satz 7.1: ein Name, eine Datei
`os.add_dll_directory` je `PATH`-Eintrag	entfällt; `SL_PATH`, Aufruferverzeichnis	Satz 7.3, Satz 7.4; aber Satz 10.5
Sonderfall MinGW in `build_shim()`	entfällt für den Namen; `sl build` bestimmt Flags	Satz 11.4; die Laufzeitfrage bleibt (**B9**)
Makro `CSUB_EXPORT`	`SL_EXPORT` im erzeugten Header	Satz 11.1; Exportdurchsetzung: Satz 9.8
statisches Linken der MinGW-Laufzeit	Feld `runtime`	*nicht belegt*: das Feld hat keine Semantik (**B9**)
README: PowerShell, vier Dateinamen	ein Absatz	Satz 7.1

Tabelle 12.1: Was ersetzt wird und was die Ersetzung trägt. Die vorletzte Zeile ist der Punkt, an dem das Proposal mehr verspricht, als die Referenzimplementierung hält.

M4 Die Suchordnung findet n.`sl` am vorgesehenen Ort (Satz 7.3).

Satz 12.1 (Verhaltensgleichheit der Migration)**.** Erfüllt die Migration M1 bis M4, so führen `CDLL`(p_b) und `sl.open`(n) denselben nativen Code aus; jeder Aufruf einer gebundenen Funktion mit denselben Argumenten liefert bis auf die Ladeadresse dasselbe Ergebnis. Der Unterschied ist die Zeit des Öffnens (Satz 10.4) und der Ablageort der Datei (ein privates temporäres Verzeichnis).

Proof. Nach M4 wird n.`sl` gefunden, nach Satz 8.3 liefert `payload_for` den Payload byteweise. Nach M1 ist er gleich p_b. Der Adapter schreibt ihn unverändert in eine Datei und lädt diese. Der Lader des Betriebssystems bildet dann dieselben Bytes ab; die Auflösung der Abhängigkeiten des Payloads richtet sich bei beiden Wegen nach dem Betriebssystem, nach M3 ist das Ergebnis dasselbe, und nach M2 hängt der Code nicht davon ab, wo seine Datei liegt. Der gebundene Funktionszeiger ist der gleiche wie bei `CDLL` (Satz 10.4). Ein deterministischer Code ohne Abhängigkeit von der Ladeadresse liefert bei gleichen Argumenten gleiche Ergebnisse. □

Die Bedingung M2 trifft auf die Wiederverwendung in Gen-DB zu, wo die C++-Laufzeit unter Windows statisch gelinkt wird, damit `libstdc++-6.dll` nicht gesucht werden muss: Dort ist M3 erfüllt. Wo eine Bibliothek ihre Laufzeit dagegen als *Nachbardatei* findet (üblich unter Windows), ist M2 verletzt, und `runtime = "external"` wirkt nicht (**B9**). **[Ob `csubgraph_shim` und `wsat_shim` M2 und M3 erfüllen, ist nicht geprüft]**

12.3 Prozess-Pool und Speicher

Gen-DB benutzt für Vergleiche einen Prozess-Pool (`compare_many`, **W12** der Vorarbeit). Jeder Prozess lädt die Bibliothek für sich. Mit `ctypes.CDLL(p)` teilen sich alle Prozesse *dieselbe Datei* und damit im Seitencache des Betriebssystems dieselben Seiten für den Code. Mit `sl` extrahiert jeder Prozess den Payload in ein *eigenes* Verzeichnis (Satz 10.6).

Satz 12.2 (Speicher im Prozess-Pool)**.** Laden N Prozesse dieselbe Bibliothek der Codegröße s über `sl`, belegen die Codeseiten im Seitencache $N \cdot s$ statt s, und es werden $N \cdot s$ Bytes auf Platte geschrieben, wenn jeder Prozess seine Datei extrahiert.

Proof. Der Seitencache ist je Datei (Inode) angelegt. N extrahierte Dateien sind N verschiedene Inodes, also N getrennte Ausschnitte im Cache, auch wenn die Inhalte gleich sind. Bei N Prozessen mit je einer eigenen Extraktion werden N Dateien der Größe s geschrieben. □

Das ist keine Aussage über die Praxis: Moderne Dateisysteme können Gleiches zusammenlegen (Deduplikation), und Verzeichnisse im Arbeitsspeicher (`tmpfs`) ersparen den Plattenzugriff. **[Messung für den Pool von Gen-DB ausstehend]** Eine Abhilfe ist, den Payload einmal in ein gemeinsames, inhaltsadressiertes Verzeichnis zu extrahieren (Ausblick, Abschnitt 17.5).

12.4 Vergleich der zwei Ladeverfahren

Die Vorarbeit vergleicht die Verfahren von Gen-DB und `subgraph-sat-solver-econ`. Beide lösen dieselbe Aufgabe mit verschiedener Technik; Tabelle 12.2 stellt sie mit dem Weg über `sl` nebeneinander. Die Einträge der ersten beiden Spalten stammen aus Abschnitt 2.1 des Proposals.

12.5 Ein Migrationsplan mit Abnahmekriterien

Aus Satz 12.1 und den Befunden folgt eine Reihenfolge, in der die Migration prüfbar bleibt.

1. **Ein Container je Shim bauen** (`sl gen`, `sl build`); Abnahme: M1, das heißt der Payload des Hosts ist byteweise die bisherige Bibliothek (Vergleich der Hashes aus `sl info` mit dem der Datei).

2. **M2 und M3 prüfen**: Abhängigkeiten des Payloads auflisten (`ldd`, `objdump -p`, Dependency Walker); Abnahme: keine Abhängigkeit von Nachbardateien.

Aspekt	Gen-DB	`subgraph-sat-solver-ecosl`	
Dateiname	Wrapper `csubgraph_shim`, Name in `csubgraph_native.py`	vier Namen in `library_filename()`	ein Name `wsat_shim` (Satz 7.1)
Abhängigkeiten	MinGW-Laufzeit statisch gelinkt	`add_dll_directory` je `PATH`-Eintrag	Metadaten; nur Ladereihenfolge (Satz 10.5)
Export	Makro `CSUB_EXPORT`	**[nicht angegeben]**	`SL_EXPORT`, Exportliste (Satz 9.8)
Toolchain	MinGW	MSVC oder MinGW (Sonderfall)	`sl build`, Build-Matrix (Satz 11.4)
Fehlerdiagnose	**[nicht angegeben]**	**[nicht angegeben]**	Statuscodes, `failed_name`, `searched` (Satz 9.7)
Aufruf	`ctypes`, C-Wrapper	`ctypes`	`ctypes` über `sl` (kein Zusatzaufwand je Aufruf, Satz 10.4)

Tabelle 12.2: Die zwei Ladeverfahren und der Weg über `sl`.

3. **Parallelbetrieb**: in einem Testlauf beide Wege (`CDLL` und `sl.open`) aufrufen und die Ergebnisse einer Menge von Eingaben vergleichen (Differenzialtest); Abnahme: gleiche Ergebnisse (Satz 12.1).
4. **Umstellung**, nach erfolgreichem Parallelbetrieb; die Plattformzeilen aus Tabelle 12.1 entfernen und zählen.
5. **Windows und macOS prüfen**: die Adapter sind dort nur über Mocks getestet (README). **[Echte Systeme nicht geprüft]**

Schritt 3 ist der Teil, der die Frage “Wer sagt, dass sich nichts geändert hat?” beantwortet; es ist die Anwendung der Wiederverwendungsformen der Vorarbeit: Wie bei einer Portierung (**W1**) wird ein Ersatz gegen das Original geprüft, nur ist der Ersatz hier die Ladeschicht und nicht der Algorithmus.

Kapitel 13

Prüfung und Messung

Dieses Kapitel berichtet, was an der Referenzimplementierung selbst geprüft wurde. Die Läufe stützen die Beweise der Teile II und III (sie belegen, dass das Modell den Quelltext trifft) und reproduzieren die Befunde des Kapitels 14. Sie ersetzen keine Beweise: Ein Zufallstest zeigt Abwesenheit von Fehlern in den gezogenen Fällen, ein Satz in allen.

13.1 Umgebung und Methode

Alle Läufe geschahen in einer Linux-Umgebung (Kernel 6,18, zwei Kerne eines Xeon mit 2,8 GHz) mit Python 3,13 und GCC 13,3. Die Prüfskripte benutzen nur die Standardbibliothek und das Paket `sl` aus dem untersuchten Archiv; sie werden mit `python -I` gestartet, damit keine Datei des Arbeitsverzeichnisses importiert wird (Anhang B). Zufallsprüfungen verwenden feste Startwerte, die Läufe sind wiederholbar.

13.2 Die Testsuite des Repositories

Das Repository enthält elf Testdateien mit 85 Testfunktionen (statisch gezählt); mit parametrisierten Fällen gibt das README 126 bestandene Tests und eine Abdeckung von 99 % an. `pytest` und `pytest-cov` ließen sich in der Arbeitsumgebung nicht installieren (der Paketindex ist für die Sitzung gesperrt; ein Umweg wurde nicht gesucht). Die Suite wurde deshalb *nicht ausgeführt*. **[Ausführung der Suite und Prüfung der 126 Tests offen]**

Ausgewertet wurde der mitgelieferte Abdeckungsbericht (`doc/coverage/status.json`, Werkzeug `coverage` 7,16,2): 780 Anweisungen, davon 4 nicht erreicht (alle in `search.py`), 200 Zweige, davon 4 nur teilweise. Daraus ergibt sich eine Anweisungsabdeckung von 99,5 % und, mit den 4 Teilzweigen, eine Zweigabdeckung der Zweige von 98 %. Das README gibt dagegen an, die Zeilenabdeckung habe 100 % betragen und die Zweigabdeckung sei nicht gemessen worden; der Bericht enthält Zweige und vier ungedeckte Anweisungen. Die Dateipfade im Bericht haben Rückstriche (`src\sl\…`): Der Lauf geschah unter Windows. Das passt zum Zweig `os.name == "nt"` in `search.py`, dessen POSIX-Teil auf dem Windows-Lauf ungedeckt

bleibt, und ist zugleich ein Hinweis darauf, dass die Suite unter Windows ausgeführt wurde, was das README nicht sagt.

13.3 Eigenschaftstests

Tabelle 13.1 fasst die Zufalls- und Modellprüfungen zusammen. Jede Zeile nennt den Satz, den sie stützt.

Zu P5: Die Modellläufe benutzen einen Fake-Adapter wie die Suite des Repositories (Abschnitt 10, Satz 10.2); sie erzeugen zufällig Abhängigkeitsgraphen mit Zyklen und defekten Dateien, öffnen und schließen zufällig und prüfen die Invariante Inv mit $F = \emptyset$ und H als Menge der gehaltenen Handles. Die Anforderung an Clients (Satz 9.3) wird eingehalten: Es wird nur geschlossen, was geöffnet wurde. Gemessen wird also, dass der Beweis des Satzes zum Quelltext passt; er beweist ihn nicht.

Zu P1: Eine Beobachtung am Rande, die die Wirkung des Laufwerkspräfix zeigt: Bei einem früheren Lauf mit $200\,000$ zufälligen Listen (mit Alphabet `a b / \ : ; .` und Leerzeichen, Listen mit `;` ausgelassen) scheiterte die Rundreise in $2\,809$ von $115\,864$ zulässigen Listen ($2{,}4\,\%$); die Zahl hängt vom Alphabet ab und ist nur eine Illustration, keine Häufigkeit in der Praxis.

13.4 Reproduktionen der Befunde

Tabelle 13.2 nennt die Läufe, die einen Befund an einem konkreten Fall zeigen. Sie sind Existenzbeweise: Sie zeigen, dass der Fehler auftritt, nicht, wie häufig er in der Praxis ist.

13.5 Kosten des Öffnens

Zur Frage, was die Ladeschicht kostet, wurde das Öffnen, Nachschlagen eines Symbols, Aufrufen und Schließen mit `sl` gegen `ctypes.CDLL`, Nachschlagen, Aufrufen und `dlclose` gemessen. Die Bibliotheken enthalten eine Funktion und ein Feld konstanter Daten von 0, 64, 512 und $4\,096$ KiB; die Dateien sind $15\,232$, $80\,768$, $539\,520$ und $4\,209\,536$ Bytes groß. Je Größe und Verfahren wurden 300 Wiederholungen gemessen; Tabelle 13.3 zeigt den Median.

Eine Ausgleichsgerade durch die vier Punkte jedes Laufs ergibt eine Zeit

$$t(s) \approx a + b \cdot s, \qquad a \in [0{,}16;\, 0{,}39]\ \text{ms}, \quad b \in [1{,}0;\, 3{,}4]\ \text{ms/MiB},$$

mit Bestimmtheitsmaß $R^2 \geq 0{,}999$ in jedem Lauf (Lauf 1: $a = 0{,}39$, $b = 0{,}99$; Lauf 2: $a = 0{,}18$, $b = 3{,}30$; Lauf 3: $a = 0{,}16$, $b = 3{,}40$). Der lineare Zusammenhang bestätigt Satz 10.4. Die Steigung schwankt zwischen den Läufen um den Faktor $3{,}4$; eine mögliche Ursache sind der Zustand des Seitencaches und des temporären Dateisystems, sie wurde nicht untersucht. Als Größenordnung gilt: *Das Öffnen kostet einige hundert Mikrosekunden plus ein bis drei Millisekunden je Mebibyte*. Für Bibliotheken, die einmal je

Nr.	Gegenstand	Prüfung und Ergebnis	Stützt
P1	`SL_PATH`-Rundreise	$300\,000$ zufällige Listen nichtleerer Einträge über dem Alphabet `a b / \ : .` und Leerzeichen: $\mathrm{Dec}(\mathrm{Enc}(L)) \neq L$ genau dann, wenn $\mathrm{Drive}(L)$; 0 Abweichungen	Satz 7.2
P2	Versionsbereiche, Konvexität	$3\,000$ zufällige Bereiche über 27 Versionen: 0 Verletzungen der Konvexität	Satz 8.1
P2b	kleinste Version	$20\,000$ zufällige Bereiche (davon $10\,732$ nichtleer): $S \neq \emptyset \iff \mathrm{lo} \in S$ und $\mathrm{lo} = \min S$, 0 Abweichungen	Satz 8.2
P3	Container-Rundreise	$2\,000$ zufällige Metadaten und Payloads: $\mathrm{unpack} \circ \mathrm{pack} = \mathrm{id}$, pack wiederholbar	Satz 8.3, Kor. 8.1
P3b	Bitänderungen	Container mit zwei Zielen, 519 Bytes: 80 Änderungen im Datenbereich, alle erkannt; 439 im Kopf, Metadaten und Tabelle: 369 beim Lesen zurückgewiesen, 66 beim Hash erkannt, 4 als andere Metadaten akzeptiert	Sätze 8.4, 8.5
P3c	Totalität	$20\,000$ zufällige Mutationen oder Kürzungen: nur `SLError` oder Erfolg; $100\,000$ öffnende Klammern als Metadaten: `RecursionError`	Satz 8.6
P3d	nichtkanonisch	angehängte Bytes ändern das Ergebnis nicht	Satz 8.7
P4	Signaturparser	$100\,000$ zufällige Signaturen der Sprache $\mathcal{S}$: 0 Abweichungen	Satz 11.2
P5	Modellläufe des Loaders	$3\,000$ Läufe mit zufälligen Abhängigkeitsgraphen (2 bis 7 Bibliotheken, $30\,\%$ mit Zyklus, $10\,\%$ der Dateien defekt): $12\,181$ erfolgreiche und $4\,621$ gescheiterte Öffnungen, $5\,757$ Schließungen; nach jedem Schritt die Invariante von Definition 9.4 geprüft ($22\,559$ Prüfungen, 0 Verletzungen); nach jeder gescheiterten Öffnung gleiche Zähler und leerer Stapel ($4\,621$ von $4\,621$); Init- und Fini-Reihenfolge je Lauf geprüft ($3\,000$ von $3\,000$); #map = #unmap und #init = #fini am Ende jedes Laufs	Sätze 9.3, 9.4, 9.5, 9.2
P6	Adapterzugriffe	statische Analyse des Syntaxbaums von `loader.py` und `search.py`: Zugriffe auf `lookup`, `map_payload`, `system_dirs`, `unmap`	Satz 10.1

Tabelle 13.1: Zufalls- und Modellprüfungen. Alle Läufe mit festem Startwert.

Prozess geöffnet und danach vielfach aufgerufen werden, ist das gering; für Dienste, die sie je Anfrage öffnen, wäre es der dominierende Posten.

Die Kosten je *Aufruf* einer gebundenen Funktion unterscheiden sich nicht: gemessen wurden $200\,000$ Aufrufe einer Funktion `int f(int)` mit `argtypes` und `restype`; die Mittelwerte lagen je Lauf und Größe zwischen 309

Nr.	Befund	Beobachtung
R1	**B1**, **B2**	`split_sl_path("a:/opt/x")` liefert `["a:/opt/x"]`; `split_sl_path("C:\x:D:\y")` liefert `["C:\x","D","\y"]`; `split_sl_path("/a:b;/c")` liefert `["/a","b"]` für die Liste `/a:b` und `/c` im Semikolonmodus: es liefert `["/a:b","/c"]`
R2	**B3**	`satisfies("1.5.0", ">=1.2,<2")`, `"1.0.0"`, `">=1.0"`: je `INVALID_CONTAINER`
R3	**B5**	Ein `init`, das einen Thread mit `open` startet und höchstens 2 s wartet: der Thread blieb nach 2,0 s blockiert
R4	**B7**	`open("b")` mit `b` gegen `a_func` aus `a`: `INTERNAL`, `undefined symbol: a_func`, danach leerer Zustand
R5	**B8**	zwei Instanzen von `d` bei Auflösung über `sl/`-Unterverzeichnisse zweier Aufrufer; ein zweites `close` desselben Handles wurde angenommen und entlud `base`, während `top` geladen blieb
R6	**B10**	`failed_name=''` bei beschädigtem Container einer Abhängigkeit und beim Hashfehler; `RecursionError` bei tief geschachteltem JSON; angehängte Bytes ohne Wirkung
R7	**B11**	nach dem Ende eines Prozesses ohne `close`: ein Verzeichnis `sl-*` im Temp-Verzeichnis
R8	**B12**	Symbol `d_init` (Init-Haken, nicht exportiert) → `SYMBOL_NOT_FOUND`, obwohl der Loader es aufgerufen hat
R9	**B13**	`-Wall -Wextra` auf der erzeugten C-Quelle: `-Wunused-const-variable`; Build-Matrix für zwei Linux-Ziele: zweimal `wsat.so`; im CMake-Text kein `CXX_VISIBILITY_PRESET` und kein `set(SL_TARGET_PLATFORM`; deklariertes Ziel `linux:aarch64` fehlt im Container
R10	**B14**	`L.f is L.f` → `True`; `L["f"] is L["f"]` → `False`; `L.__x__` → `AttributeError`, `L["__x__"]()` → `7`; `argtypes` wird zwischen `f1` und `f2` geteilt
R11	**B4**	`ntpath.isabs("C:\\lib")` → `True`, `posixpath.isabs` → `False`; unter Linux ergibt `SL_PATH="C:\lib;D:\lib"` nur die Systemverzeichnisse als Kandidaten

Tabelle 13.2: Reproduktionen. Die Nummern der Befunde verweisen auf Kapitel 14.

und 645 ns für `CDLL` und zwischen 323 und 489 ns für `sl`, ohne gleichbleibende Richtung des Unterschieds (Lauf 1: 341 gegen 347, 362 gegen 489, 359 gegen 354, 309 gegen 323 ns). Das Rauschen der Umgebung überwiegt; ein Zusatzaufwand ist weder zu erwarten (Satz 10.4) noch zu erkennen. **[Messung auf Windows und macOS; Messung mit Bibliotheken der Größe der echten Shims (`csubgraph_shim`, `wsat_shim`)]**

Dateigröße (Bytes)	Lauf 1 (µs)		Lauf 2 (µs)		Lauf 3 (µs)
	CDLL	sl	CDLL	sl	sl
15 232	39,8	351,5	35,7	292,1	281,5
80 768	60,1	484,9	52,7	501,3	487,0
539 520	39,0	961,6	35,6	1 784,5	1 805,7
4 209 536	33,8	4 472,4	40,9	13 760,7	14 145,3

Tabelle 13.3: Median der Zeit für Öffnen, Symbol, ein Aufruf, Schließen. `CDLL` ist von der Größe unabhängig, `sl` wächst mit ihr. Drei Läufe in derselben Umgebung.

13.6 Grenzen der Prüfung

- **Nur Linux.** Die Adapter für macOS und Windows wurden nicht ausgeführt; die Aussagen zu ihnen stützen sich auf Quelltext und Beweise.
- **Fake-Adapter.** Die Modellläufe testen den Loader-Kern, nicht den Adapter. Die Beobachtungen am echten Adapter (R4, R5, R7, R10) sind Einzelfälle.
- **Keine Suite.** Die Testsuite des Repositories wurde nicht ausgeführt (Abschnitt 13.2).
- **Nebenläufigkeit.** Nur R3 prüft einen nebenläufigen Fall; die Verschränkung von `symbol` und `close` (Satz 9.10(d)) wurde nicht dynamisch reproduziert.
- **Zufall.** Zufällige Eingaben erreichen seltene Fälle nicht gesichert; die Beweise decken sie ab, soweit das Modell trägt.

Kapitel 14

Befunde

Dieses Kapitel stellt die fünfzehn Befunde zusammen, die in den Kapiteln 7 bis 12 auftauchten. Jeder Befund hat vier Teile: die *Beobachtung*, den *Beleg* (Satz, Quelltext oder Reproduktion aus Tabelle 13.2), die *Folge* und eine *Verbesserung*. Die Schwere ist eine Einschätzung der Folgen für Nutzer der Referenzimplementierung, keine Messung.

14.1 Pfadkodierung und Suche (B1, B2, B4)

Beobachtung. Die Zerlegung von `SL_PATH` verwendet unter POSIX den Doppelpunkt, unter Windows das Semikolon, und eine Heuristik für Laufwerkspräfixe (`C:\`); je nach Eintragsform ändert sich die Struktur der Liste.

Beleg. Satz 7.2 charakterisiert die Rundreise exakt: $\mathrm{Dec}(\mathrm{Enc}(L)) = L$ genau dann, wenn kein Eintrag die Laufwerksform hat und kein Eintrag das Trennzeichen enthält. R1 zeigt Gegenbeispiele; P1 findet keine Abweichung außerhalb dieser Bedingung.

Folge. Eine Konfiguration, die über die Umgebung weitergereicht wird (Eltern- an Kindprozess), kann ihre Struktur ändern. Die Wirkung ist klein, weil ein falsch zerlegter Eintrag meist nur ein nicht vorhandener Pfad ist; sie wird aber zur Fehlkonfiguration, die erst beim Laden auffällt.

Verbesserung. Eine einzige Kodierung für alle Plattformen mit festem Trennzeichen und Fluchtzeichen: Enc ersetzt `\` durch `\\` und das Trennzeichen durch `\:`; Dec ist die Umkehrung. Dann gilt $\mathrm{Dec} \circ \mathrm{Enc} = \mathrm{id}$ ohne Vorbedingung (Beweis: Induktion über die Zeichen, Fluchtzeichen sind eindeutig als Präfix lesbar). Kein Semikolonmodus, keine Laufwerksheuristik. Der Preis: Windows-Pfade mit Rückstrichen müssen beim Setzen der Variablen verdoppelt werden; die Bibliothek bietet dafür Enc an.

Zu B4. Satz 7.4 beweist die Unabhängigkeit vom Arbeitsverzeichnis unter der Voraussetzung, dass alle Kandidaten absolut sind. Die Prüfung `isabs` ist hostabhängig (R11): `ntpath.isabs` und `posixpath.isabs` urteilen verschieden.

Nr.	Befund (Kurzform)	Beleg	Art	Schwere
B1	`SL_PATH`: Laufwerksheuristik macht die Rundreise formabhängig	Satz 7.2, R1	bewiesen	mittel
B2	`SL_PATH`: Semikolonmodus ist nicht rundreisefähig, Modus hängt vom Inhalt ab	Kor. zu 7.2, R1	bewiesen	mittel
B3	Versionsbereiche des Proposals (`>=1.2,<2`) werden abgelehnt	Abschn. 8.2, R2	geprüft	niedrig
B4	Suche hängt vom Host ab (`isabs`, Arbeitsverzeichnis bei relativen Pfaden)	Satz 7.4, R11	bewiesen	mittel
B5	Sperre während `init`: Nebenläufiges `open` blockiert (Widerspruch zum Proposal)	Satz 9.10(c), R3	reproduziert	hoch
B6	`symbol` und `close` nicht atomar verschränkt	Satz 9.10(d)	aus Quelltext	mittel
B7	Native Symbolabhängigkeit zwischen Bibliotheken bleibt unaufgelöst	Satz 10.5, R4	reproduziert	hoch
B8	Instanz je Datei, nicht je Name; doppeltes `close` wird angenommen	Sätze 9.9, 9.3, R5	reproduziert	mittel
B9	Feld `runtime` (`"external"`) hat im Loader keine Semantik	Satz 12.1, Kap. 5	aus Quelltext	niedrig
B10	Metadaten nicht authentifiziert; `failed_name` leer; Rekursionsfehler; nichtkanonische Container	Sätze 8.5, 8.6, R6	gemischt	hoch
B11	Temporäre Dateien bleiben bei Prozessende ohne `close` liegen	Satz 10.6, R7	reproduziert	mittel
B12	Init/Fini-Haken sind Symbole, aber nicht auffindbar	Satz 9.5, R8	reproduziert	niedrig
B13	Codegenerator: `long` nicht zurückgewiesen, Warnungen, Build-Matrix mit gleichem Dateinamen, CMake-Lücken	Satz 5.2, R9	reproduziert	mittel
B14	Binding: Funktionsobjekte nicht stabil, `argtypes` geteilt	Satz 10.8, R10	reproduziert	niedrig
B15	Legacy-Modus des Proposals fehlt	Abschn. 15.1	aus Quelltext	mittel

Tabelle 14.1: Die fünfzehn Befunde. Art: *bewiesen* (aus dem Modell), *reproduziert* (Lauf), *geprüft*, *aus Quelltext* (gelesen, nicht ausgeführt).

Verbesserung: Die Suche entscheidet über Absolutheit nach der Konvention des *Ziels* (Modell: Parameter π), nicht des Hosts; Verzeichnisse des Aufrufers und der Anwendung werden vor der Verwendung absolut gemacht; für `hardened` wird zusätzlich das Eigentum geprüft (Ausblick, Abschnitt 17.7).

14.2 Versionen (B3)

Das Proposal nennt in Anhang A `>=1.2,<2`; die Implementierung verlangt dreiteilige Versionen. Beleg: R2. Die Folge ist ein Ladefehler (`INVALID_CONTAINER`) bei einer im Proposal zulässigen Eingabe. Verbesserung: Teilversionen als Abkürzung ($1.2 \mapsto 1.2.0$, $2 \mapsto 2.0.0$) in $\mathbb{N}^3$ aufnehmen; die Sätze 8.1 und 8.2 bleiben gültig, weil die Abbildung monoton und injektiv auf die Normalformen ist. Alternativ das Proposal anpassen.

14.3 Nebenläufigkeit (B5, B6)

Das Proposal verlangt, dass eine Initialisierung nebenläufige Öffnungen anderer Bibliotheken nicht sperrt; der Loader hält die Sperre während `init`. **B5** ist damit ein *Widerspruch* zum Proposal, kein Mangel des Modells: R3 zeigt den Thread, der blockiert bleibt. Folge: Ein `init`, das über andere Threads öffnet, blockiert; mit nur einer Sperre auch Selbstverklemmung bei Rückruf. Verbesserung: Zustand zuerst unter der Sperre eintragen (Status *initializing*), Sperre freigeben, `init` aufrufen, Sperre erneut nehmen und den Status auf *ready* setzen; Wartende auf denselben Schlüssel warten auf eine Bedingungsvariable. Das erhält die Invariante, wenn Schlüssel im Zustand *initializing* in F gezählt werden (Definition 9.4). **B6**: `symbol` liest den Handle-Zustand, `close` kann dazwischen entladen; Verbesserung: ein Aufruf hält einen Verweis (Zähler) für die Dauer der Auflösung.

14.4 Native Abhängigkeiten (B7)

Satz 10.5 zeigt: Der Loader löst Abhängigkeiten der *Metadaten* auf, aber nicht die dynamischen Symbole des Maschinencodes. Eine Bibliothek, die `a_func` aus `a` importiert, scheitert beim Öffnen mit einem Systemfehler (R4), obwohl `a` zuvor geladen wurde; der Rollback hinterlässt einen leeren Zustand (Satz 9.4), der Fehler ist also sauber, aber vermeidbar. Verbesserung: Abhängigkeiten global verfügbar machen (`RTLD_GLOBAL`) oder in ein gemeinsames Verzeichnis entpacken, in dem der native Lader sie über den Namen findet; Folgen und Entwurfsalternativen im Ausblick (Abschnitt 17.5).

14.5 Instanzen, Authentizität, Aufräumen (B8 bis B12)

B8. Der Schlüssel ist ein Dateipfad, kein logischer Name (Satz 9.9); zwei Aufrufer mit verschiedenen `sl/`-Verzeichnissen laden zwei Instanzen. Doppeltes `close` desselben Handles wird nicht erkannt (Satz 9.3, Vorbedingung; R5). Verbesserung: Handles tragen ein Token; `close` prüft, ob das Token noch gültig ist.

B9. Der Container führt ein Feld `runtime`, das die Laufzeitbibliothek des Payloads benennt (`"external"`: Laufzeit liegt außerhalb). Der Loader liest es nicht aus; es verhindert keine Mischung von MinGW- und MSVC-Laufzeiten und ersetzt keine Suche nach der Laufzeit (Bedingung M2 und M3 von Satz 12.1). Verbesserung: entweder dem Feld eine Semantik geben (Laufzeit als Abhängigkeit im Sinn von **B7** auflösen und prüfen) oder es streichen.

B10. Satz 8.5: Die Metadaten sind weder gehasht noch signiert; R6 zeigt zudem leeres `failed_name` bei Fehlern des Containers einer Abhängigkeit, `RecursionError` bei tiefer Schachtelung und die Akzeptanz angehängter Bytes (Satz 8.7). Verbesserung: Hash über Kopf, Metadaten und Tabelle als Teil des Containers; Tiefenschranke beim Parsen; Kanonizität erzwingen (Länge = Dateigröße).

B11. Extrahierte Dateien werden bei geordnetem `close` entfernt, nicht bei Abbruch (Satz 10.6; R7). Verbesserung: Verzeichnis beim Start aufräumen (Prozess-ID im Namen, Prüfung auf Lebendigkeit) und, wo möglich, anonyme Dateien (`memfd_create`, `O_TMPFILE`).

B12. Init-Haken werden als Symbole der Bibliothek aufgerufen, aber nicht in der Exportliste geführt; `symbol("d_init")` scheitert (R8). Das ist eine Spezifikationslücke: Das Proposal sagt nicht, ob Haken Exporte sind. Verbesserung: festlegen, dass Haken nicht Teil der Exporte sind, und es im Proposal ausdrücklich sagen.

14.6 Codegenerator und Binding (B13, B14)

B13 bündelt vier Beobachtungen (R9): Der Parser weist `long` nicht zurück, obwohl das Proposal feste Breiten verlangt (Satz 5.2); die erzeugte C-Quelle warnt unter `-Wall -Wextra`; die Build-Matrix erzeugt für verschiedene Ziele denselben Dateinamen (`wsat.so`), sodass das zweite Ziel das erste überschreibt; im CMake-Text fehlen Sichtbarkeit und Zielplattform. Verbesserung: Typprüfung gegen {`int8_t`, …, `uint64_t`, `float`, `double`, `void*`}; Zielname als Teil des Ausgabepfads; ein Test je Warnung. **B14**: `L.f is L.f`, aber `L["f"] is L["f"]` falsch; `argtypes` wird zwischen Funktionen geteilt; Namen mit doppelten Unterstrichen sind nur per Elementzugriff erreichbar (R10, Satz 10.8). Verbesserung: Funktionsobjekte einmal je Symbol erzeugen und zwischenspeichern, einen Zugriffsweg.

14.7 Legacy-Modus (B15)

Das Proposal sieht vor, gewöhnliche `.dll`-, `.so`- und `.dylib`-Dateien über dieselbe API zu öffnen. Die Referenzimplementierung öffnet nur Container

(Abschnitt 15.1). Das ist keine Fehlfunktion, sondern ein nicht umgesetzter Teil; die Konsequenz betrifft die Migration: Bestehende Bibliotheken müssen neu verpackt werden (Satz 12.1).

14.8 Gewichtung

Hoch sind **B5**, **B7** und **B10**: Sie betreffen Korrektheit unter Nebenläufigkeit, das Kernversprechen der Abhängigkeitsauflösung und die Vertrauensgrenze. Die übrigen sind lokal behebbar. Keine der Verbesserungen ändert das Modell der Kapitel 6 bis 9; **B5** und **B8** erweitern die Zustandsmenge und müssen im Beweis von Satz 9.3 nachgezogen werden **[Neubeweis der Invariante für das verbesserte Protokoll von B5]**.

Kapitel 15

Konformität von Proposal und Implementierung

Tabelle 15.1 gleicht die Zusagen des Proposals mit der Referenzimplementierung ab. Die Spalte *Status* nennt *erfüllt* (bewiesen oder geprüft), *teilweise* (mit Befund) und *fehlt*. Die Abschnittsnummern verweisen auf das Proposal [3].

Von den siebzehn Zeilen sind fünf erfüllt, zehn teilweise und zwei fehlen (eine Zeile ist nur auf Linux belegt). Die Lücken sind überwiegend lokal; sie widersprechen dem Proposal in Details, nicht im Entwurf. Die zwei fehlenden Punkte sind der Legacy-Modus und die entkoppelte Initialisierung. Das zweite ist ein Verstoß gegen das Proposal, das erste eine Auslassung.

15.1 Der Legacy-Modus

Der Legacy-Modus soll bestehende Bibliotheken unverändert öffnen, sodass Projekte sich schrittweise der Schicht nähern können (Satz 12.1, Stufe 1). Formal wäre er eine zweite Quelle von Kandidaten: Statt $\mathrm{unpack}(f)$ liefert er eine Datei, die der native Lader öffnet, mit leeren Metadaten ($\mathrm{deps} = \emptyset$, exports = alle Symbole). Zu klären ist, wie die Exportliste ohne Metadaten entsteht (Symboltabelle lesen oder ungeprüft lassen) und wie Versionen fehlen. Sinnvoll ist, den Modus erst nach **B7** umzusetzen, denn gerade bestehende Bibliotheken bringen native Abhängigkeiten mit.

15.2 Erzeugen und Aufrufen

Die Codegenerierung braucht für das Übersetzen einen C-Compiler.

- `sl gen` erzeugt aus einer Beschreibung Header, C-Quelle, CMake-Datei und Binding; `sl build` und `sl matrix` bauen für ein oder mehrere Ziele; `sl pack` packt die Payloads in einen Container; `sl info` liest Metadaten und Tabelle aus.

- Im Programm: `open`, `symbol`, `close` der C-ähnlichen API (Rückgabe $(\mathrm{status}, \mathrm{value}, \mathrm{ErrorInfo})$) oder die Python-Fassung mit Ausnahmen.

Zusage des Proposals	Abschn.	Stand in `sl`	Status
Einheitlicher Container mit Metadaten, Tabelle und SHA-256 je Payload	5.1	Rundreise und Injektivität (Satz 8.3); Payload-Integrität (Satz 8.4); Metadaten nicht gehasht (**B10**)	teilweise
Feste Suchordnung, nie Arbeitsverzeichnis, `PATH`, `LD_LIBRARY_PATH`	5.3	Satz 7.3, 7.4; Host-Abhängigkeit (**B4**)	teilweise
Gehärteter Modus: fail closed	10	Satz 7.5; Eigentumsprüfung fehlt	teilweise
Abhängigkeiten vor der Bibliothek auflösen	5.3 (4)	Metadaten ja; native Symbole nein (**B7**)	teilweise
Zyklen erkennen	5.3	Satz 9.2	erfüllt
Referenzzählung, Entladen beim letzten `close`	5.4	Satz 9.3; doppeltes `close` nicht erkannt (**B8**)	teilweise
Atomares Öffnen (kein Zustand nach Fehler)	5.6	Satz 9.4	erfüllt
Init Abhängigkeiten zuerst, Fini zuletzt	5.3 (6, 9)	Satz 9.5 (Lesart: Abhängige vor Abhängigkeiten beim Entladen)	erfüllt
`failed_name` nennt den Verursacher	5.6	Satz 9.7 für fehlende Abhängigkeiten; sonst leer (**B10**)	teilweise
Nebenläufige Öffnungen sperren sich nicht bei `init`	5.5	Sperre während `init` (**B5**)	fehlt
Eine Instanz je Bibliothek	5.4	eine je *Datei* (**B8**)	teilweise
Versionsbereiche `>=1.2,<2`	A	dreiteilige Versionen verlangt (**B3**)	teilweise
Feste Typbreiten in der Schnittstelle	5.8	`long` wird nicht zurückgewiesen (**B13**)	teilweise
Codegenerierung: Header, C-Quelle, CMake, Binding, Build-Matrix	7	funktional; Warnungen, gleiche Dateinamen (**B13**)	teilweise
Plattformadapter austauschbar	5.2	Satz 10.1, 10.2; Linux ausgeführt, Windows und macOS nicht	erfüllt (Linux)
Legacy-Modus für `.dll`/`.so`/`.dylib`	6	nicht umgesetzt (**B15**)	fehlt
Keine Laufzeitabhängigkeiten	5.1	nur Standardbibliothek (Quelltext gelesen)	erfüllt

Tabelle 15.1: Konformität der Referenzimplementierung mit dem Proposal.

– Reproduktion der Prüfläufe: Anhang B.

Kapitel 16

Zusammenfassung

Ausgangspunkt. Die Vorarbeit [1] endet im Gedanken einer einheitlichen Shared Library mit standardisiertem Ladeverfahren. Diese Arbeit hat den Gedanken in ein mathematisches Modell überführt, die Eigenschaften des Proposals bewiesen, die Referenzimplementierung `sharedlib` dagegen geprüft und das Ergebnis gemessen.

Was sich beweisen lässt. Maschinencode lässt sich zwischen Aufrufkonventionen nicht vereinheitlichen (Satz 5.1); ebenso wenig Typbreiten ohne feste Typen (Satz 5.2). Vereinheitlichen lassen sich Suche, Container, Versionen und Lebenszyklus, und genau diese Teile hat das Modell erfasst:

- *Suche.* Erster Treffer, Unabhängigkeit vom Arbeitsverzeichnis und fail closed im gehärteten Modus (Sätze 7.3, 7.4, 7.5); die Rundreise von `SL_PATH` gilt genau außerhalb zweier Ausnahmefälle (Satz 7.2).
- *Container.* Rundreise, Injektivität, Integrität der Payloads, Totalität des Lesers (Sätze 8.3, 8.4, 8.6); Authentizität der Metadaten gibt es nicht (Satz 8.5).
- *Versionen.* Bereiche sind konvex; die kleinste erfüllende Version ist die untere Grenze (Sätze 8.1, 8.2).
- *Loader.* Terminierung mit höchstens $|E_r| + 1$ Aufrufen, korrekte und vollständige Zykluserkennung, Invariante der Referenzzählung, Rollback, Ordnung von Init und Fini, Fehlerkausalität (Sätze 9.1, 9.2, 9.3, 9.4, 9.5, 9.7).
- *Adapter.* Der Kern ist adapterunabhängig, aber nicht hostunabhängig (Sätze 10.1, 10.2, 10.3); das Öffnen kostet linear in der Dateigröße (Satz 10.4).

Was die Läufe zeigen. $300\,000$ Pfadlisten, $100\,000$ Signaturen, $3\,000$ Modellläufe mit $22\,559$ Invariantenprüfungen ohne Verletzung stützen die Beweise (Tabelle 13.1). Das Öffnen kostet einige hundert Mikrosekunden plus ein bis drei Millisekunden je Mebibyte; der Aufruf einer gebundenen Funktion kostet nicht mehr (Abschnitt 13.5).

Was nicht stimmt. Fünfzehn Befunde (Tabelle 14.1): Drei wiegen schwer, nämlich die Sperre während der Initialisierung (**B5**, Widerspruch zum Proposal), die unaufgelöste native Symbolabhängigkeit (**B7**) und die fehlende Authentizität der Metadaten (**B10**). Die Konformitätstabelle zeigt fünf erfüllte, zehn teilweise erfüllte und zwei fehlende Zusagen (Tabelle 15.1).

Anwendung. Für Gen-DB und `subgraph-sat-solver-econ` führt die Migration, sofern M1 bis M4 gelten, zu gleichem Verhalten bei weniger Plattformcode (Satz 12.1); fünf Wiederverwendungen **W19** bis **W23** sind als Kandidaten benannt.

Grenzen. Gelaufen ist nur Linux; Windows und macOS stützen sich auf Quelltext und Beweis. Die Testsuite des Repositories wurde nicht ausgeführt, die Quellen der Projekte Gen-DB, Solver und `csubgraph` wurden nicht erneut geprüft, und der Wortlaut von Kapitel 13 der Vorarbeit lag nicht vor. Offene Punkte stehen rot im Text.

Gesamturteil. Die Idee trägt: Der Kern ist klein, im Modell beweisbar und unabhängig vom Betriebssystem, soweit der Adapter den Vertrag einhält. Die Referenzimplementierung ist ein belastbarer Prototyp mit behebbaren Lücken, aber noch kein Standard.

Kapitel 17

Ausblick

Dieses Kapitel leitet aus den Befunden (Kapitel 14), den offenen Stellen im Text (rot gesetzt) und den Entwurfsfragen des Proposals ein Arbeitsprogramm ab. Es ist nach Zeithorizont und Thema geordnet. Aussagen über Aufwand sind Schätzungen des Verfassers, keine Messungen; Aussagen über Eigenschaften zukünftiger Verfahren sind als *Vermutung*, *Beweisskizze* oder *Beweis* gekennzeichnet. Wo ein Satz bewiesen wird, geschieht es vollständig; Skizzen sind als solche benannt.

17.1 Überblick und Reihenfolge

Die Vorhaben hängen voneinander ab. Tabelle 17.1 ordnet sie in vier Stufen; innerhalb einer Stufe sind sie unabhängig.

17.2 Kurzfristig: Befunde beheben und Prüfung vervollständigen

17.2.1 Eine Änderung je Befund, ein Test je Änderung

Jede Verbesserung aus Kapitel 14 sollte als eigene Änderung mit einem Test kommen, der den Befund vorher reproduziert (Tabelle 13.2) und nachher besteht. Das macht jede Zeile der Tabelle zu einem Regressionstest und verhindert, dass Befunde unbemerkt zurückkehren. Die Reihenfolge nach Schwere und Abhängigkeit: erst **B10** (Container ändert sich, alles Weitere liest ihn), dann **B1**, **B2**, **B4** (Pfade), dann **B7** und **B5**, dann der Rest.

17.2.2 Die ausstehende Prüfung

- *Testsuite ausführen* und die 126 Tests des READMEs nachzählen; die Abdeckung (Abschnitt 13.2) unter Linux, macOS und Windows messen, Zweigabdeckung eingeschlossen.
- *Adapter auf Windows und macOS ausführen.* Die Modellläufe (P5) benutzen einen Fake-Adapter; derselbe Zufallstest gegen die echten

Stufe	Vorhaben	Voraussetzung / Ertrag	Aufwand
A	Befunde **B1** bis **B15** beheben; Tests je Befund	Kap. 14; schließt die Lücke zum Proposal	Tage bis Wochen
A	Testsuite ausführen, Windows und macOS in die Prüfung aufnehmen	Abschn. 13.2; macht Aussagen zu Adaptern prüfbar	Tage
B	Verbessertes Sperrprotokoll (**B5**) mit Neubeweis, Modellprüfung	Abschn. 17.6; ersetzt den roten Vermerk in Kap. 14	Wochen
B	Konformitätssuite aus dem Modell erzeugen	Abschn. 17.4; Voraussetzung für Zweitimplementierungen	Wochen
B	Native Abhängigkeiten lösen (**B7**)	Abschn. 17.5; macht die Auflösung vollständig	Wochen
B	Authentifizierter Container, Signaturen	Abschn. 17.7; schließt **B10**	Wochen
C	Resolver mit SAT/PubGrub	Abschn. 17.8; verbindet mit der Subgraph-SAT-Kette	Monate
C	Loader-Kern in C oder Rust, Bindings für weitere Sprachen	Abschn. 17.9, 17.9.2	Monate
C	Mechanisierte Beweise (Coq/Isabelle/TLA$^+$)	Abschn. 17.3	Monate
D	Eigener Abbilder für ELF, PE, Mach-O	Abschn. 17.5.4; hebt die Bindung an `dlopen`	Jahre
D	Standardisierung, Governance	Abschn. 17.12	Jahre

Tabelle 17.1: Arbeitsprogramm. Aufwand ist die Schätzung für eine Person.

Adapter würde den Vertrag des Adapters (Kapitel 10) dynamisch prüfen.

- *Messung wiederholen* mit den echten Shim-Bibliotheken von Gen-DB und `subgraph-sat-solver-econ`, mit warmem und kaltem Cache und auf einem Dateisystem im Arbeitsspeicher, um die Streuung der Steigung (Faktor $3{,}4$ zwischen den Läufen) zu erklären.
- *Die Projekte selbst prüfen:* Die Aussagen über Gen-DB, den Solver und `csubgraph` stammen aus der Vorarbeit (Tabelle 3.3); ein Lauf der Migration (Satz 12.1) auf einer Kopie jedes Projekts würde M1 bis M4

prüfen statt annehmen.

17.3 Mechanisierte Beweise

Die Beweise dieser Arbeit sind von Hand geführt und durch Läufe gestützt. Mechanisierung schließt den Rest.

17.3.1 Zustandsmaschine in TLA$^+$ oder Alloy

Das Modell der Kapitel 6 und 9 ist endlich beschreibbar (endliche Menge von Bibliotheken, Kanten und Handles). Ein Modellprüfer kann die Invariante Inv für alle Zustände bis zu einer Größe erschöpfend prüfen, nicht nur für Zufallsläufe. Der Gewinn gegenüber P5: Vollständigkeit bis zur Schranke, Gegenbeispiele als kürzeste Spuren. Das Protokoll von **B5** (Abschnitt 17.6) ist das erste Ziel, denn dort ist von Hand am leichtesten ein Fehler möglich. *Vermutung:* Eine Schranke von vier Bibliotheken, zwei Threads und drei Handles genügt, um jede Verletzung der Invariante in einem Protokoll dieser Bauart zu finden (Kleine-Modell-Eigenschaft, nicht bewiesen).

17.3.2 Interaktiver Beweisassistent

Die Sätze über Terminierung, Zykluserkennung, Referenzzählung und Rollback sind für *beliebig große* Graphen formuliert; hier braucht es Induktion, die ein Modellprüfer nicht liefert. In Coq oder Isabelle/HOL lassen sich die Zustandsübergänge als induktive Relation und die Invariante als Prädikat formulieren; der Beweis von Satz 9.3 ist eine Induktion über die Übergänge, die sich fast wörtlich übertragen lässt. Eine Beweisverpflichtung je Satz:

(i) *Sicherheit:* jede erreichbare Konfiguration erfüllt Inv;

(ii) *Lebendigkeit:* jede Öffnung terminiert mit Erfolg oder Fehler (Satz 9.1);

(iii) *Atomarität:* nach einer fehlgeschlagenen Öffnung ist der Zustand gleich dem vorherigen (Satz 9.4);

(iv) *Ordnung:* Init- und Fini-Folge sind lineare Erweiterungen der Abhängigkeitsordnung (Satz 9.5).

Aufwandsschätzung: einige Wochen für die Modelle, einige Monate für die Beweise, wenn die Zustandsmaschine zuerst vereinfacht wird (Streichung der Fehlerursachen, die nur die Meldung betreffen).

17.3.3 Verfeinerung zum Quelltext

Die Lücke zwischen Modell und Quelltext schließt eine *Verfeinerung*: Eine Abbildung α von Programmzuständen auf Modellzustände, sodass jeder

Schritt des Programms einem Schritt des Modells oder einem Stottern entspricht. Für den Python-Quelltext ist das nicht mechanisch machbar; für einen Kern in Rust oder C (Abschnitt 17.9) mit einem Verifikationswerkzeug (Kani, Frama-C, VST) ist es das; Vorbild sind die Verifikation von Betriebssystemkernen [10] und Übersetzern [11]. Ein verifizierter Loader-Kern mit wenigen hundert Zeilen ist erreichbar; ein verifizierter Plattformadapter nicht, weil sein Verhalten das des Betriebssystems ist. Der Beweis bleibt dann *relativ zum Adaptervertrag* (Satz 10.2).

17.3.4 Zufalls- und Fuzztests des Containers

Der Container-Leser ist die Stelle, an der fremde Daten auf Code treffen. P3c ist ein Anfang; ein dauerhaft laufender, abdeckungsgeführter Fuzzer (etwa mit `atheris` oder `libFuzzer` gegen einen Leser in C) mit dem Orakel *nur Erfolg oder* `SLError` würde Satz 8.6 fortlaufend prüfen. Zusätzlich lässt sich differenziell testen: Python-Leser und C-Leser müssen auf jeder Eingabe dasselbe Ergebnis liefern (Zweitimplementierung als Orakel).

17.4 Spezifikation und Konformitätssuite

17.4.1 Spezifikationstext

Das Proposal ist erzählend; Lücken wie **B12** (sind Haken Exporte?) und die Mehrdeutigkeit der Ordnung beim Entladen (Abschnitt zu Satz 9.5) zeigen, dass eine normative Fassung nötig ist. Vorschlag: eine Spezifikation in drei Schichten.

1. *Format:* Bytes des Containers (ein Dokument, Rundreise und Kanonizität als Anforderungen, Satz 8.3, 8.7).
2. *Verfahren:* Suche, Auflösung, Öffnen, Schließen als Zustandsmaschine mit Vor- und Nachbedingungen; das Modell dieser Arbeit liefert die Gerüste.
3. *Schnittstelle:* C-API, Fehlercodes, Handles (Satz 10.7).

Normative Wörter (MUSS, SOLL, KANN) nach RFC 2119 und ein Abschnitt *Nicht festgelegt*, der offen lässt, was der Adapter bestimmt, entsprechen der Unterscheidung dieser Arbeit zwischen Kern und Adapter.

17.4.2 Konformitätssuite aus dem Modell

Das Modell liefert ein Orakel: Zu jeder Folge von Aufrufen und jeder Wahl von Dateisystem und Umgebung bestimmt es Ergebnis und Folgezustand. Daraus lässt sich eine Suite erzeugen, die jede Implementierung (Python, C, Rust, Go) gegen dasselbe Orakel prüft: Die Suite würfelt Dateisysteme (Container mit Abhängigkeiten, Zyklen, defekten Dateien) und Aufruffolgen, führt sie gegen den Prüfling und das Modell aus und vergleicht Rückgabecodes, `failed_name` und die Zählerstände. P5 ist ein Prototyp dieser

Suite für den Fake-Adapter. Der Schritt dazu: die Fake-Dateisysteme als *Testvektoren* serialisieren (Container und erwartete Ergebnisse), die jede Sprache einliest. Das ist das Mittel, mit dem sich Aussagen der Art “die Schicht verhält sich überall gleich” in Aussagen über Implementierungen verwandeln.

17.4.3 Zu klärende Entwurfsfragen

- *Identität:* Ist die Instanz durch die Datei (Ist-Zustand, **B8**), den logischen Namen oder den Inhaltshash bestimmt? Der Hash als Schlüssel macht zwei Kopien derselben Bibliothek zu einer Instanz und ist konsistent mit der inhaltsadressierten Ablage (Abschnitt 17.5.2); der Preis ist, dass bewusst getrennte Instanzen einen anderen Mechanismus brauchen.
- *Entladen:* `dlclose` entlädt nicht immer (Symbole mit `STB_GNU_UNIQUE`, Thread-lokaler Speicher, Hooks); die Aussage “entladen” ist im Modell die Abmeldung des Handles. Die Spezifikation sollte es als *Soll* fassen.
- *Fehlerkodierung:* Ob der Fehlercode eine Aufzählung oder eine offene Menge ist, bestimmt, ob Erweiterungen kompatibel sind.
- *Versionierung des Containers selbst:* Der Kopf trägt eine Formatversion; die Regeln, wann ein Leser unbekannte Felder ignoriert, fehlen.

17.5 Laden: Abbilder, native Abhängigkeiten und Entladen

17.5.1 Native Abhängigkeiten lösen (B7)

Der Befund ist, dass die Metadatenabhängigkeit nicht bis zum nativen Lader reicht (Satz 10.5). Drei Entwurfsvarianten.

(V1) Globale Sichtbarkeit. Abhängigkeiten werden mit `RTLD_GLOBAL` geöffnet, sodass der native Lader sie beim Auflösen der Symbole der abhängigen Bibliothek findet. Einfach; aber globale Symbolräume haben eine Wirkung auf alle später geladenen Bibliotheken (Interposition) und gibt es unter Windows nicht (dort sind Abhängigkeiten Namen von Bibliotheken, die im Suchpfad liegen müssen).

(V2) Gemeinsames Ablageverzeichnis. Alle Payloads eines Prozesses (oder einer Abhängigkeitsgruppe) werden in *ein* Verzeichnis extrahiert, mit den Namen, die der native Lader erwartet (soname, Importname). Der native Lader findet sie über den Pfad des Verzeichnisses (`RUNPATH`, `SetDllDirectory`, `AddDllDirectory`). Plattformneutral im Modell: Ein Adapter bildet logische Namen auf Dateinamen ab; der Beweisbedarf ist, dass diese Abbildung injektiv ist (zwei Bibliotheken mit demselben Dateinamen und verschiedenem Inhalt dürfen nicht kollidieren).

(V3) Eigener Auflöser. Der Loader liest die Importtabelle des Payloads (ELF `DT_NEEDED`, PE-Importverzeichnis, Mach-O `LC_LOAD_DYLIB`), bildet sie auf die Metadatenabhängigkeiten ab und bindet selbst. Das ist der Abbilder von Abschnitt 17.5.4.

Empfehlung: (V2) als nächste Stufe, weil sie auf allen drei Systemen mit dokumentierten Mitteln auskommt und die Modellannahme (Satz 10.5) zu erfüllen erlaubt: Das Eintreffen der Abhängigkeit im Verzeichnis *vor* dem Öffnen der abhängigen Bibliothek folgt aus der Ordnung der Auflösung (Satz 9.5).

17.5.2 Inhaltsadressierte gemeinsame Ablage

Der Ist-Zustand extrahiert jeden Payload in ein eigenes temporäres Verzeichnis (Befunde **B11**, **B8**). Eine Ablage nach Hash, store/sha256(p)/name, bringt: (1) Wiederverwendung über Prozesse (der Pool von Gen-DB, Satz 12.2, spart den Faktor N im Plattenverbrauch); (2) Idempotenz: Wer denselben Payload zweimal extrahiert, schreibt dieselbe Datei; (3) Aufräumen als Speicherbereinigung mit Verweiszählung über Prozesse statt Aufräumen beim Beenden. Sicherheitsfragen: Das Verzeichnis darf nur dem Nutzer gehören (Modus, Eigentum), Dateien werden schreibgeschützt gesetzt, und die Hash-Prüfung beim Lesen verhindert Manipulation nach dem Extrahieren (*time of check to time of use* bleibt, wenn der Angreifer das Verzeichnis schreiben kann).

17.5.3 Ohne Extraktion: Speicherdateien und Streaming

Unter Linux erlaubt `memfd_create` (und `dlopen` über `/proc/self/fd/N`) das Laden ohne Datei im Dateisystem; macOS und Windows haben keinen gleichwertigen Weg ohne eigenen Abbilder. Der Container kann dazu seine Payloads *seitenausgerichtet* ablegen, sodass ein Abbilder sie unmittelbar mit `mmap` abbilden kann; das ergäbe das Öffnen in $O(1)$ statt $O(s)$ (Satz 10.4) und würde die gemessene Steigung von ein bis drei Millisekunden je Mebibyte beseitigen. *Vermutung:* Bei Bibliotheken von mehreren Mebibyte wäre der Gewinn deutlich, bei den heutigen Shim-Bibliotheken nur klein.

17.5.4 Eigener Abbilder für ELF, PE und Mach-O

Der größte Schritt ist, `dlopen` und `LoadLibrary` zu umgehen. Dann ist der Loader nicht mehr an die Schleife der Betriebssysteme gebunden: Er bestimmt, wohin abgebildet wird, löst Importe nach den Metadaten auf, führt Relokationen selbst durch und ruft die Initialisierer in der Reihenfolge von Satz 9.5 auf. Gewinne: (1) native Abhängigkeiten sind gelöst, ohne globale Namensräume (**B7**); (2) kein Temp-Verzeichnis (**B11**); (3) das Verhalten ist auf allen Plattformen dasselbe, soweit das Abbilden es ist; (4) Entladen ist

definierbar. Kosten: Der Abbilder muss Relokationen, thread-lokalen Speicher, `.init_array`, Ausnahmetabellen (`.eh_frame`, `RtlAddFunctionTable`) und die Eigenheiten jeder ABI kennen; Fehler sind Abstürze. Referenz sind die Lader selbst [4, 5]. Teilschritte mit Nutzen schon vorher: erst nur lesen (Importtabelle auslesen und gegen die Metadaten prüfen: das findet **B7**-Fälle *vor* dem Laden und ist ein reiner Lesevorgang), dann abbilden.

17.5.5 Threads, Entladen und Ausnahmen

- *Thread-lokaler Speicher und Entladen:* Eine Bibliothek, die Threads startet oder Rückrufe registriert, darf nicht entladen werden, solange diese laufen; der Loader kennt das nicht. Eine Option "nie entladen" (`RTLD_NODELETE`) und die Zusage "entladen nur auf Wunsch" wären sichere Voreinstellungen.
- *C++-Ausnahmen über die Grenze:* Eine Ausnahme, die die C-Schnittstelle verlässt, ist undefiniertes Verhalten. Die erzeugte Hülle sollte jede exportierte Funktion mit `try/catch` umgeben und einen Fehlercode zurückgeben; das ist eine Eigenschaft des Codegenerators (Kapitel 11), die sich als Satz formulieren und testen lässt.
- *Laufzeiten:* Mehrere C++-Laufzeiten in einem Prozess (**B9**); das Feld `runtime` verdient eine Semantik, die Konflikte beim Öffnen erkennt.

17.6 Nebenläufigkeit

17.6.1 Das Protokoll für B5

Vorschlag eines Protokolls, das die Sperre während `init` aufgibt. Jeder Schlüssel k hat einen Zustand $\sigma(k) \in \{\mathsf{absent}, \mathsf{init}, \mathsf{ready}, \mathsf{fini}\}$ und eine Bedingungsvariable.

1. *open*(k) nimmt die globale Sperre. Ist $\sigma(k) = \mathsf{absent}$: setze $\sigma(k) = \mathsf{init}$, löse die Abhängigkeiten rekursiv (Sperre ggf. freigeben), gib die Sperre frei und rufe `init` auf, nimm die Sperre, setze $\sigma(k) = \mathsf{ready}$ und wecke Wartende. Ist $\sigma(k) = \mathsf{init}$ und der Aufrufer ein *anderer* Thread: warte auf ready oder Scheitern. Ist der Aufrufer derselbe Thread (Zyklus): Fehler (Zykluserkennung, Satz 9.2).
2. Scheitert `init`: Rollback wie im Satz 9.4, $\sigma(k) = \mathsf{absent}$, Wartende melden denselben Fehler.
3. *close*(k) setzt bei Zähler 0 $\sigma(k) = \mathsf{fini}$, gibt die Sperre frei, ruft `fini`, nimmt die Sperre, setzt absent.

Zu zeigen (Beweisverpflichtungen): (a) Inv bleibt erhalten, wenn Schlüssel in init und fini als in F gezählt werden; (b) Freiheit von Verklemmung: Der Wartegraph (Thread $\to$ Schlüssel, auf den er wartet) ist zyklenfrei, weil Wartende nur auf Schlüssel warten, deren Besitzer nicht selbst auf sie

warten; die Zykluserkennung bildet auf demselben Thread ab, *Zyklen über zwei Threads* (T_1 öffnet a mit Abhängigkeit b, T_2 öffnet b mit Abhängigkeit a) fangen sie nicht ab, und das ist die eigentliche Schwierigkeit: Es braucht eine globale Ordnung, etwa die Auflösung der *ganzen* Abhängigkeitsmenge unter der Sperre vor jedem `init`, sodass Zyklen vor dem Aufruf eines Haken erkannt werden; (c) kein `init` läuft unter der globalen Sperre. Dieses Protokoll ist ein Kandidat für die Modellprüfung (Abschnitt 17.3).

17.6.2 Feinere Sperren

Eine globale Sperre serialisiert alle Öffnungen, auch unabhängige. Für Dienste mit vielen Threads: Sperre je Schlüssel, Reihenfolge der Sperrnahme nach der Auflösungsordnung (topologisch), schreibfreie Nachschlagepfade für `symbol` (read-copy-update) und ein Atomzähler für den Handle. Beweisbedarf: Linearisierbarkeit der Operationen bezüglich des sequentiellen Modells (Satz 9.10 deckt sie bisher nur für die globale Sperre ab).

17.7 Sicherheit

17.7.1 Der Authentifizierte Container (B10)

Der Ist-Zustand hasht nur die Payloads. Eine Merkle-Struktur [9] über *alle* Teile schließt die Lücke:

$$\rho = H\big(H(\text{Kopf}) \,\|\, H(\text{Metadaten}) \,\|\, H(\text{Tabelle}) \,\|\, h_1 \,\|\, \cdots \,\|\, h_n\big),$$

wobei h_i der Hash des i-ten Payloads ist. Die Signatur $\mathrm{Sig}(\rho)$ liegt in einem Anhang des Containers.

Satz 17.1 (Authentizität des Containers, Beweisskizze)**.** Ist H kollisionsresistent und das Signaturverfahren gegen existenzielle Fälschung unter gewählten Nachrichten sicher, so gilt: Wenn ein Leser einen Container c' mit gültiger Signatur akzeptiert und c der Container ist, den der Schlüsselinhaber signiert hat, dann ist $\mathrm{unpack}(c') = \mathrm{unpack}(c)$, bis auf eine vernachlässigbare Wahrscheinlichkeit.

Skizze. Ist $\rho(c') = \rho(c)$, so folgt aus der Kollisionsresistenz, dass die hashgebundenen Teile von c und c' gleich sind (Kopf, Metadaten, Tabelle, Payloads); nach Satz 8.3 bestimmen diese die Entpackung. Ist $\rho(c') \neq \rho(c)$, so ist $\mathrm{Sig}(\rho(c'))$ eine gültige Signatur auf einer nicht signierten Nachricht, im Widerspruch zur Sicherheit des Verfahrens. Nichtkanonische Anhänge (Satz 8.7) werden durch die Forderung “Länge gleich Dateigröße” ausgeschlossen, andernfalls wäre $c' \neq c$ mit gleicher Entpackung möglich, was dem Satz nicht widerspricht, aber den Container formbar macht. □

Offen ist die *Vertrauensfrage*: Welche Schlüssel gelten als vertrauenswürdig, wie werden sie verteilt und widerrufen? Das ist keine Frage des Loaders, sondern eine des Betriebs (Abschnitt 17.12); der Loader prüft gegen eine konfigurierbare Menge von Schlüsseln und meldet den Fehler, wenn keine Signatur passt.

17.7.2 Der gehärtete Modus

Satz 7.5 gilt für die Reihenfolge der Suche, nicht für das Eigentum. Ein ernsthafter gehärteter Modus prüft zusätzlich, dass jedes Verzeichnis im Pfad dem Benutzer oder dem Administrator gehört und nicht für andere schreibbar ist (rekursiv bis zur Wurzel), dass die Datei nicht über eine symbolische Verknüpfung in ein fremdes Verzeichnis führt, und dass *TOCTOU* (zwischen Prüfung und Öffnen) durch das Arbeiten mit Dateideskriptoren (`openat`, `fstat`) ausgeschlossen wird. Diese Eigenschaften sind *Eigenschaften des Dateisystems*, die das Modell von Kapitel 6 bisher nicht abbildet; ein erweitertes Modell braucht Eigentum und Rechte in FS und Zeit als Parameter.

17.7.3 Isolation

`sl` lädt Code in den Prozess; eine Sicherheitsgrenze ist es nicht (Abschnitt zu Satz 9.8). Für nicht vertrauenswürdigen Code bleibt ein eigener Prozess, eine Sandbox oder die Übersetzung nach WebAssembly (Abschnitt 17.9.4). Der Container kann Fähigkeiten *deklarieren* (Dateizugriff, Netz), die ein Host durchsetzt; das ist eine Erweiterung der Metadaten, die der Loader nicht prüfen kann, aber weiterreichen.

17.7.4 Lieferkette und Reproduzierbarkeit

Signierte Container sind nur so gut wie ihr Bauprozess. Reproduzierbare Builds (bitgleiches Ergebnis aus gleichem Quelltext, festem Compiler, fester Umgebung) erlauben unabhängige Nachprüfung. Der Container ist dafür günstig, weil er kanonisch ist (Satz 8.3): pack ist wiederholbar (P3); bleibt noch, die Payloads reproduzierbar zu bauen (feste Zeitstempel, feste Pfade, keine Zufallsnamen). Eine Software-Stückliste (SBOM) im Metadatenteil, die Bibliotheken, Versionen und Hashes der Abhängigkeiten nennt, ließe sich aus dem Abhängigkeitsgraphen erzeugen.

17.8 Abhängigkeitsauflösung: Resolver und SAT

17.8.1 Von der Prüfung zur Wahl

Der Loader *prüft* jede Kante beim Öffnen (Satz 8.1 erlaubt es, einen Konflikt auch vorher zu entscheiden). Er *wählt* nicht: Findet er zu einem Namen eine Datei, nimmt er sie, auch wenn eine andere Version alle Bedingungen erfüllen würde. Ein Resolver wählt aus einer Menge verfügbarer Container die Versionen, die alle Bereiche erfüllen. Das Problem ist NP-vollständig, wenn mehrere Versionen derselben Bibliothek zu wählen sind und höchstens eine zulässig ist; der Beweis ist bekannt (Reduktion von 3-SAT) und wird hier nicht wiederholt [13]. Praktisch lösen es Verfahren mit Konfliktlernen (PubGrub) oder SAT-Löser [12].

17.8.2 Kodierung nach SAT

Sei $\mathcal{R}$ die Menge der verfügbaren Paare (n, v) (Name, Version), und für (n, v) sei $D(n, v)$ die Menge der Abhängigkeiten (m, S) mit Versionsbereich S. Variablen $x_{n,v}$ für $(n, v) \in \mathcal{R}$. Klauselmenge Φ:

$$
\begin{aligned}
&x_{r,v_r}, && && \text{(Wurzel gewählt)}\\
&\neg x_{n,v} \vee \neg x_{n,w} && (v \neq w), && \text{(höchstens eine Version je Name)}\\
&\neg x_{n,v} \vee \bigvee_{(m,w)\in\mathcal{R},\ w\in S} x_{m,w} && \forall (m, S) \in D(n, v). && \text{(Abhängigkeit erfüllt)}
\end{aligned}
$$

Eine *Auswahl* ist eine Teilmenge $A \subseteq \mathcal{R}$ mit höchstens einem Eintrag je Name, die die Wurzel enthält und *abgeschlossen* ist: Für jedes $(n, v) \in A$ und jedes $(m, S) \in D(n, v)$ gibt es $(m, w) \in A$ mit $w \in S$.

Satz 17.2 (Korrektheit der Kodierung)**.** Die erfüllenden Belegungen von Φ entsprechen bijektiv den Auswahlen: Zu jeder Belegung $\beta \models \Phi$ ist $A_\beta = \{(n, v) \mid \beta(x_{n,v}) = 1\}$ eine Auswahl, und zu jeder Auswahl A ist die charakteristische Belegung β_A erfüllend; $A_{\beta_A} = A$ und $\beta_{A_\beta} = \beta$.

Proof. ($\Rightarrow$) Sei $\beta \models \Phi$. Die erste Klausel gibt $(r, v_r) \in A_\beta$. Die zweite Klausel verbietet zwei Einträge mit gleichem Namen. Für $(n, v) \in A_\beta$ ist $\beta(x_{n,v}) = 1$, also muss jede Abhängigkeitsklausel für (n, v) durch ein wahres Disjunktionsglied erfüllt sein, d. h. es gibt (m, w) mit $w \in S$ und $\beta(x_{m,w}) = 1$, also $(m, w) \in A_\beta$. Damit ist A_β abgeschlossen.
($\Leftarrow$) Sei A eine Auswahl und $\beta_A(x_{n,v}) = 1 \iff (n, v) \in A$. Klausel 1: $(r, v_r) \in A$. Klausel 2: Zwei verschiedene Versionen desselben Namens wären zwei Einträge gleichen Namens, ausgeschlossen. Klausel 3: Ist $\beta_A(x_{n,v}) = 0$, so ist die Klausel wahr; ist es 1, so ist $(n, v) \in A$ und die Abgeschlossenheit liefert ein $(m, w) \in A$ mit $w \in S$, also ein wahres Disjunktionsglied.
Die zwei Abbildungen sind zueinander invers, weil A und β dieselben Daten tragen (die Menge der wahren Variablen). □

Bemerkung 17.1. Die Auswahl enthält möglicherweise überflüssige Einträge (Bibliotheken, die niemand braucht). Minimalität erreicht man, indem man eine Auswahl zum Erreichbaren von der Wurzel zurückschneidet: Das erhält die Abgeschlossenheit. Ferner entscheidet die Kodierung über die Optimierung: Die Zielfunktion “neueste Versionen” ist eine lexikographische Minimierung über die Versionsordnung (Satz 8.2), die ein MaxSAT-Löser oder ein Resolver mit Präferenz (PubGrub wählt in der Reihenfolge der Neuheit) erledigt.

Aus Satz 8.1 folgt, dass jede Klausel 3 eine *Intervallklausel* ist: die Versionen $w \in S$ bilden ein zusammenhängendes Stück der geordneten Versionen von m. Das erlaubt eine kompaktere Kodierung mit Ordnungsvariablen $y_{m,\geq w}$ (“Version von m ist mindestens w”), bei der jede Abhängigkeitsklausel aus zwei Literalen besteht, statt einer Disjunktion über alle Versionen im Bereich: $\neg x_{n,v} \vee y_{m,\geq \mathrm{lo}}$ und $\neg x_{n,v} \vee \neg y_{m,\geq \mathrm{hi}}$. Die Klauselzahl wird linear in der Zahl der Kanten statt in der Zahl der Versionen je Kante; die Korrektheit zu zeigen ist eine Übung nach dem Muster des Beweises oben, mit den Kettenklauseln $y_{m,\geq w'} \to y_{m,\geq w}$ für $w \leq w'$.

17.8.3 Anschluss an die Subgraph-SAT-Kette

Die Vorarbeit [1] verwendet den Subgraph-Algorithmus als Baustein in einer SAT-Kette. Zwei Berührungspunkte:

- *Der Resolver als Anwendungsfall des Solvers:* Abhängigkeitsauflösung ist ein SAT-Problem mit besonderer Struktur (hierarchisch, viele Zwei-Literal-Klauseln). Ob der Solver aus `subgraph-sat-solver-econ` für diese Klasse gut ist, ist eine Messfrage; als Prüfstein taugen reale Abhängigkeitsgraphen von Paketverwaltungen.

- *Der Solver als Bibliothek hinter `.sl`:* Wird der Solver als `.sl`-Container ausgeliefert (Wiederverwendung **W20**), kann der Resolver ihn über die Schicht benutzen; der Loader löst damit *sein eigenes* Abhängigkeitsproblem mit einer Bibliothek, die er lädt. Die Henne-Ei-Frage (der Resolver braucht den Solver, der Solver braucht den Resolver) wird durch einen kleinen eingebauten Resolver für den Solver selbst gelöst.

17.8.4 Weitere Fragen des Resolvers

- *Plattformen:* Die Verfügbarkeit hängt vom Ziel ab ($\mathcal{R}$ ist je Plattform und Architektur zu bilden); Fat-Container (Satz 11.5) verlangen, dass dieselbe Version für alle Ziele vorliegt.

- *Mehrere Instanzen:* Erlaubt man zwei Versionen desselben Namens (wie Rust-Pakete), entfällt Klausel 2 für ausgewählte Namen; die Identität der Instanz ist dann die Frage von Abschnitt 17.4.

- *Sperrdatei:* Die Auswahl A ist ein Wert; er kann in einer Datei mit Hashes gespeichert werden, sodass jeder Lauf dieselben Dateien lädt (Reproduzierbarkeit, Satz 7.3 als Ergänzung).

- *Fehlererklärung:* Ein unerfüllbares Φ liefert einen Kern von Klauseln; die Übersetzung in Meldungen („`b` verlangt `a` ≥ 2, `c` verlangt `a` < 2“) ist das, was Nutzer brauchen, und wiederum die Frage der Verursacherkette (Satz 9.7).

17.9 Kern und Bindungen

17.9.1 Loader-Kern in C oder Rust

Die Referenzimplementierung ist Python; die Bibliotheken, die sie lädt, werden aber in C/C++ geschrieben und von Python, C++, Rust, Go und anderen verwendet. Ein Kern in Rust (oder C) hätte: keine Abhängigkeit von einer Laufzeit (ein Python-Interpreter für das Laden einer Bibliothek für C++ ist ein Hindernis in Gen-DB), Verifizierbarkeit (Abschnitt 17.3), ein stabiles C-ABI als Schnittstelle (Satz 10.7), das jede Sprache anbindet, und geringere Kosten je Öffnung (die Konstante von $0{,}16$ bis $0{,}39\,\mathrm{ms}$ ist überwiegend die Python-Schicht, Vermutung). Schritte: (1) Container-Leser

und -Schreiber (kleinster Teil, am leichtesten differenziell testbar gegen Python), (2) Suche, (3) Kern mit Adaptervertrag als Funktionstabelle, (4) Adapter je Plattform, (5) Python-Bindung über den C-Kern, sodass die Python-Fassung zur Bindung wird.

17.9.2 Weitere Sprachen

Die erzeugte Bindung für Python (Satz 10.8) ist ein Fall; vergleichbar sind C++ (Klasse mit RAII-Handle: `close` im Destruktor), Rust (`Drop`, Lebenszeit des Handles als Lebenszeit der Funktionszeiger, sodass der Übersetzer die Benutzung nach `close` verbietet; das beseitigt **B8** für Rust-Nutzer und **B6** im Ansatz), Go (`cgo`), Java (Panama) und C#. Der Generator braucht dafür nur die Signaturen (Satz 11.2) und die Abbildung der Typen (Satz 11.3); ein Vertrag je Zielsprache sagt, was die Abbildung leistet: Typbreiten, Eigentum an Zeigern, Fehlerbehandlung. Wer Zeiger über die Schnittstelle gibt, braucht eine Regel, wem der Speicher gehört; sie kann der Container deklarieren (Eigenschaften je Signatur), der Generator prüft sie dann gegen die Hülle.

17.9.3 C++-ABI

Die Schnittstelle ist C; eine reine C++-Schnittstelle (Klassen, Vorlagen, Ausnahmen) lässt sich nicht stabil durch eine Containergrenze führen, weil das ABI vom Übersetzer abhängt (Name Mangling, Layout, `libstdc++` gegen `libc++`; Satz 5.1 gilt entsprechend). Wege: Klassen als Handles mit freien Funktionen, oder eine Beschreibung der Klasse im Container, aus der der Generator die Hüllen erzeugt (*Generator* statt *ABI*). Die Shim-Bibliotheken in Gen-DB gehen diesen Weg bereits.

17.9.4 Einbettung in Laufzeiten

Für Plattformen ohne Prozessadressraum (WebAssembly im Browser, auf dem Server) ist der Abbilder (Abschnitt 17.5.4) kein gangbarer Weg, die Idee des Containers aber schon: Ein `.sl`-Container könnte ein Wasm-Modul als eine weitere “Plattform” tragen (`wasm:wasm32`); das Komponentenmodell von WebAssembly beschreibt Schnittstellen in einer eigenen Sprache und löst die Typbreitenfrage (Satz 5.2) auf andere Weise: durch ein kanonisches ABI mit festen Typen. Die Frage ist, ob die Schnittstellenbeschreibung des Containers (Signaturen in C-Syntax) in diese Sprache übersetzbar ist (für die Teilsprache $\mathcal{S}$ von Satz 11.2 wahrscheinlich, für Zeiger auf Strukturen nicht ohne Zusatzangaben). Ein Wasm-Payload wäre zudem in einer Sandbox lauffähig und beantwortet damit die Isolationsfrage (Abschnitt 17.7).

17.10 Build, Paketverwaltung und Betrieb

17.10.1 Build-Matrix

Die Matrix (**B13**) soll für jedes Ziel einen eigenen Ausgabepfad erzeugen und die Payloads zu einem Container zusammenfassen. Eine Hilfsprüfung: *jedes* deklarierte Ziel hat einen Payload, *kein* Payload ohne deklariertes Ziel, jeder Payload hat die Maschine des Ziels (ELF `e_machine`, PE `Machine`, Mach-O `cputype`); die Leseteile liefern der Lader (M2 von Satz 12.1) und der Prüfer. Die Cross-Compilation (Zig, Docker, osxcross) ist ein Betriebsthema; das Modell verlangt nur, dass das Ergebnis die Prüfung besteht.

17.10.2 Anschluss an Paketverwaltungen

Der Container ist ein Artefakt; er kann über bestehende Verteilwege (PyPI-Räder, Conan, vcpkg, Cargo-Kisten, OCI-Images) transportiert werden. Wichtig ist die Zuordnung: Die Version des Containers ist die Version der Bibliothek (Satz 8.2), und der Resolver (Abschnitt 17.8) kann denselben Datenbestand lesen wie die Paketverwaltung. Gen-DB als Python-Paket mit nativen Teilen wäre der erste Fall: ein Rad mit einem Container statt je Plattform ein Rad (Satz 11.5), womit die Menge der Räder von $|P| \cdot |A|$ auf eins schrumpft; ob die Paketindizes Container als Rad akzeptieren, ist eine organisatorische Frage.

17.10.3 Beobachtbarkeit

Ein Loader, der Fehler erklärt, braucht Spuren: welche Kandidaten wurden in welcher Reihenfolge geprüft (die Suchordnung ist im Modell die Liste cand), welcher wurde genommen, warum die anderen nicht. Eine Option für eine *Suchspur* als strukturierte Ausgabe ist billig (die Daten sind da) und gewinnt, sobald zwei Maschinen sich verschieden verhalten (**B4**).

17.11 Anwendungen und Systematik

17.11.1 Gen-DB und der Solver

Die Migration (Satz 12.1) ist als Folge von vier Schritten angelegt: (1) Container aus den vorhandenen Shims packen und M1 bis M4 prüfen, (2) `sl.open` neben dem bestehenden Pfad betreiben und Ergebnisse vergleichen (differenzielle Prüfung), (3) umstellen, (4) den bestehenden Plattformcode streichen. Ein Erfolgsmaß aus Kapitel 12: gestrichene Zeilen Plattformcode, Zahl der Testfälle nach dem Zählsatz (Lemma 4.1), Öffnungszeit im Vergleich (Abschnitt 13.5). Sind die Messungen schlecht, bleibt die Rückfalloption, ohne Schicht weiterzuarbeiten.

17.11.2 Neue Wiederverwendungen

Die Kandidaten **W19** bis **W23** sind ausgeführt, soweit das Proposal sie beschreibt, nicht durchgeführt. Zwei weitere Formen liegen nahe: (a) *Plugins*: Eine Anwendung lädt zur Laufzeit Erweiterungen als Container; der Loader gibt Versionsprüfung und Abhängigkeiten umsonst, die Anwendung die Schnittstelle. (b) *Eingebettete Datenbanken mit Erweiterungen*, wie sie Gen-DB brauchen würde, laden Operatoren als Container; hier wird die Sicherheit (Abschnitt 17.7) wichtig.

17.11.3 Die Systematik der Vorarbeit

Die Wiederverwendungsformen der Vorarbeit lassen sich als Abbildungen zwischen Quellen deuten; die Frage, ob sich zu *jeder* Form formal ein Äquivalenzbegriff angeben lässt (so wie M1 bis M4 die Äquivalenz für die Form “Bibliothek hinter C-Schnittstelle” angeben), ist offen. Der Gewinn wäre ein Katalog der Wiederverwendungen mit je einem Satz “unter diesen Bedingungen sind Original und Wiederverwendung beobachtungsgleich”.

17.12 Standardisierung und Organisation

Ziel und Alternativen. Ein Standard hat nur dann Wert, wenn ihn mehrere unabhängig implementieren und benutzen. Drei Wege: (1) *De-facto-Standard* durch eine gut gepflegte Referenzimplementierung mit offener Spezifikation und Konformitätssuite; (2) *Arbeitsgruppe* bei einer bestehenden Organisation (etwa im Umfeld der Sprachgemeinschaften, die Bibliotheken laden), (3) *formale Normung* (ISO, ECMA), die langsam ist und für ein Format dieser Größe wohl nicht angemessen. Die Stufen 1 und 2 des Proposals brauchen keine Organisation (Kapitel 5); sie sind nur dann tragfähig, wenn die Referenzimplementierung stabil ist.

Voraussetzungen. (a) Spezifikation und Konformitätssuite (Abschnitt 17.4), (b) eine Zweitimplementierung, die die Suite besteht (nur sie zeigt, dass die Spezifikation vollständig ist; Lücken wie **B12** wären so früher gefunden worden), (c) Regeln für Änderungen am Format (Versionierung, Abwärtskompatibilität; der Container trägt bereits eine Formatversion), (d) Lizenz und Patentfrage, (e) Namen und Typ des Mediums (`application/vnd.…`; die Endung `.sl` kollidiert in der Praxis mit anderen Formaten und sollte vorab geprüft werden).

Governance. Wer entscheidet über Änderungen? Ein kleines Gremium mit offenem Verfahren (Änderungsvorschläge als Texte mit Motivation, Sicherheitsabschnitt und Testvektoren) hat sich bei vergleichbaren Formaten bewährt. Aus dieser Arbeit folgt ein Mindestmaß: jeder Änderungsvorschlag nennt, welche Sätze des Modells er berührt und ob sie weiter gelten.

17.13 Bedrohungen der Gültigkeit

- *Modell gegen Wirklichkeit:* Die Sätze gelten im Modell; der Quelltext wurde gegen das Modell gelesen und mit Läufen geprüft, nicht verifiziert (Abschnitt 17.3).
- *Eine Plattform:* Die Läufe sind Linux-Läufe.
- *Ein Prüfer:* Quelltext, Modell und Prüfung stammen vom selben Verfasser; unabhängige Prüfung (und die Zweitimplementierung) ist das Gegenmittel.
- *Annahmen des Adaptervertrags:* Er behauptet Verhalten des Betriebssystems (Atomarität der Zuordnung, Wirkung von `dlclose`), das sich zwischen Versionen ändern kann.
- *Kleine Stichproben:* Drei Messläufe mit großer Streuung der Steigung; die Zahlen sind Größenordnungen.
- *Fehlender Wortlaut:* Kapitel 13 der Vorarbeit lag nicht vor; der Abgleich steht aus (Kapitel 1).

17.14 Konkreter nächster Schritt

Der kleinste Schritt mit dem größten Nutzen ist ein Änderungssatz in drei Teilen: (1) Container mit Hash über alle Teile und Kanonizität (**B10**); das ändert das Format und sollte vor allem anderen kommen, solange es kaum Container gibt; (2) die Pfadkodierung mit Fluchtzeichen (**B1**, **B2**); (3) die Testvektoren aus P3 und P5 als Dateien, die jede künftige Implementierung einliest. Danach folgen **B7** (Variante V2) und das Protokoll von **B5** mit Modellprüfung. Mit diesen Schritten ist die Referenzimplementierung ein tragfähiger Ausgangspunkt für alles Weitere dieses Kapitels.

Anhang A

Beweisindex

Tabelle A.1 nennt zu jeder bewiesenen Aussage das Kapitel, die Art des Belegs im Quelltext und die stützende Prüfung (Tabelle 13.1). Quelltextbelege betreffen das Paket `sl` im Archiv `sharedlib-main`.

Tabelle A.1: Beweisindex.

Aussage	Thema	Quelltext	Prüfung
Satz 5.1	ABI	— (Argument)	—
Satz 5.2	Typbreiten	`codegen`, Parser	P4
Satz 7.1	Namen	`search.py`	P1
Satz 7.2	`SL_PATH`	`search.py`	P1, R1
Satz 7.3	erster Treffer	`search.py`	P5
Satz 7.4	kein Arbeitsverz.	`search.py`	R11
Satz 7.5	gehärtet	`search.py`	—
Satz 8.1, 8.2	Versionen	`version.py`	P2, P2b
Satz 8.3	Rundreise	`container.py`	P3
Satz 8.4	Integrität	`container.py`	P3b
Satz 8.5	Authentizität (negativ)	`container.py`	P3b
Satz 8.6	Totalität	`container.py`	P3c, R6
Satz 8.7	Kanonizität (negativ)	`container.py`	P3d
Satz 8.8	Kanten	`loader.py`	P5
Satz 9.1, 9.2	Terminierung, Zyklen	`loader.py`	P5
Satz 9.3	Referenzzählung	`loader.py`	P5, R5
Satz 9.4	Rollback	`loader.py`	P5
Satz 9.5	Init/Fini	`loader.py`	P5
Satz 9.6, 9.7	Fehler	`loader.py`	P5, R6
Satz 9.8, 9.9	Export, Instanz	`loader.py`	R5
Satz 9.10	Nebenläufigkeit	`loader.py`	R3

Aussage	Thema	Quelltext	Prüfung
Satz 10.1, 10.2, 10.3	Adapter	`loader.py`, `search.py`	P6
Satz 10.4	Aufwand	Adapter	Tab. 13.3
Satz 10.5, 10.6	native Abh., Aufräumen	Adapter	R4, R7
Satz 10.7, 10.8	C-API, Binding	`capi.py`	R10
Satz 11.1, 11.2, 11.3, 11.4, 11.5	Codegenerierung	`codegen`	P4, R9
Satz 12.1, 12.2	Migration, Pool	— (Modell)	—

Anhang B

Prüfskripte

Die Skripte `pruef1.py` bis `pruef7.py` liegen der Arbeit bei. Sie benutzen nur die Standardbibliothek und das untersuchte Paket; Aufruf:

```
python3 -I pruefN.py <Pfad zu sharedlib/src>
```

- `pruef1.py`: Pfadkodierung (P1), Versionen (P2, P2b), Container (P3, P3b, P3c, P3d), ctypes-Zwischenspeicher.
- `pruef2.py`: modellbasierter Loader-Test (P5), Parametrizität (P6), Messung der Öffnungskosten.
- `pruef3.py`: Diamant, doppeltes `close`, Verlust von Handles.
- `pruef4.py`: Signaturparser (P4), native Abhängigkeit, Abdeckung, Testzählung.
- `pruef5.py`: doppeltes `close`, `failed_name`, Verklemmung (R3, R5).
- `pruef6.py`: Codegenerator: Warnungen, Matrix, Ziele (R9).
- `pruef7.py`: Versionssatz Teil (b).

Die Läufe sind mit festen Startwerten wiederholbar; Laufzeiten und Messwerte hängen von der Maschine ab.

Literatur

[1] S. Epp: *Der Subgraph Algorithmus und seine Wiederverwendungen* (`csubgraph`). Wissenschaftliche Arbeit, 2026. `github.com/naphets86/csubgraph`.

[2] S. Epp: *Der Subgraph Algorithmus*. Wissenschaftliche Arbeit, 2026.

[3] S. Epp: *Proposal: Einheitliche Shared Library (`.sl`) mit standardisiertem Ladeverfahren*. Dokument im Repository `github.com/naphets86/sharedlib` (`proposal.md`), 2026.

[4] J. R. Levine: *Linkers and Loaders*. Morgan Kaufmann, 2000.

[5] U. Drepper: *How To Write Shared Libraries*. 2011.

[6] The Open Group: *POSIX.1-2017, `dlopen`, `dlsym`, `dlclose`*. IEEE Std 1003.1.

[7] T. Bray (Hrsg.): *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, 2017.

[8] NIST: *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015.

[9] R. C. Merkle: A Digital Signature Based on a Conventional Encryption Function. In: *CRYPTO '87*, LNCS 293, Springer, 1988, S. 369–378.

[10] G. Klein et al.: seL4: Formal Verification of an OS Kernel. In: *SOSP '09*, ACM, 2009, S. 207–220.

[11] X. Leroy: Formal Verification of a Realistic Compiler. *Communications of the ACM* 52(7), 2009, S. 107–115.

[12] N. Eén, N. Sörensson: An Extensible SAT-solver. In: *SAT 2003*, LNCS 2919, Springer, 2004, S. 502–518.

[13] R. Cox: *Version SAT*. Blogbeitrag, 2016.